

codebase

AXIOM Codebase Documentation

Complete source code for the **AXIOM - Industrial Knowledge Intelligence Platform**.

Total files documented: **38**

Table of Contents

Infrastructure

- [docker-compose.yml](#) - Docker Compose - Infrastructure (Neo4j + Qdrant + Redis + Ollama + Backend)
- [.env.example](#) - Environment Configuration
- [backend/Dockerfile](#) - Backend Docker Image
- [backend/requirements.txt](#) - Python Dependencies

Backend Core

- [backend/app/config.py](#) - Application Configuration (Pydantic Settings)
- [backend/app/main.py](#) - FastAPI Application Entry Point
- [backend/app/models/schemas.py](#) - Pydantic Data Models

API Routers

- [backend/app/routers/ingest.py](#) - Document Ingestion API Router
- [backend/app/routers/query.py](#) - RAG Query API Router
- [backend/app/routers/graph.py](#) - Knowledge Graph API Router
- [backend/app/routers/agents.py](#) - Multi-Agent Intelligence API Router

Document Ingestion Services

- [backend/app/services/ingestion/pdf_parser.py](#) - PDF & DOCX Parser
- [backend/app/services/ingestion/ocr_engine.py](#) - OCR Engine (Tesseract)
- [backend/app/services/ingestion/entity_extractor.py](#) - Industrial Entity Extractor
- [backend/app/services/ingestion/relationship_extractor.py](#) - Relationship Extractor
- [backend/app/services/ingestion/document_classifier.py](#) - Document Classifier
- [backend/app/services/ingestion/chunker.py](#) - Text Chunker
- [backend/app/services/ingestion/llm_extractor.py](#) - LLM Entity Extractor
- [backend/app/services/ingestion/pipeline.py](#) - Main Ingestion Pipeline Orchestrator

Knowledge Graph Services

- [backend/app/services/knowledge_graph/neo4j_client.py](#) - Neo4j Client
- [backend/app/services/knowledge_graph/graph_builder.py](#) - Knowledge Graph Builder

Vector Store Services

- [backend/app/services/vectorstore/qdrant_service.py](#) - Qdrant Vector Store + Embedding Engine

RAG / Retrieval Services

- [backend/app/services/rag/retriever.py](#) - Hybrid Retriever
- [backend/app/services/rag/generator.py](#) - Answer Generator

Multi-Agent System

- [backend/app/services/agents/sensor_monitor_agent.py](#) - Sensor Health Monitor Agent
- [backend/app/services/agents/fault_diagnosis_agent.py](#) - Fault Diagnosis & Suggestion Agent
- [backend/app/services/agents/predictive_maintenance_agent.py](#) - Predictive Maintenance Agent
- [backend/app/services/agents/shift_handover_agent.py](#) - Shift Handover Agent
- [backend/app/services/agents/report_saver.py](#) - Report Persistence Service
- [backend/app/services/agents/orchestrator.py](#) - Agent Orchestrator

Frontend (Next.js)

- [frontend/next.config.js](#) - Next.js Configuration
- [frontend/src/app/layout.js](#) - Root Layout
- [frontend/src/app/globals.css](#) - Global Styles
- [frontend/src/app/page.js](#) - Main Application Page (All 6 Tabs)

Scripts & Demos

- [scripts/generate_synthetic_docs.py](#) - Sample Data Generator
- [scripts/demo_standalone.py](#) - Standalone Pipeline Demo (Zero Dependencies)
- [scripts/demo_agents.py](#) - Multi-Agent Demo (Zero Dependencies)
- [scripts/test_pdf_ingest.py](#) - PDF Ingestion Test

docker-compose.yml

Docker Compose - Infrastructure (Neo4j + Qdrant + Redis + Ollama + Backend)

Defines all 5 containers. One command `docker-compose up -d` starts the entire stack. Ollama provides FREE local LLM, Neo4j stores the knowledge graph, Qdrant stores vector embeddings, Redis handles async task queues.

```

version: "3.9"

services:
  # FREE: Local LLM server (Ollama)
  ollama:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    # After startup, pull models with:
    # docker exec -it axiom-ollama-1 ollama pull llama3.1:8b
    # docker exec -it axiom-ollama-1 ollama pull nomic-embed-text

  # FREE: Graph database
  neo4j:
    image: neo4j:5.22-community
    ports:
      - "7474:7474"
      - "7687:7687"
    environment:
      NEO4J_AUTH: neo4j/axiom_password
      NEO4J_PLUGINS: '["apoc"]'
    volumes:
      - neo4j_data:/data

  # FREE: Vector database
  qdrant:
    image: qdrant/qdrant:v1.11.0
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - qdrant_data:/qdrant/storage

  # FREE: Message broker / cache
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:

```

```

- "8000:8000"
env_file:
- .env
depends_on:
- neo4j
- qdrant
- redis
- ollama
volumes:
- ./data:/app/data

volumes:
neo4j_data:
qdrant_data:
ollama_data:

```

.env.example

Environment Configuration

All settings with defaults. `LLM_PROVIDER=ollama` means zero API costs. Copy to `.env` to use.

```

# LLM Provider: "ollama" (FREE local) or "openai" (paid)
LLM_PROVIDER=ollama

# Ollama (FREE - runs locally)
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_CHAT_MODEL=llama3.1:8b
OLLAMA_EMBEDDING_MODEL=nomic-embed-text

# Local Embedding Model (FREE - sentence-transformers)
LOCAL_EMBEDDING_MODEL=all-MiniLM-L6-v2
EMBEDDING_DIMENSION=384

# OpenAI (OPTIONAL - only if you want to pay for better quality)
# OPENAI_API_KEY=sk-your-key-here
# OPENAI_EMBEDDING_MODEL=text-embedding-3-small
# OPENAI_CHAT_MODEL=gpt-4o

# Neo4j (FREE - Community Edition)
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=axiom_password

# Qdrant (FREE - open source)
QDRANT_HOST=localhost
QDRANT_PORT=6333
QDRANT_COLLECTION=axiom_documents

# Redis (FREE - open source)
REDIS_URL=redis://localhost:6379/0

# App
UPLOAD_DIR=./data/uploads
LOG_LEVEL=INFO

```


backend/Dockerfile

Backend Docker Image

Python 3.11 slim image with Tesseract OCR pre-installed. Installs all Python deps and downloads spaCy English model.

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    tesseract-ocr \
    tesseract-ocr-eng \
    libgl1-mesa-glx \
    libgl2.0-0 \
    poppler-utils \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN python -m spacy download en_core_web_sm

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

backend/requirements.txt

Python Dependencies

All FREE/open-source. sentence-transformers for local embeddings, PyMuPDF for PDF parsing, neo4j/qdrant-client for databases, rank-bm25 for keyword search. OpenAI is optional (commented out).

```
# Core
fastapi==0.115.0
uvicorn[standard]==0.30.6
python-multipart==0.0.9
pydantic==2.9.0
pydantic-settings==2.5.0
python-dotenv==1.0.1

# Document Processing
PyMuPDF==1.24.9
pytesseract==0.3.13
Pillow==10.4.0
python-docx==1.1.2
openpyxl==3.1.5
camelot-py[cv]==0.11.0

# AI/ML - ALL FREE LOCAL MODELS
sentence-transformers==3.1.0 # FREE local embeddings
langchain==0.3.0
langchain-community==0.3.0
langgraph==0.2.0
tiktoken==0.7.0
httpx==0.27.2 # For Ollama API calls
```

```
# OpenAI (OPTIONAL - paid fallback, not required)
# openai==1.45.0
# langchain-openai==0.2.0

# Knowledge Graph (FREE - Neo4j Community)
neo4j==5.24.0

# Vector Store (FREE - Qdrant open source)
qdrant-client==1.11.0

# NLP & Entity Extraction (FREE)
spacy==3.7.6

# Search (FREE)
rank-bm25==0.2.2

# Async (FREE)
celery[redis]==5.4.0
redis==5.0.8

# Utilities
tenacity==9.0.0
structlog==24.4.0
uuid6==2024.7.10
```

backend/app/config.py

Application Configuration (Pydantic Settings)

Loads settings from .env with defaults. Supports two LLM providers: Ollama (free) and OpenAI (paid). Embedding dimension set to 384 for MiniLM model. Chunk size 512 chars with 64 overlap.

```
from pydantic_settings import BaseSettings
from pathlib import Path

class Settings(BaseSettings):
    # LLM Provider: "ollama" (free local) or "openai" (paid)
    llm_provider: str = "ollama"

    # Ollama (FREE - local)
    ollama_base_url: str = "<http://localhost:11434>"
    ollama_chat_model: str = "llama3.1:8b" # or mistral, phi3, gemma2
    ollama_embedding_model: str = "nomic-embed-text" # free embedding model

    # OpenAI (optional fallback - PAID)
    openai_api_key: str = ""
    openai_embedding_model: str = "text-embedding-3-small"
    openai_chat_model: str = "gpt-4o"
```

```

# Local Embeddings (FREE - sentence-transformers)
local_embedding_model: str = "all-MiniLM-L6-v2" # fast, 384 dims
# Alternative: "BAAI/bge-small-en-v1.5" (384 dims, better quality)

# Neo4j (FREE - Community Edition)
neo4j_uri: str = "bolt://localhost:7687"
neo4j_user: str = "neo4j"
neo4j_password: str = "axiom_password"

# Qdrant (FREE - open source, self-hosted)
qdrant_host: str = "localhost"
qdrant_port: int = 6333
qdrant_collection: str = "axiom_documents"

# Redis (FREE - open source)
redis_url: str = "redis://localhost:6379/0"

# App
upload_dir: str = "./data/uploads"
log_level: str = "INFO"

# Embedding dimension (384 for MiniLM/BGE-small, 768 for nomic-embed
-text)
embedding_dimension: int = 384
chunk_size: int = 512
chunk_overlap: int = 64

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

settings = Settings()

# Ensure upload directory exists
Path(settings.upload_dir).mkdir(parents=True, exist_ok=True)

```

backend/app/main.py

FastAPI Application Entry Point

Creates the FastAPI app with lifespan management. On startup: initializes Neo4j client (creates schema) and Qdrant (creates collection). Registers 4 routers: ingest, query, graph, agents. CORS enabled for frontend.

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
import structlog

from app.config import settings
from app.routers import ingest, query, graph, agents
from app.services.knowledge_graph.neo4j_client import Neo4jClient
from app.services.vectorstore.qdrant_service import QdrantService

logger = structlog.get_logger()

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Initialize and cleanup resources on startup/shutdown."""
    logger.info("Starting AXIOM platform...")

    # Initialize Neo4j
    neo4j_client = Neo4jClient()
    await neo4j_client.initialize()
    app.state.neo4j = neo4j_client

    # Initialize Qdrant
    qdrant_service = QdrantService()
    await qdrant_service.initialize()
    app.state.qdrant = qdrant_service

    logger.info("AXIOM platform ready.")
    yield

    # Cleanup
    await neo4j_client.close()
    logger.info("AXIOM platform shut down.")

app = FastAPI(
    title="AXIOM - Industrial Knowledge Intelligence",
    description="AI-powered platform for industrial document ingestion, knowledge graph construction, and intelligent retrieval.",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,

```

```

        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    app.include_router(ingest.router, prefix="/api/v1/ingest", tags=["Ingestion"])
    app.include_router(query.router, prefix="/api/v1/query", tags=["Query"])
    app.include_router(graph.router, prefix="/api/v1/graph", tags=["Knowledge Graph"])
    app.include_router(agents.router, prefix="/api/v1/agents", tags=["Multi-Agent Intelligence"])

@app.get("/health")
async def health_check():
    return {"status": "healthy", "service": "AXIOM"}

```

backend/app/models/schemas.py

Pydantic Data Models

Defines all data structures: DocumentType/Category enums, ExtractedEntity, ExtractedRelationship, DocumentChunk, IngestedDocument, QueryRequest/Response, RetrievedContext, GraphNode/Edge. These are the contracts between all services.

```

from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime
from enum import Enum

# --- Document Models ---

class DocumentType(str, Enum):
    PDF = "pdf"
    DOCX = "docx"
    XLSX = "xlsx"
    IMAGE = "image"
    EMAIL = "email"
    UNKNOWN = "unknown"

class DocumentCategory(str, Enum):
    PID = "p&id"

```

```

WORK_ORDER = "work_order"
SOP = "sop"
INSPECTION_REPORT = "inspection_report"
INCIDENT_REPORT = "incident_report"
OEM_MANUAL = "oem_manual"
REGULATORY = "regulatory"
GENERAL = "general"

```

```

class ExtractedEntity(BaseModel):
    entity_type: str # equipment, personnel, date, parameter, regulation, location
    value: str
    confidence: float = Field(ge=0, le=1)
    source_page: Optional[int] = None
    context: Optional[str] = None

```

```

class ExtractedRelationship(BaseModel):
    source_entity: str
    relationship_type: str
    target_entity: str
    confidence: float = Field(ge=0, le=1)
    source_context: Optional[str] = None

```

```

class DocumentChunk(BaseModel):
    chunk_id: str
    document_id: str
    content: str
    page_number: Optional[int] = None
    chunk_index: int
    metadata: dict = {}

```

```

class IngestedDocument(BaseModel):
    document_id: str
    filename: str
    file_type: DocumentType
    category: DocumentCategory
    total_pages: int = 0
    extracted_text: str
    chunks: list[DocumentChunk] = []
    entities: list[ExtractedEntity] = []
    relationships: list[ExtractedRelationship] = []
    ingested_at: datetime = Field(default_factory=datetime.utcnow)

```

```
# --- Query Models ---

class QueryRequest(BaseModel):
    question: str
    filters: Optional[dict] = None
    top_k: int = Field(default=5, ge=1, le=20)
    include_graph_context: bool = True

class RetrievedContext(BaseModel):
    chunk_id: str
    document_id: str
    filename: str
    content: str
    page_number: Optional[int] = None
    relevance_score: float
    source_type: str # "vector", "graph", "keyword"

class QueryResponse(BaseModel):
    answer: str
    confidence: str # "high", "medium", "low"
    sources: list[RetrievedContext]
    related_entities: list[dict] = []
    suggested_followups: list[str] = []

# --- Graph Models ---

class GraphNode(BaseModel):
    node_id: str
    node_type: str
    properties: dict

class GraphEdge(BaseModel):
    source_id: str
    target_id: str
    relationship: str
    properties: dict = {}
```

backend/app/routers/ingest.py

Document Ingestion API Router

POST /document: Accepts file upload, validates extension (.pdf/.docx/.txt/.json/.csv/images), saves to disk, runs the full IngestionPipeline, indexes chunks in Qdrant, populates Neo4j knowledge graph. POST /batch: Processes multiple files. Returns entities, relationships, chunks count.

```
"""
Document Ingestion API – upload documents and trigger the full processing pipeline.
"""

from fastapi import APIRouter, UploadFile, File, Request, HTTPException
from pathlib import Path
import shutil
import structlog

from app.config import settings
from app.services.ingestion.pipeline import IngestionPipeline
from app.services.knowledge_graph.graph_builder import GraphBuilder
from app.services.vectorstore.qdrant_service import QdrantService
from app.services.knowledge_graph.neo4j_client import Neo4jClient

logger = structlog.get_logger()
router = APIRouter()

@router.post("/document")
async def ingest_document(request: Request, file: UploadFile = File(...)):
    """
    Upload and process a document through the full ingestion pipeline.

    Steps:
    1. Save uploaded file
    2. Parse document (PDF/DOCX/Image)
    3. OCR fallback for scanned pages
    4. Classify document category
    5. Extract entities and relationships
    6. Chunk text for vector storage
    7. Index in vector store (Qdrant)
    8. Populate knowledge graph (Neo4j)

    Returns the full ingestion result with extracted entities and metadata.
    """
    # Validate file type
    allowed_extensions = {".pdf", ".docx", ".doc", ".xlsx", ".png", ".jpg"}
```



```

g", ".jpeg", ".tiff", ".tif", ".txt", ".json", ".csv"}
file_ext = Path(file.filename).suffix.lower()
if file_ext not in allowed_extensions:
    raise HTTPException(
        status_code=400,
        detail=f"Unsupported file type: {file_ext}. Allowed: {allowed_extensions}",
    )

# Save uploaded file
upload_dir = Path(settings.upload_dir)
upload_dir.mkdir(parents=True, exist_ok=True)
file_path = upload_dir / file.filename

try:
    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to save file: {str(e)}")

logger.info("File uploaded", filename=file.filename, path=str(file_path))

# Run ingestion pipeline
try:
    pipeline = IngestionPipeline()
    document = await pipeline.ingest(file_path, filename=file.filename)
except Exception as e:
    logger.error("Ingestion failed", error=str(e), filename=file.filename)
    raise HTTPException(status_code=500, detail=f"Ingestion failed: {str(e)}")

# Index chunks in vector store
try:
    qdrant: QdrantService = request.app.state.qdrant
    await qdrant.index_chunks(document.chunks, document.document_id)
except Exception as e:
    logger.error("Vector indexing failed", error=str(e))

# Populate knowledge graph
try:
    neo4j: Neo4jClient = request.app.state.neo4j
    graph_builder = GraphBuilder(neo4j)

```

```

        await graph_builder.populate_from_document(document)
    except Exception as e:
        logger.error("Graph population failed", error=str(e))

    return {
        "status": "success",
        "document_id": document.document_id,
        "filename": document.filename,
        "file_type": document.file_type.value,
        "category": document.category.value,
        "total_pages": document.total_pages,
        "entities_extracted": len(document.entities),
        "relationships_found": len(document.relationships),
        "chunks_created": len(document.chunks),
        "entities": [
            {
                "type": e.entity_type,
                "value": e.value,
                "confidence": e.confidence,
            }
            for e in document.entities
        ],
        "relationships": [
            {
                "source": r.source_entity,
                "relation": r.relationship_type,
                "target": r.target_entity,
                "confidence": r.confidence,
            }
            for r in document.relationships
        ],
    }

```

```

@router.post("/batch")
async def ingest_batch(request: Request, files: list[UploadFile] = File(
    ...)):
    """Upload and process multiple documents."""
    results = []
    for file in files:
        try:
            # Reuse single document endpoint logic
            result = await ingest_document(request, file)
            results.append(result)
        except HTTPException as e:
            results.append({

```

```

        "status": "failed",
        "filename": file.filename,
        "error": e.detail,
    })
except Exception as e:
    results.append({
        "status": "failed",
        "filename": file.filename,
        "error": str(e),
    })

return {
    "total": len(files),
    "successful": sum(1 for r in results if r.get("status") == "succ
ess"),
    "failed": sum(1 for r in results if r.get("status") == "faile
d"),
    "results": results,
}

```

backend/app/routers/query.py

RAG Query API Router

POST /ask: Takes a question, runs HybridRetriever (vector+graph+BM25), assembles context, generates answer via Ollama LLM with citation enforcement. Returns answer + confidence + sources + follow-ups. POST /search: Returns raw ranked chunks without LLM generation.

```

"""
Query API – handles user questions with hybrid retrieval and LLM answer
generation.
"""

from fastapi import APIRouter, Request, HTTPException
import structlog

from app.models.schemas import QueryRequest, QueryResponse
from app.services.rag.retriever import HybridRetriever
from app.services.rag.generator import AnswerGenerator
from app.services.vectorstore.qdrant_service import QdrantService
from app.services.knowledge_graph.neo4j_client import Neo4jClient

logger = structlog.get_logger()
router = APIRouter()

```

```

@router.post("/ask", response_model=QueryResponse)
async def ask_question(request: Request, query: QueryRequest):
    """
    Ask a question and get an AI-generated answer with source citations.

    Pipeline:
    1. Parse and classify the user query
    2. Hybrid retrieval (vector + graph + keyword)
    3. Context assembly with source metadata
    4. LLM answer generation with citations
    5. Confidence scoring
    6. Follow-up suggestions
    """
    if not query.question.strip():
        raise HTTPException(status_code=400, detail="Question cannot be empty")

    logger.info("Query received", question=query.question[:100])

    # Get services from app state
    qdrant: QdrantService = request.app.state.qdrant
    neo4j: Neo4jClient = request.app.state.neo4j

    # Initialize retriever
    retriever = HybridRetriever(qdrant=qdrant, neo4j=neo4j)

    # Perform hybrid retrieval
    category_filter = query.filters.get("category") if query.filters else None
    retrieved = await retriever.retrieve(
        query=query.question,
        top_k=query.top_k,
        use_graph=query.include_graph_context,
        category_filter=category_filter,
    )

    # Get additional graph context for equipment-specific queries
    graph_context = None
    if query.include_graph_context:
        graph_context = await _get_graph_context(neo4j, query.question)

    # Generate answer
    generator = AnswerGenerator()
    response = await generator.generate(
        query=query.question,

```

```

        retrieved_chunks=retrieved,
        graph_context=graph_context,
    )

    logger.info(
        "Query answered",
        confidence=response.confidence,
        sources=len(response.sources),
    )

    return response

@router.post("/search")
async def search_documents(request: Request, query: QueryRequest):
    """
    Search documents without answer generation – returns ranked chunks.
    Useful for exploring the document corpus.
    """
    qdrant: QdrantService = request.app.state.qdrant
    neo4j: Neo4jClient = request.app.state.neo4j

    retriever = HybridRetriever(qdrant=qdrant, neo4j=neo4j)

    category_filter = query.filters.get("category") if query.filters else None

    retrieved = await retriever.retrieve(
        query=query.question,
        top_k=query.top_k,
        category_filter=category_filter,
    )

    return {
        "query": query.question,
        "results": [
            {
                "chunk_id": r.chunk_id,
                "document_id": r.document_id,
                "filename": r.filename,
                "content": r.content,
                "page_number": r.page_number,
                "relevance_score": r.score,
                "source_type": r.source_type,
            }
            for r in retrieved
        ],
    },

```

```

        "total_results": len(retrieved),
    }

async def _get_graph_context(neo4j: Neo4jClient, question: str) -> dict | None:
    """Extract graph context relevant to the question."""
    import re

    # Look for equipment tags in the question
    equipment_tags = re.findall(r"\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b", question)

    if equipment_tags:
        # Get context for the first equipment tag found
        context = await neo4j.get_equipment_context(equipment_tags[0])
        if context:
            return context

    return None

```

backend/app/routers/graph.py

Knowledge Graph API Router

POST /search: Full-text search across all node types. GET /equipment/{tag}: Returns complete equipment context (work orders, incidents, regulations, inspections). GET /neighbors: Graph traversal. GET /stats: Node/relationship counts.

```

"""
Knowledge Graph API – direct graph exploration and visualization endpoints.
"""

from fastapi import APIRouter, Request, HTTPException
from pydantic import BaseModel
import structlog

from app.services.knowledge_graph.neo4j_client import Neo4jClient

logger = structlog.get_logger()
router = APIRouter()

class GraphSearchRequest(BaseModel):
    query: str

```

```

node_type: str | None = None
depth: int = 2
limit: int = 50

@router.post("/search")
async def search_graph(request: Request, search: GraphSearchRequest):
    """Search the knowledge graph for nodes matching a query."""
    neo4j: Neo4jClient = request.app.state.neo4j
    results = await neo4j.search_nodes(search.query, limit=search.limit)
    return {"results": results, "total": len(results)}

@router.get("/equipment/{tag}")
async def get_equipment(request: Request, tag: str):
    """Get full context for a piece of equipment from the knowledge graph."""
    neo4j: Neo4jClient = request.app.state.neo4j
    context = await neo4j.get_equipment_context(tag.upper())
    if not context:
        raise HTTPException(status_code=404, detail=f"Equipment {tag} not found")
    return context

@router.get("/neighbors/{node_type}/{key}/{value}")
async def get_neighbors(
    request: Request, node_type: str, key: str, value: str, depth: int = 2
):
    """Get neighboring nodes in the knowledge graph."""
    neo4j: Neo4jClient = request.app.state.neo4j
    neighbors = await neo4j.get_neighbors(node_type, key, value, depth)
    return {"node": {"type": node_type, "key": value}, "neighbors": neighbors}

@router.get("/stats")
async def graph_stats(request: Request):
    """Get knowledge graph statistics."""
    neo4j: Neo4jClient = request.app.state.neo4j

    stats_query = """
    CALL {
        MATCH (n) RETURN count(n) as total_nodes
    }
    """

```

```

CALL {
    MATCH ()-[r]->() RETURN count(r) as total_relationships
}
CALL {
    MATCH (n) RETURN labels(n)[0] as label, count(n) as count
    ORDER BY count DESC
}
RETURN total_nodes, total_relationships, collect({label: label, count: count}) as node_types
"""

try:
    results = await neo4j.execute_query(stats_query)
    if results:
        return results[0]
except Exception:
    pass

# Fallback: simpler queries
try:
    nodes = await neo4j.execute_query("MATCH (n) RETURN count(n) as count")
    rels = await neo4j.execute_query("MATCH ()-[r]->() RETURN count(r) as count")
    return {
        "total_nodes": nodes[0]["count"] if nodes else 0,
        "total_relationships": rels[0]["count"] if rels else 0,
    }
except Exception as e:
    return {"total_nodes": 0, "total_relationships": 0, "error": str(e)}

```

backend/app/routers/agents.py

Multi-Agent Intelligence API Router

POST /analyze-sensors: Feeds sensor readings through Monitor→Diagnosis pipeline, auto-saves reports. POST /predict-maintenance: RUL estimation with Weibull+trend. POST /shift-handover: Generates shift report. GET /reports: List/filter saved reports. GET /reports/{cat}/{file}: Retrieve specific report. All POST endpoints auto-save to data/reports/.

```

"""
Agent API Router – exposes the multi-agent system via REST endpoints.
All reports are persisted to data/reports/ with naming convention:
    {equipment}_{shift}_{status}_{timestamp}.json
"""

```



```

from fastapi import APIRouter, Request
from pydantic import BaseModel
from datetime import datetime
from typing import Optional
import structlog

from app.services.agents.orchestrator import AgentOrchestrator
from app.services.agents.sensor_monitor_agent import SensorReading
from app.services.agents.report_saver import ReportSaver

logger = structlog.get_logger()
router = APIRouter()
report_saver = ReportSaver()

class SensorDataPoint(BaseModel):
    equipment_tag: str
    parameter: str
    value: float
    unit: str
    timestamp: Optional[datetime] = None

class SensorBatchRequest(BaseModel):
    readings: list[SensorDataPoint]

class PredictiveRequest(BaseModel):
    equipment_tag: str
    parameter_history: dict[str, list[float]]
    critical_limits: dict[str, float]
    equipment_type: str = "pump"
    hours_in_service: float = 10000
    next_pm_date: Optional[datetime] = None

class HandoverRequest(BaseModel):
    shift_start: datetime
    shift_end: datetime

@router.post("/analyze-sensors")
async def analyze_sensors(request: Request, data: SensorBatchRequest):
    """
    Process sensor readings through the multi-agent pipeline.

```

Flow:

1. Sensor Monitor Agent checks all readings for anomalies
2. Fault Diagnosis Agent analyzes anomalies (root cause + suggestions)
3. Events logged for shift handover

Returns alerts, diagnoses, and health summary.

"""

```
neo4j = getattr(request.app.state, "neo4j", None)
orchestrator = AgentOrchestrator(neo4j_client=neo4j)

# Convert to internal format
readings = [
    SensorReading(
        equipment_tag=r.equipment_tag,
        parameter=r.parameter,
        value=r.value,
        unit=r.unit,
        timestamp=r.timestamp or datetime.utcnow(),
    )
    for r in data.readings
]

result = await orchestrator.process_sensor_batch(readings)

response_data = {
    "total_readings": len(readings),
    "anomalies_detected": len(result.alerts),
    "diagnoses_generated": len(result.diagnoses),
    "actions_suggested": result.actions_generated,
    "alerts": [
        {
            "equipment_tag": a.equipment_tag,
            "parameter": a.parameter,
            "value": a.current_value,
            "unit": a.unit,
            "status": a.status.value,
            "severity_score": a.severity_score,
            "reason": a.reason,
            "trend": a.trend_info,
        }
        for a in result.alerts
    ],
    "diagnoses": [
        {
```

```

        "equipment_tag": d.equipment_tag,
        "alert_summary": d.alert_summary,
        "confidence": d.confidence_level,
        "hypotheses": [
            {
                "cause": h.cause,
                "confidence": h.confidence,
                "category": h.category,
                "evidence": h.evidence,
            }
            for h in d.hypotheses
        ],
        "recommended_actions": [
            {
                "action": a.action,
                "priority": a.priority,
                "effort": a.estimated_effort,
                "parts_needed": a.parts_needed,
                "sop_reference": a.sop_reference,
            }
            for a in d.recommended_actions
        ],
    }
    for d in result.diagnoses
],
"health_summary": result.health_summary,
}

# Save sensor analysis report
equipment_tags = list(set(r.equipment_tag for r in data.readings))
sensor_report_path = report_saver.save_sensor_analysis(
    report_data=response_data,
    equipment_tags=equipment_tags,
    alerts=result.alerts,
)
response_data["report_saved_to"] = sensor_report_path

# Save individual diagnosis reports
diagnosis_report_paths = []
for d in result.diagnoses:
    diag_data = {
        "equipment_tag": d.equipment_tag,
        "alert_summary": d.alert_summary,
        "confidence": d.confidence_level,
        "hypotheses": [
            {"cause": h.cause, "confidence": h.confidence, "category": h.category, "evidence": h.evidence}
            for h in d.hypotheses
        ],
        "recommended_actions": [
            {"action": a.action, "priority": a.priority, "effort": a.estimated_effort, "parts_needed": a.parts_needed, "sop_reference": a.sop_reference}
            for a in d.recommended_actions
        ],
    }
    diagnosis_report_paths.append(report_saver.save_diagnosis_report(diag_data, d.equipment_tag))

```

```

y": h.category, "evidence": h.evidence}
        for h in d.hypotheses
    ],
    "recommended_actions": [
        {"action": a.action, "priority": a.priority, "effort":
a.estimated_effort,
        "parts_needed": a.parts_needed, "sop_reference": a.sop_
reference}
        for a in d.recommended_actions
    ],
    "similar_past_incidents": d.similar_past_incidents,
}
path = report_saver.save_fault_diagnosis(
    report_data=diag_data,
    equipment_tag=d.equipment_tag,
    confidence_level=d.confidence_level,
)
diagnosis_report_paths.append(path)
response_data["diagnosis_reports_saved_to"] = diagnosis_report_paths

return response_data

```

```

@router.post("/predict-maintenance")
async def predict_maintenance(request: Request, data: PredictiveReques
t):
    """
    Run predictive maintenance analysis for specific equipment.
    Returns Remaining Useful Life estimate and maintenance recommendatio
ns.
    """
    neo4j = getattr(request.app.state, "neo4j", None)
    orchestrator = AgentOrchestrator(neo4j_client=neo4j)

    report = await orchestrator.get_predictive_report(
        equipment_tag=data.equipment_tag,
        parameter_history=data.parameter_history,
        critical_limits=data.critical_limits,
        equipment_type=data.equipment_type,
        hours_in_service=data.hours_in_service,
        next_pm=data.next_pm_date,
    )

    response_data = {
        "equipment_tag": report.equipment_tag,
        "overall_health_score": report.overall_health_score,

```

```

        "risk_summary": report.risk_summary,
        "rul_estimates": [
            {
                "parameter": r.parameter,
                "days_to_failure": r.estimated_days_to_failure,
                "confidence_interval": r.confidence_interval,
                "risk_score": r.risk_score,
                "method": r.method,
                "health_pct": r.current_health_pct,
            }
            for r in report.rul_estimates
        ],
        "recommendations": [
            {
                "action": r.action,
                "recommended_date": r.recommended_date.isoformat(),
                "urgency": r.urgency,
                "cost_early_intervention": r.cost_of_early_intervention,
                "cost_unplanned_failure": r.cost_of_unplanned_failure,
                "reasoning": r.reasoning,
            }
            for r in report.maintenance_recommendations
        ],
    }

    # Save predictive report
    pred_report_path = report_saver.save_predictive_report(
        report_data=response_data,
        equipment_tag=data.equipment_tag,
        risk_level=report.risk_summary,
    )
    response_data["report_saved_to"] = pred_report_path

    return response_data

```

```

@router.post("/shift-handover")
async def generate_handover(request: Request, data: HandoverRequest):
    """
    Generate a shift handover report for the specified time window.
    Summarizes all events, alarms, and pending items.
    """
    neo4j = getattr(request.app.state, "neo4j", None)
    orchestrator = AgentOrchestrator(neo4j_client=neo4j)

    report, formatted_text = await orchestrator.generate_shift_handover(

```

```

        shift_start=data.shift_start,
        shift_end=data.shift_end,
    )

    response_data = {
        "shift": {
            "type": report.shift_type,
            "start": report.shift_start.isoformat(),
            "end": report.shift_end.isoformat(),
        },
        "summary": report.executive_summary,
        "stats": {
            "total_alarms": report.total_alarms,
            "critical_alarms": report.critical_alarms,
            "maintenance_actions": report.maintenance_actions,
        },
        "critical_items": report.critical_items,
        "work_in_progress": report.work_in_progress,
        "watch_items": report.watch_items,
        "upcoming": report.upcoming_activities,
        "formatted_report": formatted_text,
    }

    # Collect equipment tags from critical items and health changes
    equipment_tags = list(set(
        item.get("equipment", "")
        for item in report.critical_items + report.equipment_health_changes + report.work_in_progress
        if item.get("equipment")
    ))

    # Save shift handover report
    handover_path = report_saver.save_shift_handover(
        report_data=response_data,
        shift_type=report.shift_type,
        has_critical=report.critical_alarms > 0,
        shift_start=report.shift_start,
        equipment_tags=equipment_tags,
    )
    response_data["report_saved_to"] = handover_path

    return response_data

@router.get("/reports")
async def list_reports(

```

```

category: Optional[str] = None,
equipment: Optional[str] = None,
status: Optional[str] = None,
):
    """
    List all saved reports. Filter by category, equipment tag, or status.

    Categories: sensor_analysis, fault_diagnosis, predictive, shift_handover
    Status in filename: NORMAL, WARNING, CRITICAL, HIGH-RISK, LOW-RISK, etc.
    """
    from pathlib import Path
    import json

    base = report_saver.base_dir
    results = []

    # Determine which subdirs to scan
    if category and (base / category).is_dir():
        subdirs = [base / category]
    else:
        subdirs = [d for d in base.iterdir() if d.is_dir()]

    for subdir in subdirs:
        for filepath in sorted(subdir.glob("*.json"), reverse=True):
            name = filepath.stem # e.g. P-101A_day_CRITICAL_20240701_14
            parts = name.split("_")

            # Apply filters
            if equipment and equipment.upper() not in name.upper():
                continue
            if status and status.upper() not in name.upper():
                continue

            # Parse naming convention
            file_info = {
                "filename": filepath.name,
                "category": subdir.name,
                "path": str(filepath),
                "size_bytes": filepath.stat().st_size,
                "created": datetime.fromtimestamp(filepath.stat().st_ctime).isoformat(),
            }

```

```

        # Extract equipment/shift/status from filename parts
        if len(parts) >= 4:
            # Last two parts are date_time, work backwards
            file_info["timestamp"] = f"{parts[-2]}_{parts[-1]}"
            file_info["status_label"] = parts[-3] if len(parts) >= 4
        else ""

            file_info["shift"] = parts[-4] if len(parts) >= 5 else ""

            file_info["equipment"] = "_".join(parts[:-4]) if len(parts) > 4 else parts[0]

        results.append(file_info)

    return {
        "total_reports": len(results),
        "reports_directory": str(base),
        "reports": results[:100], # Limit to 100 most recent
    }

@router.get("/reports/{category}/{filename}")
async def get_report(category: str, filename: str):
    """Retrieve a specific saved report by category and filename."""
    from pathlib import Path
    import json

    filepath = report_saver.base_dir / category / filename
    if not filepath.exists() or not filepath.is_file():
        from fastapi import HTTPException
        raise HTTPException(status_code=404, detail=f"Report not found: {category}/{filename}")

    # Prevent path traversal
    if ".." in filename or "/" in filename or "\\\" in filename:
        from fastapi import HTTPException
        raise HTTPException(status_code=400, detail="Invalid filename")

    with open(filepath, "r", encoding="utf-8") as f:
        data = json.load(f)

    return {
        "filename": filename,
        "category": category,
        "data": data,
    }

```


backend/app/services/ingestion/pdf_parser.py

PDF & DOCX Parser

PDFParser: Uses PyMuPDF (fitz) to extract text with layout preservation, tables via built-in table finder, and images for OCR. Handles CMYK→RGB conversion. DOCXParser: Uses python-docx for paragraphs and tables. Both return ParsedDocument with per-page ParsedPage objects.

```
"""
PDF and DOCX document parser with layout-aware text extraction.
Uses PyMuPDF for PDFs and python-docx for Word documents.
"""

import fitz  # PyMuPDF
from pathlib import Path
from dataclasses import dataclass
import structlog

logger = structlog.get_logger()

@dataclass
class ParsedPage:
    page_number: int
    text: str
    tables: list[list[list[str]]]
    images: list[bytes]
    has_text: bool

@dataclass
class ParsedDocument:
    filename: str
    total_pages: int
    pages: list[ParsedPage]
    metadata: dict

class PDFParser:
    """Extracts text, tables, and images from PDF documents."""

    def parse(self, file_path: str | Path) -> ParsedDocument:
        file_path = Path(file_path)
        logger.info("Parsing PDF", filename=file_path.name)
```

```

doc = fitz.open(str(file_path))
pages = []

for page_num in range(len(doc)):
    page = doc[page_num]

    # Extract text with layout preservation
    text = page.get_text("text")

    # Extract tables using built-in table finder
    tables = self._extract_tables(page)

    # Extract images (for OCR fallback or P&ID processing)
    images = self._extract_images(page)

    has_text = len(text.strip()) > 50

    pages.append(ParsedPage(
        page_number=page_num + 1,
        text=text,
        tables=tables,
        images=images,
        has_text=has_text,
    ))

metadata = doc.metadata or {}
doc.close()

logger.info(
    "PDF parsed",
    filename=file_path.name,
    pages=len(pages),
    pages_with_text=sum(1 for p in pages if p.has_text),
)

return ParsedDocument(
    filename=file_path.name,
    total_pages=len(pages),
    pages=pages,
    metadata=metadata,
)

def _extract_tables(self, page: fitz.Page) -> list[list[list[str]]]:
    """Extract tables from a PDF page using PyMuPDF's table finder.
    """
    tables = []

```

```

try:
    tab_finder = page.find_tables()
    for table in tab_finder:
        extracted = table.extract()
        if extracted:
            tables.append(extracted)
except Exception as e:
    logger.warning("Table extraction failed", error=str(e))
return tables

def _extract_images(self, page: fitz.Page) -> list[bytes]:
    """Extract images from a PDF page for OCR processing."""
    images = []
    try:
        image_list = page.get_images(full=True)
        doc = page.parent
        for img_info in image_list:
            xref = img_info[0]
            pix = fitz.Pixmap(doc, xref)
            if pix.n - pix.alpha > 3: # CMYK → RGB
                pix = fitz.Pixmap(fitz.csRGB, pix)
            images.append(pix.tobytes("png"))
            pix = None
    except Exception as e:
        logger.warning("Image extraction failed", error=str(e))
    return images

class DOCXParser:
    """Extracts text and tables from Word documents."""

    def parse(self, file_path: str | Path) -> ParsedDocument:
        from docx import Document

        file_path = Path(file_path)
        logger.info("Parsing DOCX", filename=file_path.name)

        doc = Document(str(file_path))

        # Extract all paragraph text
        full_text = []
        for para in doc.paragraphs:
            if para.text.strip():
                full_text.append(para.text)

        # Extract tables

```

```

tables = []
for table in doc.tables:
    table_data = []
    for row in table.rows:
        row_data = [cell.text.strip() for cell in row.cells]
        table_data.append(row_data)
    tables.append(table_data)

text = "\n".join(full_text)

page = ParsedPage(
    page_number=1,
    text=text,
    tables=tables,
    images=[],
    has_text=len(text.strip()) > 50,
)

return ParsedDocument(
    filename=file_path.name,
    total_pages=1,
    pages=[page],
    metadata={"format": "docx"},
)

```

backend/app/services/ingestion/ocr_engine.py

OCR Engine (Tesseract)

Gracefully handles missing pytesseract/PIL (HAS_OCR flag). Preprocessing pipeline: grayscale → resize if small → contrast enhance 1.5x → sharpen → binarize at threshold 140. Returns OCRResult with text + confidence (0-1). needs_ocr() checks if direct text extraction failed (<50 chars or <80% printable).

```

"""
OCR Engine for scanned documents and images.
Uses Tesseract via pytesseract with image preprocessing for better accuracy.
"""

from __future__ import annotations
import io
from pathlib import Path
from dataclasses import dataclass
import structlog

```

```

try:
    import pytesseract
    from PIL import Image, ImageFilter, ImageEnhance
    HAS_OCR = True
except ImportError:
    HAS_OCR = False

logger = structlog.get_logger()

@dataclass
class OCRResult:
    text: str
    confidence: float
    page_number: int

class OCREngine:
    """Performs OCR on images and scanned PDF pages."""

    def __init__(self, lang: str = "eng"):
        self.lang = lang

    def ocr_image_bytes(self, image_bytes: bytes, page_number: int = 1)
-> OCRResult:
        """Run OCR on raw image bytes."""
        if not HAS_OCR:
            return OCRResult(text="", confidence=0.0, page_number=page_n
umber)
        image = Image.open(io.BytesIO(image_bytes))
        return self._process_image(image, page_number)

    def ocr_image_file(self, file_path: str | Path, page_number: int =
1) -> OCRResult:
        """Run OCR on an image file."""
        if not HAS_OCR:
            return OCRResult(text="", confidence=0.0, page_number=page_n
umber)
        image = Image.open(str(file_path))
        return self._process_image(image, page_number)

    def ocr_pdf_page_image(self, page_pixmap_bytes: bytes, page_number:
int) -> OCRResult:
        """Run OCR on a rendered PDF page (for scanned PDFs)."""
        if not HAS_OCR:
            return OCRResult(text="", confidence=0.0, page_number=page_n

```

```

umber)
    image = Image.open(io.BytesIO(page_pixmap_bytes))
    return self._process_image(image, page_number)

def _process_image(self, image, page_number: int) -> OCRResult:
    """Preprocess and run OCR on a PIL Image."""
    if not HAS_OCR:
        return OCRResult(text="", confidence=0.0, page_number=page_n
umber)

    # Preprocessing pipeline for better OCR accuracy
    processed = self._preprocess(image)

    # Run OCR with confidence data
    ocr_data = pytesseract.image_to_data(
        processed, lang=self.lang, output_type=pytesseract.Output.DI
CT
    )

    # Extract text and compute average confidence
    text_parts = []
    confidences = []

    for i, word in enumerate(ocr_data["text"]):
        conf = int(ocr_data["conf"][i])
        if conf > 0 and word.strip():
            text_parts.append(word)
            confidences.append(conf)

    text = " ".join(text_parts)
    avg_confidence = sum(confidences) / len(confidences) if confiden
ces else 0.0

    logger.info(
        "OCR completed",
        page=page_number,
        chars=len(text),
        confidence=f"{avg_confidence:.1f}%",
    )

    return OCRResult(
        text=text,
        confidence=avg_confidence / 100.0,
        page_number=page_number,
    )

def _preprocess(self, image: Image.Image) -> Image.Image:

```

```

"""Apply preprocessing steps to improve OCR accuracy."""
# Convert to grayscale
if image.mode != "L":
    image = image.convert("L")

# Resize if too small (improves OCR on low-res scans)
width, height = image.size
if width < 1000:
    scale = 1500 / width
    image = image.resize(
        (int(width * scale), int(height * scale)), Image.LANCZOS
    )

# Enhance contrast
enhancer = ImageEnhance.Contrast(image)
image = enhancer.enhance(1.5)

# Sharpen
image = image.filter(ImageFilter.SHARPEN)

# Binarize (adaptive threshold via point operation)
threshold = 140
image = image.point(lambda x: 255 if x > threshold else 0, "1")

return image

def needs_ocr(self, text: str, page_image_bytes: bytes | None = None) -> bool:
    """Determine if a page needs OCR (too little extractable text)."""
    # If we got very little text from direct extraction, likely scanned
    if len(text.strip()) < 50:
        return True

    # If text has too many garbage characters, likely bad extraction
    printable_ratio = sum(1 for c in text if c.isprintable()) / max(
        len(text), 1
    )
    if printable_ratio < 0.8:
        return True

    return False

```

backend/app/services/ingestion/entity_extractor.py

Industrial Entity Extractor

Multi-strategy extraction: (1) Regex patterns for equipment tags (XX-NNNN ISA format), process parameters (value+unit), regulations (OISD/API/ISO/ASME), dates, document references (WO/MOC/PTW). (2) spaCy NER for people/orgs/locations. False-positive filtering removes months (JAN/FEB) and standard prefixes (OISD/API) from equipment matches. Deduplicates keeping highest confidence.

```
"""
Industrial Entity Extraction Engine.
Combines regex patterns (for structured tags) with spaCy NER and LLM-based
extraction
for equipment tags, personnel, dates, parameters, and regulatory references.
"""

import re
from dataclasses import dataclass, field
from typing import Optional
import structlog

logger = structlog.get_logger()

@dataclass
class Entity:
    entity_type: str
    value: str
    confidence: float
    start: int = 0
    end: int = 0
    context: str = ""

class IndustrialEntityExtractor:
    """
    Multi-strategy entity extraction for industrial documents.

    Strategy 1: Regex patterns for well-structured tags (equipment IDs,
    parameters)
    Strategy 2: spaCy NER for people, organizations, dates
    Strategy 3: LLM extraction for complex/ambiguous entities
    """

    # Month abbreviations to exclude from equipment tag matching
    _MONTHS = {"JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG",
    "SEP", "OCT", "NOV", "DEC"}
    # Common prefixes that are NOT equipment (standards, document refs)
```



```

_NOT_EQUIPMENT = {"SOP", "WO", "MOC", "PTW", "NCR", "IR", "OR", "IS", "OEM"}

# Industrial equipment tag patterns (ISA standard formats)
EQUIPMENT_PATTERNS = [
    # Standard tag format: XX-NNNA (e.g., P-101A, V-201B, C-301)
    r"\b([A-Z]{1,4})-(\d{2,5}[A-Z]?)\b",
    # Extended format: XX-NNNN-NNA (e.g., FV-1001-01A)
    r"\b([A-Z]{1,4})-(\d{3,5})-(\d{2}[A-Z]?)\b",
    # Instrument tags: XXX-NNNN (e.g., TIT-1001, FIC-2001, PSV-3001)
    r"\b([A-Z]{2,4})-(\d{4,5})\b",
    # Named equipment: "Pump 101A", "Compressor C-201"
    r"\b(Pump|Compressor|Valve|Reactor|Vessel|Tank|Heat Exchanger|Boiler|Turbine|Motor|Fan|Blower)\s+([A-Z]?-\d{2,5}[A-Z]?)\b",
]

# Process parameter patterns
PARAMETER_PATTERNS = [
    # Temperature: 150°C, 300 deg F
    r"(\d+\.\d*)\s*(?:°C|°F|deg\s*[CF]|celsius|fahrenheit)",
    # Pressure: 10.5 bar, 150 psi, 1.2 MPa
    r"(\d+\.\d*)\s*(?:bar|psi|MPa|kPa|kg/cm2|atm)",
    # Flow: 100 m3/h, 500 GPM, 1000 kg/hr
    r"(\d+\.\d*)\s*(?:m3/h|GPM|kg/hr|l/min|SCFM|Nm3/h)",
    # Vibration: 2.5 mm/s, 0.1 in/s
    r"(\d+\.\d*)\s*(?:mm/s|in/s|mils|µm)",
]

# Regulatory reference patterns
REGULATION_PATTERNS = [
    # OISD standards
    r"\b(OISD[-\s]?\d{3})\b",
    # IS/BIS standards
    r"\b(IS[-:\s]?\d{3,5}(?:[-\s]?Part[-\s]?\d+))\b",
    # API standards
    r"\b(API[-\s]?\d{3,4}[A-Z]?)\b",
    # ASME codes
    r"\b(ASME[-\s]?[A-Z]+[-\s]?\d+(?:\.\d+)?)\b",
    # Factory Act reference
    r"\b(Factory Act|Factories Act)(?:\s*(?:Section|Sec\.)\s*(\d+[A-Z]?))?",
    # PESO regulations
    r"\b(PESO|SMPV|Gas Cylinder Rules)\b",
    # ISO standards
    r"\b(ISO[-\s]?\d{4,5}(?:[-:]?\d+))\b",
]

```

```

# Date patterns common in industrial documents
DATE_PATTERNS = [
    r"\b(\d{1,2}[-/]\d{1,2}[-/]\d{2,4})\b",
    r"\b(\d{1,2}\s+(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\s+\d{2,4})\b",
    r"\b((?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\s+\d{1,2},?\s+\d{2,4})\b",
]

# Work order / document reference patterns
DOCUMENT_REF_PATTERNS = [
    r"\b(WO[-\s]?\d{5,10})\b", # Work orders
    r"\b(MOC[-\s]?\d{4,8})\b", # Management of Change
    r"\b(PTW[-\s]?\d{4,8})\b", # Permit to Work
    r"\b(NCR[-\s]?\d{4,8})\b", # Non-conformance report
    r"\b(IR[-\s]?\d{4,8})\b", # Incident report
]

def __init__(self):
    self._nlp = None

@property
def nlp(self):
    """Lazy-load spaCy model."""
    if self._nlp is None:
        try:
            import spacy
            try:
                self._nlp = spacy.load("en_core_web_sm")
            except OSError:
                self._nlp = spacy.blank("en")
        except ImportError:
            logger.warning("spaCy not installed, NER disabled")
            self._nlp = None
    return self._nlp

def extract_all(self, text: str, page_number: Optional[int] = None)
-> list[Entity]:
    """Run all extraction strategies and merge results."""
    entities = []

    # Strategy 1: Regex-based extraction (high precision for structured tags)
    entities.extend(self._extract_equipment_tags(text, page_number))
    entities.extend(self._extract_parameters(text, page_number))

```

```

        entities.extend(self._extract_regulations(text, page_number))
        entities.extend(self._extract_dates(text, page_number))
        entities.extend(self._extract_document_refs(text, page_number))

        # Strategy 2: spaCy NER (people, organizations, locations)
        entities.extend(self._extract_spacy_entities(text, page_number))

        # Deduplicate
        entities = self._deduplicate(entities)

        logger.info(
            "Entity extraction complete",
            total_entities=len(entities),
            page=page_number,
        )
        return entities

    def _extract_equipment_tags(self, text: str, page: Optional[int]) ->
list[Entity]:
        """Extract industrial equipment tags, filtering out false posi-
ves."""
        entities = []
        for pattern in self.EQUIPMENT_PATTERNS:
            for match in re.finditer(pattern, text, re.IGNORECASE):
                value = match.group(0).strip().upper()
                # Filter out date-like patterns (MAR-2024, OCT-2023)
                prefix = value.split("-")[0] if "-" in value else ""
                if prefix in self._MONTHS:
                    continue
                # Filter out known non-equipment prefixes
                if prefix in self._NOT_EQUIPMENT:
                    continue
                # Filter out regulation-like patterns (OISD-154, API-61
0, ISO-10816)
                if prefix in {"OISD", "API", "ISO", "ASME", "ASTM"}:
                    continue

                context = text[max(0, match.start() - 50):match.end() +
50]

                entities.append(Entity(
                    entity_type="equipment",
                    value=value,
                    confidence=0.9,
                    start=match.start(),
                    end=match.end(),
                    context=context,

```

```

        ))
    return entities

    def _extract_parameters(self, text: str, page: Optional[int]) -> list[Entity]:
        """Extract process parameters with units."""
        entities = []
        for pattern in self.PARAMETER_PATTERNS:
            for match in re.finditer(pattern, text, re.IGNORECASE):
                value = match.group(0).strip()
                context = text[max(0, match.start() - 50):match.end() +
50]

                entities.append(Entity(
                    entity_type="parameter",
                    value=value,
                    confidence=0.85,
                    start=match.start(),
                    end=match.end(),
                    context=context,
                ))
        return entities

    def _extract_regulations(self, text: str, page: Optional[int]) -> list[Entity]:
        """Extract regulatory standard references."""
        entities = []
        for pattern in self.REGULATION_PATTERNS:
            for match in re.finditer(pattern, text):
                value = match.group(0).strip()
                context = text[max(0, match.start() - 50):match.end() +
50]

                entities.append(Entity(
                    entity_type="regulation",
                    value=value,
                    confidence=0.95,
                    start=match.start(),
                    end=match.end(),
                    context=context,
                ))
        return entities

    def _extract_dates(self, text: str, page: Optional[int]) -> list[Entity]:
        """Extract dates from the document."""
        entities = []
        for pattern in self.DATE_PATTERNS:

```

```

        for match in re.finditer(pattern, text, re.IGNORECASE):
            value = match.group(1).strip()
            context = text[max(0, match.start() - 40):match.end() +
40]

            entities.append(Entity(
                entity_type="date",
                value=value,
                confidence=0.85,
                start=match.start(),
                end=match.end(),
                context=context,
            ))
        return entities

    def _extract_document_refs(self, text: str, page: Optional[int]) ->
list[Entity]:
        """Extract document/work order references."""
        entities = []
        for pattern in self.DOCUMENT_REF_PATTERNS:
            for match in re.finditer(pattern, text):
                value = match.group(0).strip()
                context = text[max(0, match.start() - 40):match.end() +
40]

                entities.append(Entity(
                    entity_type="document_reference",
                    value=value,
                    confidence=0.9,
                    start=match.start(),
                    end=match.end(),
                    context=context,
                ))
            return entities

    def _extract_spacy_entities(self, text: str, page: Optional[int]) ->
list[Entity]:
        """Extract named entities using spaCy (people, orgs, location
s)."""
        entities = []
        if self.nlp is None:
            return entities
        # Limit text length to avoid spaCy memory issues
        truncated = text[:100000] if len(text) > 100000 else text
        doc = self.nlp(truncated)

        type_map = {
            "PERSON": "personnel",

```

```

        "ORG": "organization",
        "GPE": "location",
        "FAC": "facility",
        "DATE": "date",
    }

    for ent in doc.ents:
        if ent.label_ in type_map:
            entities.append(Entity(
                entity_type=type_map[ent.label_],
                value=ent.text,
                confidence=0.75,
                start=ent.start_char,
                end=ent.end_char,
                context=text[max(0, ent.start_char - 30):ent.end_cha
r + 30],
            ))
    return entities

def _deduplicate(self, entities: list[Entity]) -> list[Entity]:
    """Remove duplicate entities, keeping highest confidence."""
    seen = {}
    for entity in entities:
        key = (entity.entity_type, entity.value.lower())
        if key not in seen or entity.confidence > seen[key].confiden
ce:
            seen[key] = entity
    return list(seen.values())

```

backend/app/services/ingestion/relationship_extractor.py

Relationship Extractor

Two strategies: (1) Regex patterns match 'X failed due to Y', 'X as per SOP-Y', 'X protecting reactor Y'. (2) Co-occurrence analysis finds equipment+regulation in same sentence → GOVERNED_BY, equipment+equipment → CONNECTED_TO. Returns ExtractedRelation with source, relation type, target, confidence.

```

"""
Relationship extraction – identifies connections between extracted entities
using pattern matching and co-occurrence analysis.
"""

import re
from dataclasses import dataclass

```

```

from typing import Optional
import structlog

logger = structlog.get_logger()

@dataclass
class ExtractedRelation:
    source: str
    relation: str
    target: str
    confidence: float
    context: str = ""

class RelationshipExtractor:
    """
    Extracts relationships between entities based on:
    1. Proximity co-occurrence (entities mentioned close together)
    2. Syntactic patterns (e.g., "pump P-101A failed due to bearing wear")
    3. Structural patterns (e.g., entities in same table row)
    """

    # Relationship trigger patterns
    RELATION_PATTERNS = [
        # Equipment → Failure
        (
            r"(\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b).*?(?:failed|tripped|malfunctioned|broke down).*?(?:due to|because of|caused by)\s+(.+?)(?:\.|$)",
            "FAILED_DUE_TO",
        ),
        # Equipment → Maintenance action
        (
            r"(\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b).*?(?:repaired|replaced|overhauled|serviced|maintained)",
            "MAINTAINED",
        ),
        # Equipment → Inspection
        (
            r"(\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b).*?(?:inspected|surveyed|examined|tested)",
            "INSPECTED",
        ),
        # Equipment → Procedure
        (

```

```

        r"(?:as per|refer|according to|per)\s+(SOP|procedure|WI)[-
\s]?(\S+)",
        "GOVERNED_BY",
    ),
    # Personnel → Action on Equipment
    (
        r"(\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+\b).*?(?:performed|execut
ed|carried out|completed).*?(\b[A-Z]{1,4}-\d{2,5}[A-Z]?[a-z])",
        "PERFORMED_ON",
    ),
    # Equipment → Location
    (
        r"(\b[A-Z]{1,4}-\d{2,5}[A-Z]?[a-z]).*?(?:located at|installed i
n|at)\s+(.+?)(?:\s|,|$)",
        "LOCATED_AT",
    ),
]

```

```

def extract(self, text: str, entities: list) -> list[ExtractedRelati
on]:

```

```

    """Extract relationships between entities in the text."""
    relations = []

```

```

    # Strategy 1: Pattern-based extraction
    relations.extend(self._pattern_based_extraction(text))

```

```

    # Strategy 2: Co-occurrence (entities in same sentence)
    relations.extend(self._cooccurrence_extraction(text, entities))

```

```

    # Deduplicate
    seen = set()
    unique_relations = []
    for r in relations:
        key = (r.source, r.relation, r.target)
        if key not in seen:
            seen.add(key)
            unique_relations.append(r)

```

```

    logger.info("Relationships extracted", count=len(unique_relation
s))
    return unique_relations

```

```

def _pattern_based_extraction(self, text: str) -> list[ExtractedRela
tion]:

```

```

    """Extract relationships using regex patterns."""
    relations = []

```



```

        for pattern, rel_type in self.RELATION_PATTERNS:
            for match in re.finditer(pattern, text, re.IGNORECASE | re.D
OTALL):
                groups = match.groups()
                if len(groups) >= 2:
                    source = groups[0].strip()
                    target = groups[-1].strip()[:100] # Limit target le
ngth
                    context = text[max(0, match.start() - 30):match.end
() + 30]

                    relations.append(ExtractedRelation(
                        source=source,
                        relation=rel_type,
                        target=target,
                        confidence=0.8,
                        context=context,
                    ))

        return relations

    def _cooccurrence_extraction(self, text: str, entities: list) -> lis
t[ExtractedRelation]:
        """Find relationships based on entity co-occurrence in sentence
s."""
        relations = []

        # Split into sentences
        sentences = re.split(r"[.!?]\s+", text)

        # Group entities by type
        equipment_entities = [e for e in entities if e.entity_type == "e
quipment"]
        other_entities = [e for e in entities if e.entity_type != "equip
ment"]

        for sentence in sentences:
            # Find equipment and other entities in same sentence
            equip_in_sentence = [
                e for e in equipment_entities if e.value.lower() in sent
ence.lower()
            ]
            others_in_sentence = [
                e for e in other_entities if e.value.lower() in sentenc
e.lower()
            ]

```

```

    ]

    # Create co-occurrence relations
    for equip in equip_in_sentence:
        for other in others_in_sentence:
            rel_type = self._infer_relation_type(equip.entity_type, other.entity_type)
            if rel_type:
                relations.append(ExtractedRelation(
                    source=equip.value,
                    relation=rel_type,
                    target=other.value,
                    confidence=0.6,
                    context=sentence[:150],
                ))

    # Equipment → Equipment relations (in same sentence implies connection)
    if len(equip_in_sentence) > 1:
        for i, eq1 in enumerate(equip_in_sentence):
            for eq2 in equip_in_sentence[i + 1:]:
                relations.append(ExtractedRelation(
                    source=eq1.value,
                    relation="CONNECTED_TO",
                    target=eq2.value,
                    confidence=0.5,
                    context=sentence[:150],
                ))

    return relations

def _infer_relation_type(self, source_type: str, target_type: str) -
> Optional[str]:
    """Infer relationship type from entity types."""
    type_map = {
        ("equipment", "personnel"): "ASSIGNED_TO",
        ("equipment", "regulation"): "GOVERNED_BY",
        ("equipment", "parameter"): "HAS_PARAMETER",
        ("equipment", "date"): "EVENT_DATE",
        ("equipment", "document_reference"): "REFERENCED_IN",
        ("equipment", "location"): "LOCATED_AT",
        ("equipment", "organization"): "OWNED_BY",
    }
    return type_map.get((source_type, target_type))

```

backend/app/services/ingestion/document_classifier.py

Document Classifier

Classifies documents into 7 categories using weighted keyword scoring. Each category has multiple regex patterns with weights (1-3). Filename patterns add +5 bonus. Categories: work_order, sop, inspection_report, incident_report, oem_manual, regulatory, p&id. Falls back to 'general' if score < 2.

```
"""
Document classifier – determines the category of an industrial document
based on content analysis and filename patterns.
"""

import re
import structlog

from app.models.schemas import DocumentCategory

logger = structlog.get_logger()

class DocumentClassifier:
    """Classifies industrial documents into categories using keyword/pat
    tern matching."""

    # Category indicators (keyword → category with weight)
    CATEGORY_SIGNALS = {
        DocumentCategory.PID: [
            (r"P&ID|piping.*instrument.*diagram|process flow diagram",
3),
            (r"line\s+list|equipment\s+list|instrument\s+index", 2),
            (r"control\s+valve|flow\s+transmitter|level\s+indicator",
1),
        ],
        DocumentCategory.WORK_ORDER: [
            (r"work\s+order|WO[-\s]?d{4,}|maintenance\s+order", 3),
            (r"corrective\s+maintenance|preventive\s+maintenance|breakdo
wn", 2),
            (r"spare\s+parts|man[-\s]?hours|downtime|repair", 1),
            (r"job\s+card|service\s+report|maintenance\s+record", 2),
        ],
        DocumentCategory.SOP: [
            (r"standard\s+operating\s+procedure|SOP|operating\s+instruct
ion", 3),
            (r"work\s+instruction|safe\s+work\s+practice|JSA", 2),
            (r"step\s+\d+|precaution|PPE\s+required|safety\s+measure",
```

```

1),
    (r"procedure\s+no|rev(ision)?\.\?\s*\d+|approved\s+by", 2),
],
DocumentCategory.INSPECTION_REPORT: [
    (r"inspection\s+report|NDT\s+report|thickness\s+survey", 3),
    (r"corrosion|defect|finding|recommendation", 1),
    (r"ultrasonic|radiography|magnetic\s+particle|dye\s+penetrant", 2),
    (r"fitness\s+for\s+service|remaining\s+life|next\s+inspection", 2),
],
DocumentCategory.INCIDENT_REPORT: [
    (r"incident\s+report|accident\s+report|near[-\s]?miss", 3),
    (r"root\s+cause\s+analysis|RCA|investigation", 2),
    (r"injury|fatality|fire|explosion|release|spill", 1),
    (r"corrective\s+action|preventive\s+action|CAPA", 2),
],
DocumentCategory.OEM_MANUAL: [
    (r"operation\s+manual|maintenance\s+manual|OEM|manufacture", 3),
    (r"installation\s+guide|commissioning|troubleshooting", 2),
    (r"spare\s+part.*list|exploded\s+view|wiring\s+diagram", 2),
    (r"model\s+no|serial\s+no|warranty", 1),
],
DocumentCategory.REGULATORY: [
    (r"OISD|PES0|Factory\s+Act|BIS|statutory|compliance", 3),
    (r"regulation|standard|code\s+of\s+practice|guideline", 2),
    (r"shall\s+comply|mandatory|requirement|obligation", 1),
    (r"audit|certification|license|permit|approval", 1),
],
}

# Filename pattern signals
FILENAME_PATTERNS = {
    DocumentCategory.PID: r"(?i)p&?id|pfd|process.*flow",
    DocumentCategory.WORK_ORDER: r"(?i)wo[-_]|work[-_]?order|maint",
    DocumentCategory.SOP: r"(?i)sop|procedure|oper.*instruct",
    DocumentCategory.INSPECTION_REPORT: r"(?i)insp|ndt|survey|thickness",
    DocumentCategory.INCIDENT_REPORT: r"(?i)incident|accident|near.?miss|rca",
    DocumentCategory.OEM_MANUAL: r"(?i>manual|oem|vendor|catalog",
    DocumentCategory.REGULATORY: r"(?i)oisd|regulation|standard|compliance",
}

```

```

def classify(self, text: str, filename: str = "") -> DocumentCategory:
    """Classify a document based on its content and filename."""
    scores: dict[DocumentCategory, float] = {cat: 0 for cat in DocumentCategory}

    # Score based on content patterns
    text_sample = text[:5000] # Use first 5000 chars for classification

    for category, patterns in self.CATEGORY_SIGNALS.items():
        for pattern, weight in patterns:
            matches = len(re.findall(pattern, text_sample, re.IGNORECASE))

            scores[category] += matches * weight

    # Score based on filename
    for category, pattern in self.FILENAME_PATTERNS.items():
        if re.search(pattern, filename):
            scores[category] += 5 # Filename is a strong signal

    # Get highest scoring category
    best_category = max(scores, key=scores.get)
    best_score = scores[best_category]

    if best_score < 2:
        return DocumentCategory.GENERAL

    logger.info(
        "Document classified",
        category=best_category.value,
        score=best_score,
        filename=filename,
    )
    return best_category

```

backend/app/services/ingestion/chunker.py

Text Chunker

Semantic boundary-aware chunking. First splits at section boundaries (numbered headers, horizontal rules). Then splits long sections at sentence boundaries with configurable overlap. Default: 512 chars per chunk, 64 char overlap. Returns Chunk objects with chunk_id, content, page_number, metadata.

```

"""
Text chunking utility for splitting documents into retrieval-friendly se

```

```

gments.
Uses semantic-aware chunking that respects paragraph and section boundaries.
"""

from dataclasses import dataclass
import re
import uuid
import structlog

from app.config import settings

logger = structlog.get_logger()

@dataclass
class Chunk:
    chunk_id: str
    content: str
    chunk_index: int
    page_number: int | None
    metadata: dict

class TextChunker:
    """
    Splits document text into chunks optimized for retrieval.
    Respects section/paragraph boundaries for better semantic coherence.
    """

    def __init__(
        self,
        chunk_size: int = settings.chunk_size,
        chunk_overlap: int = settings.chunk_overlap,
    ):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap

    def chunk_text(
        self,
        text: str,
        document_id: str,
        page_number: int | None = None,
        metadata: dict | None = None,
    ) -> list[Chunk]:
        """Split text into overlapping chunks with semantic boundaries.

```

```

s. """
    if not text.strip():
        return []

    metadata = metadata or {}

    # First, split by section boundaries (headers, double newlines)
    sections = self._split_into_sections(text)

    chunks = []
    chunk_index = 0

    for section in sections:
        if len(section) <= self.chunk_size:
            # Section fits in one chunk
            if section.strip():
                chunks.append(Chunk(
                    chunk_id=f"{document_id}_chunk_{chunk_index}",
                    content=section.strip(),
                    chunk_index=chunk_index,
                    page_number=page_number,
                    metadata=metadata,
                ))
                chunk_index += 1
        else:
            # Section needs further splitting
            sub_chunks = self._split_long_section(section)
            for sub in sub_chunks:
                if sub.strip():
                    chunks.append(Chunk(
                        chunk_id=f"{document_id}_chunk_{chunk_index}",
                        content=sub.strip(),
                        chunk_index=chunk_index,
                        page_number=page_number,
                        metadata=metadata,
                    ))
                    chunk_index += 1

    logger.debug("Text chunked", chunks=len(chunks), document_id=document_id)
    return chunks

def _split_into_sections(self, text: str) -> list[str]:
    """Split text at natural section boundaries."""
    # Split on section headers (numbered or capitalized lines)

```

```

        section_pattern = r"\n(?:=\d+\.\s+[A-Z]|\n[A-Z][A-Z\s]{5,}\n|\n--
-)"

        sections = re.split(section_pattern, text)
        return [s for s in sections if s.strip()]

    def _split_long_section(self, text: str) -> list[str]:
        """Split a long section into overlapping chunks at sentence boun-
daries."""
        # Split into sentences
        sentences = re.split(r"(?<=[.!?])\s+", text)

        chunks = []
        current_chunk = []
        current_length = 0

        for sentence in sentences:
            sentence_len = len(sentence)

            if current_length + sentence_len > self.chunk_size and curre
nt_chunk:

                # Emit current chunk
                chunks.append(" ".join(current_chunk))

                # Keep overlap
                overlap_text = " ".join(current_chunk)
                overlap_sentences = []
                overlap_len = 0
                for s in reversed(current_chunk):
                    if overlap_len + len(s) <= self.chunk_overlap:
                        overlap_sentences.insert(0, s)
                        overlap_len += len(s)
                    else:
                        break

                current_chunk = overlap_sentences
                current_length = overlap_len

            current_chunk.append(sentence)
            current_length += sentence_len

        # Emit remaining
        if current_chunk:
            chunks.append(" ".join(current_chunk))

        return chunks

```


backend/app/services/ingestion/llm_extractor.py

LLM Entity Extractor

Second-pass extraction using Ollama or OpenAI for complex/ambiguous entities. Sends text with a structured prompt requesting equipment, personnel, parameters, regulations, failure_modes, actions. Returns JSON with entities[] and relationships[]. Uses temperature=0 and JSON mode for deterministic output.

```
"""
LLM-powered entity and relationship extraction for complex/ambiguous cases.
Uses FREE local LLM (Ollama) by default, with OpenAI as optional paid fallback.
"""

import httpx
import json
import structlog

from app.config import settings

logger = structlog.get_logger()

EXTRACTION_PROMPT = """You are an industrial document entity extraction system.
Extract structured entities and relationships from the following text.

Extract these entity types:
- equipment: Equipment tags, names, types (pumps, valves, compressors, etc.)
- personnel: People mentioned (operators, engineers, inspectors)
- parameter: Process parameters with values and units
- regulation: Standards, codes, regulatory references (OISD, API, ISO, B IS)
- date: Dates and time references
- location: Plant areas, units, buildings
- failure_mode: Equipment failure descriptions
- action: Maintenance/operational actions taken

Also extract relationships between entities in the format:
{source_entity} -[RELATIONSHIP_TYPE]-> {target_entity}

Common relationship types:
- FAILED_DUE_TO, MAINTAINED_BY, GOVERNED_BY, LOCATED_AT,
- PERFORMED_ON, CAUSED, REPLACED_WITH, CONNECTED_TO
```

Return as JSON with this structure:

```
{
  "entities": [
    {"type": "equipment", "value": "P-101A", "confidence": 0.95}
  ],
  "relationships": [
    {"source": "P-101A", "relation": "FAILED_DUE_TO", "target": "bearing wear", "confidence": 0.85}
  ]
}
```

ONLY extract entities that are clearly present in the text. Do not infer or hallucinate.

"""

```
class LLMEntityExtractor:
```

```
    """Uses FREE local LLM (Ollama) for complex entity and relationship extraction."""
```

```
    def __init__(self):
```

```
        self.provider = settings.llm_provider
```

```
    async def extract(self, text: str) -> dict:
```

```
        """Extract entities and relationships using LLM."""
```

```
        text_chunk = text[:4000]
```

```
        if self.provider == "ollama":
```

```
            return await self._extract_ollama(text_chunk)
```

```
        elif self.provider == "openai" and settings.openai_api_key:
```

```
            return await self._extract_openai(text_chunk)
```

```
        else:
```

```
            return {"entities": [], "relationships": []}
```

```
    async def _extract_ollama(self, text: str) -> dict:
```

```
        """Extract using Ollama (FREE local)."""
```

```
        try:
```

```
            async with httpx.AsyncClient(timeout=90.0) as client:
```

```
                response = await client.post(
```

```
                    f"{settings.ollama_base_url}/api/chat",
```

```
                    json={
```

```
                        "model": settings.ollama_chat_model,
```

```
                        "messages": [
```

```
                            {"role": "system", "content": EXTRACTION_PRO
```

```
MPT},
```

```
                            {"role": "user", "content": f"Extract entiti
```

```

es from:\n\n{text}"}],
        ],
        "stream": False,
        "format": "json",
        "options": {"temperature": 0.0, "num_predict": 2000},
    },
)

if response.status_code == 200:
    content = response.json()["message"]["content"]
    result = json.loads(content)
    logger.info(
        "LLM extraction complete (Ollama)",
        entities=len(result.get("entities", [])),
        relationships=len(result.get("relationships", [])),
    )
    return result
else:
    return {"entities": [], "relationships": []}

except Exception as e:
    logger.error("Ollama extraction failed", error=str(e))
    return {"entities": [], "relationships": []}

async def _extract_openai(self, text: str) -> dict:
    """Extract using OpenAI (PAID fallback)."""
    try:
        from openai import AsyncOpenAI
        client = AsyncOpenAI(api_key=settings.openai_api_key)
        response = await client.chat.completions.create(
            model=settings.openai_chat_model,
            messages=[
                {"role": "system", "content": EXTRACTION_PROMPT},
                {"role": "user", "content": f"Extract entities from:\n\n{text}"}],
            temperature=0.0,
            response_format={"type": "json_object"},
            max_tokens=2000,
        )

        result = json.loads(response.choices[0].message.content)
        logger.info(
            "LLM extraction complete (OpenAI)",

```

```

        entities=len(result.get("entities", [])),
        relationships=len(result.get("relationships", [])),
    )
    return result

except Exception as e:
    logger.error("OpenAI extraction failed", error=str(e))
    return {"entities": [], "relationships": []}

```

backend/app/services/ingestion/pipeline.py

Main Ingestion Pipeline Orchestrator

End-to-end flow: (1) Detect file type from extension. (2) Parse document (PDF/DOCX/image/text). (3) OCR fallback for scanned pages. (4) Classify category. (5) Extract entities via regex+spaCy. (6) Extract relationships. (7) Chunk text for vector storage. Returns IngestedDocument with all extracted data. Text files (.txt/.json/.csv) read directly as plain text.

```

"""
Main ingestion pipeline – orchestrates document parsing, OCR, entity ext
raction,
chunking, knowledge graph population, and vector indexing.
"""

import uuid
import fitz
from pathlib import Path
from datetime import datetime
import structlog

from app.config import settings
from app.models.schemas import (
    DocumentType, DocumentCategory, IngestedDocument,
    DocumentChunk, ExtractedEntity, ExtractedRelationship,
)
from app.services.ingestion.pdf_parser import PDFParser, DOCXParser, Par
sedDocument
from app.services.ingestion.ocr_engine import OCREngine
from app.services.ingestion.entity_extractor import IndustrialEntityExtr
actor
from app.services.ingestion.chunker import TextChunker
from app.services.ingestion.document_classifier import DocumentClassifie
r
from app.services.ingestion.relationship_extractor import RelationshipEx
tractor

```

```

logger = structlog.get_logger()

class IngestionPipeline:
    """
    End-to-end document ingestion pipeline.

    Flow:
    1. Detect file type
    2. Parse document (extract text, tables, images)
    3. OCR fallback for scanned pages
    4. Classify document category
    5. Extract entities (equipment, personnel, regulations, etc.)
    6. Extract relationships between entities
    7. Chunk text for vector storage
    8. Return structured IngestedDocument
    """

    def __init__(self):
        self.pdf_parser = PDFParser()
        self.docx_parser = DOCXParser()
        self.ocr_engine = OCREngine()
        self.entity_extractor = IndustrialEntityExtractor()
        self.chunker = TextChunker()
        self.classifier = DocumentClassifier()
        self.relationship_extractor = RelationshipExtractor()

    async def ingest(self, file_path: str | Path, filename: str | None =
None) -> IngestedDocument:
        """Process a document through the full ingestion pipeline."""
        file_path = Path(file_path)
        filename = filename or file_path.name
        document_id = str(uuid.uuid4())

        logger.info("Starting ingestion", document_id=document_id, filename=filename)

        # Step 1: Detect file type
        file_type = self._detect_file_type(file_path)

        # Step 2: Parse document
        parsed = self._parse_document(file_path, file_type)

        # Step 3: OCR fallback for pages with no/little text
        full_text = self._extract_text_with_ocr_fallback(parsed, file_pa

```

```

th)

# Step 4: Classify document category
category = self.classifier.classify(full_text, filename)

# Step 5: Extract entities
entities = self._extract_entities(full_text, parsed)

# Step 6: Extract relationships
relationships = self.relationship_extractor.extract(full_text, entities)

# Step 7: Chunk text
chunks = self._create_chunks(full_text, document_id, parsed, filename, category)

# Build final result
document = IngestedDocument(
    document_id=document_id,
    filename=filename,
    file_type=file_type,
    category=category,
    total_pages=parsed.total_pages,
    extracted_text=full_text,
    chunks=chunks,
    entities=[
        ExtractedEntity(
            entity_type=e.entity_type,
            value=e.value,
            confidence=e.confidence,
            context=e.context,
        )
        for e in entities
    ],
    relationships=[
        ExtractedRelationship(
            source_entity=r.source,
            relationship_type=r.relation,
            target_entity=r.target,
            confidence=r.confidence,
            source_context=r.context,
        )
        for r in relationships
    ],
    ingested_at=datetime.utcnow(),
)

```

```

logger.info(
    "Ingestion complete",
    document_id=document_id,
    filename=filename,
    category=category,
    entities_found=len(entities),
    relationships_found=len(relationships),
    chunks_created=len(chunks),
)

return document

def _detect_file_type(self, file_path: Path) -> DocumentType:
    """Detect document type from extension."""
    ext = file_path.suffix.lower()
    type_map = {
        ".pdf": DocumentType.PDF,
        ".docx": DocumentType.DOCX,
        ".doc": DocumentType.DOCX,
        ".xlsx": DocumentType.XLSX,
        ".xls": DocumentType.XLSX,
        ".png": DocumentType.IMAGE,
        ".jpg": DocumentType.IMAGE,
        ".jpeg": DocumentType.IMAGE,
        ".tiff": DocumentType.IMAGE,
        ".tif": DocumentType.IMAGE,
        ".txt": DocumentType.UNKNOWN, # handled as plain text
        ".json": DocumentType.UNKNOWN,
        ".csv": DocumentType.UNKNOWN,
    }
    return type_map.get(ext, DocumentType.UNKNOWN)

def _parse_document(self, file_path: Path, file_type: DocumentType)
-> ParsedDocument:
    """Parse document based on its type."""
    if file_type == DocumentType.PDF:
        return self.pdf_parser.parse(file_path)
    elif file_type == DocumentType.DOCX:
        return self.docx_parser.parse(file_path)
    elif file_type == DocumentType.IMAGE:
        # For standalone images, create a single-page parsed doc
        ocr_result = self.ocr_engine.ocr_image_file(file_path)
        from app.services.ingestion.pdf_parser import ParsedPage
        return ParsedDocument(
            filename=file_path.name,

```

```

        total_pages=1,
        pages=[ParsedPage(
            page_number=1,
            text=ocr_result.text,
            tables=[],
            images=[],
            has_text=True,
        )],
        metadata={"ocr_confidence": ocr_result.confidence},
    )
else:
    # Plain text / JSON / CSV / unknown – read as text
    from app.services.ingestion.pdf_parser import ParsedPage
    try:
        text = file_path.read_text(encoding="utf-8", errors="replace")
    except Exception:
        try:
            text = file_path.read_text(encoding="latin-1", errors="replace")
        except Exception:
            text = ""

    if not text.strip():
        # Last resort: try PDF parsing
        try:
            return self.pdf_parser.parse(file_path)
        except Exception:
            pass

    return ParsedDocument(
        filename=file_path.name,
        total_pages=1,
        pages=[ParsedPage(
            page_number=1,
            text=text,
            tables=[],
            images=[],
            has_text=len(text.strip()) > 10,
        )],
        metadata={"format": file_path.suffix.lstrip(".")},
    )

def _extract_text_with_ocr_fallback(
    self, parsed: ParsedDocument, file_path: Path
) -> str:

```



```

        """Extract text from all pages, using OCR when direct extraction
        fails."""
        all_text_parts = []

        for page in parsed.pages:
            if page.has_text and not self.ocr_engine.needs_ocr(page.tex
t):
                # Direct text extraction worked
                all_text_parts.append(page.text)
            else:
                # Need OCR – render page to image first
                logger.info("Running OCR fallback", page=page.page_numbe
r)

                try:
                    if file_path.suffix.lower() == ".pdf":
                        doc = fitz.open(str(file_path))
                        pdf_page = doc[page.page_number - 1]
                        pix = pdf_page.get_pixmap(dpi=300)
                        image_bytes = pix.tobytes("png")
                        doc.close()
                        ocr_result = self.ocr_engine.ocr_pdf_page_image(
                            image_bytes, page.page_number
                        )
                        all_text_parts.append(ocr_result.text)
                    elif page.images:
                        # Use first image from the page
                        ocr_result = self.ocr_engine.ocr_image_bytes(
                            page.images[0], page.page_number
                        )
                        all_text_parts.append(ocr_result.text)
                    else:
                        all_text_parts.append(page.text)
                except Exception as e:
                    logger.error("OCR failed", page=page.page_number, er
ror=str(e))
                    all_text_parts.append(page.text)

                # Also include table text
                for table in page.tables:
                    for row in table:
                        row_text = " | ".join(cell or "" for cell in row)
                        if row_text.strip():
                            all_text_parts.append(row_text)

        return "\n\n".join(all_text_parts)

```

```

def _extract_entities(self, full_text: str, parsed: ParsedDocument)
-> list:
    """Extract entities from the full document text."""
    all_entities = []

    # Extract from full text
    entities = self.entity_extractor.extract_all(full_text)
    all_entities.extend(entities)

    # Also extract per-page for page-level attribution
    for page in parsed.pages:
        if page.text.strip():
            page_entities = self.entity_extractor.extract_all(
                page.text, page_number=page.page_number
            )
            for ent in page_entities:
                # Only add if not already found
                if not any(
                    e.value == ent.value and e.entity_type == ent.en
tity_type
                    for e in all_entities
                ):
                    all_entities.append(ent)

    return all_entities

def _create_chunks(
    self,
    full_text: str,
    document_id: str,
    parsed: ParsedDocument,
    filename: str,
    category: DocumentCategory,
) -> list[DocumentChunk]:
    """Create text chunks for vector storage."""
    chunks = []

    # Chunk per page to maintain page attribution
    for page in parsed.pages:
        page_text = page.text.strip()
        if not page_text:
            continue

        page_chunks = self.chunker.chunk_text(
            text=page_text,
            document_id=document_id,

```

```

        page_number=page.page_number,
        metadata={
            "filename": filename,
            "category": category.value,
            "page": page.page_number,
        },
    )

    for chunk in page_chunks:
        chunks.append(DocumentChunk(
            chunk_id=chunk.chunk_id,
            document_id=document_id,
            content=chunk.content,
            page_number=chunk.page_number,
            chunk_index=chunk.chunk_index,
            metadata=chunk.metadata,
        ))

    # If no page-level chunking worked, chunk the full text
    if not chunks and full_text.strip():
        full_chunks = self.chunker.chunk_text(
            text=full_text,
            document_id=document_id,
            metadata={"filename": filename, "category": category.val
ue},
        )
        for chunk in full_chunks:
            chunks.append(DocumentChunk(
                chunk_id=chunk.chunk_id,
                document_id=document_id,
                content=chunk.content,
                page_number=chunk.page_number,
                chunk_index=chunk.chunk_index,
                metadata=chunk.metadata,
            ))

    return chunks

```

backend/app/services/knowledge_graph/neo4j_client.py

Neo4j Client

Async Neo4j driver wrapper. On init: creates uniqueness constraints (Equipment.tag, Document.document_id, etc.) and indexes. Methods: create_node (MERGE to avoid duplicates), create_relationship, get_neighbors (variable-depth traversal), search_nodes

(UNION across all types), get_equipment_context (returns all connected work orders, incidents, regulations, inspections).

```
"""
Neo4j client for knowledge graph operations.
Handles connection management, schema initialization, and CRUD operations.
"""

from neo4j import AsyncGraphDatabase, AsyncDriver
import structlog

from app.config import settings

logger = structlog.get_logger()

class Neo4jClient:
    """Async Neo4j client for the AXIOM knowledge graph."""

    def __init__(self):
        self._driver: AsyncDriver | None = None

    async def initialize(self):
        """Initialize connection and create schema constraints."""
        self._driver = AsyncGraphDatabase.driver(
            settings.neo4j_uri,
            auth=(settings.neo4j_user, settings.neo4j_password),
        )

        # Verify connectivity
        try:
            await self._driver.verify_connectivity()
            logger.info("Neo4j connected", uri=settings.neo4j_uri)
        except Exception as e:
            logger.error("Neo4j connection failed", error=str(e))
            self._driver = None
            return

        # Create schema constraints and indexes
        await self._create_schema()

    async def _create_schema(self):
        """Create uniqueness constraints and indexes for the knowledge graph."""
        constraints = [
```

```

        "CREATE CONSTRAINT IF NOT EXISTS FOR (e:Equipment) REQUIRE
e.tag IS UNIQUE",
        "CREATE CONSTRAINT IF NOT EXISTS FOR (d:Document) REQUIRE d.
document_id IS UNIQUE",
        "CREATE CONSTRAINT IF NOT EXISTS FOR (p:Procedure) REQUIRE
p.procedure_id IS UNIQUE",
        "CREATE CONSTRAINT IF NOT EXISTS FOR (w:WorkOrder) REQUIRE
w.wo_id IS UNIQUE",
        "CREATE CONSTRAINT IF NOT EXISTS FOR (i:Incident) REQUIRE i.
incident_id IS UNIQUE",
        "CREATE CONSTRAINT IF NOT EXISTS FOR (r:Regulation) REQUIRE
r.standard_id IS UNIQUE",
        "CREATE CONSTRAINT IF NOT EXISTS FOR (pe:Personnel) REQUIRE
pe.name IS UNIQUE",
    ]

    indexes = [
        "CREATE INDEX IF NOT EXISTS FOR (e:Equipment) ON (e.type)",
        "CREATE INDEX IF NOT EXISTS FOR (d:Document) ON (d.categor
y)",
        "CREATE INDEX IF NOT EXISTS FOR (d:Document) ON (d.filename
e)",
        "CREATE INDEX IF NOT EXISTS FOR (w:WorkOrder) ON (w.statu
s)",
    ]

    async with self._driver.session() as session:
        for query in constraints + indexes:
            try:
                await session.run(query)
            except Exception as e:
                logger.debug("Schema query skipped", query=query[:5
0], reason=str(e))

        logger.info("Neo4j schema initialized")

    async def close(self):
        """Close the Neo4j driver."""
        if self._driver:
            await self._driver.close()
            logger.info("Neo4j connection closed")

    async def execute_query(self, query: str, parameters: dict = None) -
> list[dict]:
        """Execute a Cypher query and return results."""
        if not self._driver:

```

```

        logger.warning("Neo4j not connected, skipping query")
        return []

    async with self._driver.session() as session:
        result = await session.run(query, parameters or {})
        records = await result.data()
        return records

    async def create_node(self, label: str, properties: dict) -> str:
        """Create or merge a node in the graph."""
        # Use MERGE to avoid duplicates
        prop_string = ", ".join(f"{k}: ${k}" for k in properties.keys())
        query = f"MERGE (n:{label} {{{prop_string}}}) RETURN elementId(n) as id"

        async with self._driver.session() as session:
            result = await session.run(query, properties)
            record = await result.single()
            return record["id"] if record else ""

    async def create_relationship(
        self,
        source_label: str,
        source_key: str,
        source_value: str,
        target_label: str,
        target_key: str,
        target_value: str,
        relationship: str,
        properties: dict = None,
    ):
        """Create a relationship between two nodes."""
        props = ""
        params = {
            "source_value": source_value,
            "target_value": target_value,
        }

        if properties:
            prop_string = ", ".join(f"{k}: ${k}" for k in properties.keys())

            props = f" {{{prop_string}}}"
            params.update(properties)

        query = f"""
MATCH (a:{source_label} {{{source_key}: $source_value}})

```

```

MATCH (b:{target_label} {{{target_key}: $target_value}})
MERGE (a)-[r:{relationship}{{props}}]->(b)
RETURN type(r) as rel_type
"""

async with self._driver.session() as session:
    await session.run(query, params)

async def get_neighbors(
    self, node_label: str, node_key: str, node_value: str, depth: in
t = 2
) -> list[dict]:
    """Get neighboring nodes up to specified depth."""
    query = f"""
MATCH path = (n:{node_label} {{{node_key}: $value}})-[*1..{dept
h}]- (m)
RETURN
    labels(m)[0] as node_type,
    properties(m) as properties,
    [r in relationships(path) | type(r)] as relationships
LIMIT 50
"""
    return await self.execute_query(query, {"value": node_value})

async def search_nodes(self, search_term: str, limit: int = 20) -> l
ist[dict]:
    """Full-text search across all node types."""
    query = """
CALL {
    MATCH (n:Equipment) WHERE n.tag CONTAINS $term
    RETURN n, labels(n)[0] as label
    UNION
    MATCH (n:Document) WHERE n.filename CONTAINS $term
    RETURN n, labels(n)[0] as label
    UNION
    MATCH (n:Personnel) WHERE n.name CONTAINS $term
    RETURN n, labels(n)[0] as label
    UNION
    MATCH (n:Regulation) WHERE n.standard_id CONTAINS $term
    RETURN n, labels(n)[0] as label
}
RETURN label, properties(n) as props
LIMIT $limit
"""
    return await self.execute_query(query, {"term": search_term, "li
mit": limit})

```

```

async def get_equipment_context(self, equipment_tag: str) -> dict:
    """Get full context for a piece of equipment (all connected nodes)."""
    query = """
    MATCH (e:Equipment {tag: $tag})
    OPTIONAL MATCH (e)-[:HAS_WORK_ORDER]->(wo:WorkOrder)
    OPTIONAL MATCH (e)-[:GOVERNED_BY]->(r:Regulation)
    OPTIONAL MATCH (e)-[:INSPECTED_BY]->(i:Inspection)
    OPTIONAL MATCH (e)-[:INVOLVED_IN]->(inc:Incident)
    OPTIONAL MATCH (e)-[:HAS_PROCEDURE]->(p:Procedure)
    OPTIONAL MATCH (e)-[:REFERENCED_IN]->(d:Document)
    RETURN
        properties(e) as equipment,
        collect(DISTINCT properties(wo)) as work_orders,
        collect(DISTINCT properties(r)) as regulations,
        collect(DISTINCT properties(i)) as inspections,
        collect(DISTINCT properties(inc)) as incidents,
        collect(DISTINCT properties(p)) as procedures,
        collect(DISTINCT properties(d)) as documents
    """
    results = await self.execute_query(query, {"tag": equipment_tag})
    return results[0] if results else {}

```

backend/app/services/knowledge_graph/graph_builder.py

Knowledge Graph Builder

Maps extracted entities to Neo4j nodes using ontology: equipment→Equipment, personnel→Personnel, regulation→Regulation, etc. Creates REFERENCED_IN relationships from entities to documents. Creates inter-entity relationships from RelationshipExtractor output. Handles label guessing from entity value format (regex pattern matching).

```

"""
Knowledge Graph Builder – populates Neo4j from extracted entities and relationships.
Maps extracted data to the industrial ontology.
"""

import structlog

from app.models.schemas import IngestedDocument, DocumentCategory
from app.services.knowledge_graph.neo4j_client import Neo4jClient

logger = structlog.get_logger()

```



```

class GraphBuilder:
    """Builds and updates the knowledge graph from ingested documents."""

    def __init__(self, neo4j_client: Neo4jClient):
        self.neo4j = neo4j_client

    async def populate_from_document(self, document: IngestedDocument):
        """Populate the knowledge graph from an ingested document."""
        logger.info(
            "Populating knowledge graph",
            document_id=document.document_id,
            entities=len(document.entities),
            relationships=len(document.relationships),
        )

        # 1. Create the Document node
        await self._create_document_node(document)

        # 2. Create entity nodes and link to document
        await self._create_entity_nodes(document)

        # 3. Create relationships between entities
        await self._create_relationships(document)

        logger.info("Knowledge graph updated", document_id=document.document_id)

    async def _create_document_node(self, document: IngestedDocument):
        """Create a Document node in the graph."""
        await self.neo4j.create_node("Document", {
            "document_id": document.document_id,
            "filename": document.filename,
            "file_type": document.file_type.value,
            "category": document.category.value,
            "total_pages": document.total_pages,
            "ingested_at": document.ingested_at.isoformat(),
        })

    async def _create_entity_nodes(self, document: IngestedDocument):
        """Create nodes for each extracted entity and link to the document."""
        for entity in document.entities:
            node_label = self._entity_type_to_label(entity.entity_type)

```

```

        if not node_label:
            continue

        # Create the entity node
        properties = self._build_entity_properties(entity)
        await self.neo4j.create_node(node_label, properties)

        # Link entity to document
        key_field = self._get_key_field(node_label)
        if key_field:
            await self.neo4j.create_relationship(
                source_label=node_label,
                source_key=key_field,
                source_value=properties[key_field],
                target_label="Document",
                target_key="document_id",
                target_value=document.document_id,
                relationship="REFERENCED_IN",
                properties={"confidence": entity.confidence},
            )

    async def _create_relationships(self, document: IngestedDocument):
        """Create relationship edges between entity nodes."""
        for rel in document.relationships:
            source_label = self._guess_label_from_value(rel.source_entity)
            target_label = self._guess_label_from_value(rel.target_entity)

            if not source_label or not target_label:
                continue

            source_key = self._get_key_field(source_label)
            target_key = self._get_key_field(target_label)

            if source_key and target_key:
                try:
                    await self.neo4j.create_relationship(
                        source_label=source_label,
                        source_key=source_key,
                        source_value=rel.source_entity,
                        target_label=target_label,
                        target_key=target_key,
                        target_value=rel.target_entity,
                        relationship=rel.relationship_type,
                        properties={

```

```

        "confidence": rel.confidence,
        "source_document": document.document_id,
    },
)
except Exception as e:
    logger.debug(
        "Relationship creation skipped",
        source=rel.source_entity,
        target=rel.target_entity,
        error=str(e),
    )

def _entity_type_to_label(self, entity_type: str) -> str | None:
    """Map entity type to Neo4j node label."""
    mapping = {
        "equipment": "Equipment",
        "personnel": "Personnel",
        "regulation": "Regulation",
        "parameter": "Parameter",
        "document_reference": "Document",
        "location": "Location",
        "organization": "Organization",
        "facility": "Location",
    }
    return mapping.get(entity_type)

def _build_entity_properties(self, entity) -> dict:
    """Build node properties from an entity."""
    type_to_props = {
        "equipment": {"tag": entity.value, "context": entity.context[:200]},
        "personnel": {"name": entity.value},
        "regulation": {"standard_id": entity.value, "context": entity.context[:200]},
        "parameter": {"value": entity.value, "context": entity.context[:200]},
        "document_reference": {"document_id": entity.value},
        "location": {"name": entity.value},
        "organization": {"name": entity.value},
    }
    return type_to_props.get(entity.entity_type, {"value": entity.value})

def _get_key_field(self, label: str) -> str | None:
    """Get the primary key field for a node label."""
    key_map = {

```

```

        "Equipment": "tag",
        "Personnel": "name",
        "Regulation": "standard_id",
        "Parameter": "value",
        "Document": "document_id",
        "Location": "name",
        "Organization": "name",
        "WorkOrder": "wo_id",
        "Incident": "incident_id",
        "Procedure": "procedure_id",
        "Inspection": "inspection_id",
    }
    return key_map.get(label)

def _guess_label_from_value(self, value: str) -> str | None:
    """Guess the node label from the entity value format."""
    import re

    # Equipment tag pattern
    if re.match(r"^[A-Z]{1,4}-\d{2,5}[A-Z]?$", value):
        return "Equipment"
    # Work order
    if re.match(r"^W0[-\s]?[d]{4,}$", value, re.IGNORECASE):
        return "WorkOrder"
    # Regulation
    if re.match(r"^(OISD|API|ISO|ASME|IS)[- \s]?[d]+$", value):
        return "Regulation"
    # If it looks like a person name
    if re.match(r"^[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+$", value):
        return "Personnel"

    return None

```

backend/app/services/vectorstore/qdrant_service.py

Qdrant Vector Store + Embedding Engine

EmbeddingEngine: 3-tier fallback: (1) sentence-transformers local (preferred, free, fast). (2) Ollama embeddings via HTTP API. (3) OpenAI embeddings (paid fallback). Supports batch encoding. QdrantService: Creates collection with cosine distance, indexes chunks with embeddings, supports filtered search by category. 384-dim vectors by default.

```

"""
Qdrant vector store service – handles embedding generation, storage, and
retrieval.
Uses FREE local sentence-transformers for embeddings (no API costs).

```

```

    Optionally supports Ollama embeddings or OpenAI (paid) as fallback.
    """

    from qdrant_client import QdrantClient
    from qdrant_client.models import (
        Distance, VectorParams, PointStruct,
        Filter, FieldCondition, MatchValue,
    )
    import uuid
    import structlog

    from app.config import settings
    from app.models.schemas import DocumentChunk

    logger = structlog.get_logger()

    class EmbeddingEngine:
        """
        Embedding generation using FREE local models.
        Priority: sentence-transformers (local) > Ollama (local) > OpenAI (paid fallback)
        """

        def __init__(self):
            self._st_model = None
            self._ollama_available = False

        def _load_sentence_transformer(self):
            """Load sentence-transformers model (FREE, local, fast)."""
            if self._st_model is None:
                try:
                    from sentence_transformers import SentenceTransformer
                    self._st_model = SentenceTransformer(settings.local_embedding_model)

                    logger.info(
                        "Loaded local embedding model",
                        model=settings.local_embedding_model,
                    )
                except Exception as e:
                    logger.warning("sentence-transformers not available", error=str(e))
                return self._st_model

        async def generate_embedding(self, text: str) -> list[float]:
            """Generate embedding using best available FREE method."""

```

```

        # Method 1: Local sentence-transformers (preferred – fast, free,
no network)
        model = self._load_sentence_transformer()
        if model is not None:
            embedding = model.encode(text[:512], normalize_embeddings=True)

            return embedding.tolist()

        # Method 2: Ollama embeddings (free, local, needs Ollama running)
        try:
            import httpx
            async with httpx.AsyncClient() as client:
                response = await client.post(
                    f"{settings.ollama_base_url}/api/embeddings",
                    json={"model": settings.ollama_embedding_model, "prompt": text[:2000]},
                    timeout=30.0,
                )
                if response.status_code == 200:
                    return response.json()["embedding"]
        except Exception as e:
            logger.debug("Ollama embeddings failed", error=str(e))

        # Method 3: OpenAI (PAID fallback – only if key configured)
        if settings.openai_api_key:
            try:
                from openai import AsyncOpenAI
                client = AsyncOpenAI(api_key=settings.openai_api_key)
                response = await client.embeddings.create(
                    model=settings.openai_embedding_model,
                    input=text[:8000],
                )
                return response.data[0].embedding
            except Exception as e:
                logger.error("OpenAI embedding failed", error=str(e))

        # Fallback: zero vector
        logger.warning("No embedding engine available, using zero vector")

        return [0.0] * settings.embedding_dimension

    async def generate_embeddings_batch(self, texts: list[str]) -> list[
list[float]]:
        """Generate embeddings for multiple texts in batch."""
        # sentence-transformers supports efficient batching

```

```

        model = self._load_sentence_transformer()
        if model is not None:
            truncated = [t[:512] for t in texts]
            embeddings = model.encode(truncated, normalize_embeddings=True, batch_size=32)
            return [e.tolist() for e in embeddings]

        # Fallback: one by one
        results = []
        for text in texts:
            emb = await self.generate_embedding(text)
            results.append(emb)
        return results

class QdrantService:
    """Manages vector embeddings in Qdrant for semantic search."""

    def __init__(self):
        self.client: QdrantClient | None = None
        self.embedding_engine = EmbeddingEngine()
        self.collection_name = settings.qdrant_collection

    async def initialize(self):
        """Initialize Qdrant client and create collection if needed."""
        try:
            self.client = QdrantClient(
                host=settings.qdrant_host,
                port=settings.qdrant_port,
            )

            # Create collection if it doesn't exist
            collections = self.client.get_collections().collections
            collection_names = [c.name for c in collections]

            if self.collection_name not in collection_names:
                self.client.create_collection(
                    collection_name=self.collection_name,
                    vectors_config=VectorParams(
                        size=settings.embedding_dimension,
                        distance=Distance.COSINE,
                    ),
                )
                logger.info("Qdrant collection created", name=self.collection_name)
            else:

```

```

        logger.info("Qdrant collection exists", name=self.collection_name)

    except Exception as e:
        logger.error("Qdrant initialization failed", error=str(e))
        self.client = None

    async def generate_embedding(self, text: str) -> list[float]:
        """Generate embedding vector using FREE local model."""
        return await self.embedding_engine.generate_embedding(text)

    async def generate_embeddings_batch(self, texts: list[str]) -> list[list[float]]:
        """Generate embeddings for multiple texts in a batch."""
        return await self.embedding_engine.generate_embeddings_batch(texts)

    async def index_chunks(self, chunks: list[DocumentChunk], document_id: str):
        """Index document chunks with their embeddings in Qdrant."""
        if not self.client:
            logger.warning("Qdrant not connected, skipping indexing")
            return

        if not chunks:
            return

        # Generate embeddings in batch
        texts = [chunk.content for chunk in chunks]
        embeddings = await self.generate_embeddings_batch(texts)

        # Create Qdrant points
        points = []
        for chunk, embedding in zip(chunks, embeddings):
            point_id = str(uuid.uuid5(uuid.NAMESPACE_DNS, chunk.chunk_id))

            points.append(PointStruct(
                id=point_id,
                vector=embedding,
                payload={
                    "chunk_id": chunk.chunk_id,
                    "document_id": document_id,
                    "content": chunk.content,
                    "page_number": chunk.page_number,
                    "chunk_index": chunk.chunk_index,
                    "filename": chunk.metadata.get("filename", ""),

```



```

        "category": chunk.metadata.get("category", ""),
    },
))

# Upsert in batches of 100
batch_size = 100
for i in range(0, len(points), batch_size):
    batch = points[i:i + batch_size]
    self.client.upsert(
        collection_name=self.collection_name,
        points=batch,
    )

logger.info(
    "Chunks indexed in Qdrant",
    document_id=document_id,
    chunks_indexed=len(points),
)

async def search(
    self,
    query: str,
    top_k: int = 5,
    category_filter: str | None = None,
) -> list[dict]:
    """Perform semantic search over indexed chunks."""
    if not self.client:
        return []

    # Generate query embedding
    query_embedding = await self.generate_embedding(query)

    # Build filter if needed
    search_filter = None
    if category_filter:
        search_filter = Filter(
            must=[
                FieldCondition(
                    key="category",
                    match=MatchValue(value=category_filter),
                )
            ]
        )

    # Search
    results = self.client.search(

```

```

        collection_name=self.collection_name,
        query_vector=query_embedding,
        limit=top_k,
        query_filter=search_filter,
    )

    # Format results
    formatted = []
    for hit in results:
        formatted.append({
            "chunk_id": hit.payload.get("chunk_id", ""),
            "document_id": hit.payload.get("document_id", ""),
            "filename": hit.payload.get("filename", ""),
            "content": hit.payload.get("content", ""),
            "page_number": hit.payload.get("page_number"),
            "score": hit.score,
        })

    return formatted

async def delete_document(self, document_id: str):
    """Delete all chunks belonging to a document."""
    if not self.client:
        return

    self.client.delete(
        collection_name=self.collection_name,
        points_selector=Filter(
            must=[
                FieldCondition(
                    key="document_id",
                    match=MatchValue(value=document_id),
                )
            ]
        ),
    )
    logger.info("Document vectors deleted", document_id=document_id)

```

backend/app/services/rag/retriever.py

Hybrid Retriever

Three retrieval strategies: (1) Vector search via Qdrant (semantic similarity). (2) Graph search: extracts equipment tags/regulations from query, calls `neo4j.get_equipment_context()`, formats graph context as text. (3) BM25 keyword search over tokenized corpus. Results fused via Reciprocal Rank Fusion: $RRF_score = \sum(1/(k+rank))$ with $k=60$.

```

"""
Hybrid Retriever – combines vector search, knowledge graph traversal,
and keyword (BM25) search for comprehensive industrial document retrieval.
"""

import re
from rank_bm25 import BM25Okapi
from dataclasses import dataclass
import structlog

from app.services.vectorstore.qdrant_service import QdrantService
from app.services.knowledge_graph.neo4j_client import Neo4jClient

logger = structlog.get_logger()

@dataclass
class RetrievalResult:
    chunk_id: str
    document_id: str
    filename: str
    content: str
    page_number: int | None
    score: float
    source_type: str # "vector", "graph", "keyword"

class HybridRetriever:
    """
    Multi-strategy retrieval engine that combines:
    1. Vector Search – semantic similarity (good for natural language queries)
    2. Graph Traversal – relationship-aware retrieval (good for entity-specific queries)
    3. BM25 Keyword Search – exact term matching (good for equipment tags, IDs)

    Results are fused using Reciprocal Rank Fusion (RRF).
    """

    def __init__(self, qdrant: QdrantService, neo4j: Neo4jClient):
        self.qdrant = qdrant
        self.neo4j = neo4j
        self._bm25_corpus: list[dict] | None = None
        self._bm25_index: BM25Okapi | None = None

```

```

async def retrieve(
    self,
    query: str,
    top_k: int = 5,
    use_vector: bool = True,
    use_graph: bool = True,
    use_keyword: bool = True,
    category_filter: str | None = None,
) -> list[RetrievalResult]:
    """
    Perform hybrid retrieval and return fused, re-ranked results.
    """
    all_results: list[list[RetrievalResult]] = []

    # 1. Vector search (semantic similarity)
    if use_vector:
        vector_results = await self._vector_search(query, top_k * 2,
category_filter)
        all_results.append(vector_results)
        logger.debug("Vector search", results=len(vector_results))

    # 2. Knowledge graph traversal
    if use_graph:
        graph_results = await self._graph_search(query, top_k * 2)
        all_results.append(graph_results)
        logger.debug("Graph search", results=len(graph_results))

    # 3. BM25 keyword search
    if use_keyword and self._bm25_index:
        keyword_results = self._keyword_search(query, top_k * 2)
        all_results.append(keyword_results)
        logger.debug("Keyword search", results=len(keyword_results))

    # Fuse results using Reciprocal Rank Fusion
    fused = self._reciprocal_rank_fusion(all_results, k=60)

    # Return top_k
    final_results = fused[:top_k]
    logger.info("Hybrid retrieval complete", total_results=len(final
_results))
    return final_results

async def _vector_search(
    self, query: str, top_k: int, category_filter: str | None
) -> list[RetrievalResult]:

```

```

        """Semantic search via Qdrant."""
        results = await self.qdrant.search(
            query=query, top_k=top_k, category_filter=category_filter
        )
        return [
            RetrievalResult(
                chunk_id=r["chunk_id"],
                document_id=r["document_id"],
                filename=r["filename"],
                content=r["content"],
                page_number=r.get("page_number"),
                score=r["score"],
                source_type="vector",
            )
            for r in results
        ]

    async def _graph_search(self, query: str, top_k: int) -> list[RetrievalResult]:
        """
        Extract entities from query and traverse the knowledge graph
        to find related document chunks.
        """
        # Extract potential equipment tags from the query
        equipment_tags = re.findall(r"\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b", query)

        regulation_refs = re.findall(r"\b(?:0ISD|API|ISO|ASME)[- \s]? \d+ \b", query)

        results = []

        # Search for equipment context
        for tag in equipment_tags:
            context = await self.neo4j.get_equipment_context(tag)
            if context:
                # Build a text representation of the graph context
                context_text = self._format_graph_context(tag, context)
                results.append(RetrievalResult(
                    chunk_id=f"graph_{tag}",
                    document_id="knowledge_graph",
                    filename="Knowledge Graph",
                    content=context_text,
                    page_number=None,
                    score=0.9,
                    source_type="graph",
                ))

```

```

        # Search for regulation context
        for ref in regulation_refs:
            neighbors = await self.neo4j.get_neighbors("Regulation", "standard_id", ref)
            if neighbors:
                context_text = f"Regulation {ref} applies to: "
                context_text += "; ".join(
                    f"{n['node_type']}: {n['properties']}" for n in neighbors[:5]
                )
                results.append(RetrievalResult(
                    chunk_id=f"graph_{ref}",
                    document_id="knowledge_graph",
                    filename="Knowledge Graph",
                    content=context_text,
                    page_number=None,
                    score=0.85,
                    source_type="graph",
                ))

    # General graph search
    search_terms = self._extract_search_terms(query)
    for term in search_terms[:3]: # Limit to avoid too many queries
        nodes = await self.neo4j.search_nodes(term, limit=5)
        for node in nodes:
            props = node.get("props", {})
            results.append(RetrievalResult(
                chunk_id=f"graph_{term}_{len(results)}",
                document_id="knowledge_graph",
                filename=props.get("filename", "Knowledge Graph"),
                content=str(props),
                page_number=None,
                score=0.7,
                source_type="graph",
            ))

    return results[:top_k]

def _keyword_search(self, query: str, top_k: int) -> list[RetrievalResult]:
    """BM25 keyword search over indexed corpus."""
    if not self._bm25_index or not self._bm25_corpus:
        return []

    tokenized_query = query.lower().split()

```

```

        scores = self._bm25_index.get_scores(tokenized_query)

        # Get top results
        top_indices = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)[:top_k]

        results = []
        for idx in top_indices:
            if scores[idx] > 0:
                doc = self._bm25_corpus[idx]
                results.append(RetrievalResult(
                    chunk_id=doc.get("chunk_id", f"bm25_{idx}"),
                    document_id=doc.get("document_id", ""),
                    filename=doc.get("filename", ""),
                    content=doc.get("content", ""),
                    page_number=doc.get("page_number", ""),
                    score=float(scores[idx]),
                    source_type="keyword",
                ))

        return results

def update_bm25_index(self, documents: list[dict]):
    """Update the BM25 index with new documents."""
    self._bm25_corpus = documents
    tokenized_corpus = [doc["content"].lower().split() for doc in documents]

    if tokenized_corpus:
        self._bm25_index = BM25Okapi(tokenized_corpus)
        logger.info("BM25 index updated", documents=len(documents))

def _reciprocal_rank_fusion(
    self, result_lists: list[list[RetrievalResult]], k: int = 60
) -> list[RetrievalResult]:
    """
    Fuse multiple ranked lists using Reciprocal Rank Fusion.
    RRF score = sum(1 / (k + rank_i)) for each list where the doc appears.
    """
    scores: dict[str, float] = {}
    result_map: dict[str, RetrievalResult] = {}

    for result_list in result_lists:
        for rank, result in enumerate(result_list):
            key = result.chunk_id
            rrf_score = 1.0 / (k + rank + 1)

```

```

        scores[key] = scores.get(key, 0) + rrf_score

        # Keep the result with highest individual score
        if key not in result_map or result.score > result_map[key].score:
            result_map[key] = result

    # Sort by fused score
    sorted_keys = sorted(scores.keys(), key=lambda x: scores[x], reverse=True)

    # Update scores in results
    fused_results = []
    for key in sorted_keys:
        result = result_map[key]
        result.score = scores[key]
        fused_results.append(result)

    return fused_results

def _format_graph_context(self, equipment_tag: str, context: dict) -
> str:
    """Format knowledge graph context into readable text."""
    parts = [f"Equipment: {equipment_tag}"]

    equipment = context.get("equipment", {})
    if equipment:
        parts.append(f"Properties: {equipment}")

    work_orders = context.get("work_orders", [])
    if work_orders:
        parts.append(f"Work Orders ({len(work_orders)}): {work_order
s[:3]}")

    incidents = context.get("incidents", [])
    if incidents:
        parts.append(f"Incidents ({len(incidents)}): {incidents[:
3]}")

    regulations = context.get("regulations", [])
    if regulations:
        parts.append(f"Applicable Regulations: {regulations}")

    inspections = context.get("inspections", [])
    if inspections:
        parts.append(f"Inspections ({len(inspections)}): {inspection

```



```

s[:3]})

    return "\n".join(parts)

def _extract_search_terms(self, query: str) -> list[str]:
    """Extract meaningful search terms from a natural language query."""
    # Remove common stop words and extract key terms
    stop_words = {
        "what", "is", "the", "a", "an", "of", "for", "in", "on", "at",
        "to", "from", "by", "with", "how", "when", "where", "which",
        "show", "me", "tell", "find", "get", "list", "all", "any",
    }
    words = query.split()
    terms = [w for w in words if w.lower() not in stop_words and len(w) > 2]
    return terms

```

backend/app/services/rag/generator.py

Answer Generator

Builds numbered [Source N] context from retrieved chunks. System prompt enforces: cite sources, note conflicts, assess confidence, suggest follow-ups. Calls Ollama (free) or OpenAI (paid) at temperature=0.1 for factual accuracy. Confidence assessed by citation count + context richness. Generates follow-up suggestions based on entity types found. Fallback mode returns raw context if no LLM available.

```

"""
Answer Generator – takes retrieved context and generates a grounded,
cited answer using an LLM with strict citation enforcement.
Uses FREE local LLM (Ollama) by default, with OpenAI as optional paid fallback.
"""

import httpx
from dataclasses import dataclass
import structlog
import json

from app.config import settings
from app.services.rag.retriever import RetrievalResult
from app.models.schemas import QueryResponse, RetrievedContext

logger = structlog.get_logger()

```

```
SYSTEM_PROMPT = """You are AXIOM, an AI-powered Industrial Knowledge Intelligence assistant.
You help maintenance engineers, field technicians, and operations teams find answers
from their industrial document corpus (SOPs, work orders, inspection reports, P&IDs,
incident reports, and regulatory documents).
```

CRITICAL RULES:

1. ONLY answer based on the provided context. If the context doesn't contain enough information, say "I don't have sufficient information to answer this confidently."
2. ALWAYS cite your sources using [Source N] notation where N corresponds to the context chunk number.
3. If you see conflicting information across sources, note the discrepancy.
4. For safety-critical information (procedures, limits, regulatory requirements), be explicit about your confidence level.
5. When mentioning equipment tags, parameters, or standards, preserve exact values.
6. Suggest follow-up questions that might help the user get more specific information.

RESPONSE FORMAT:

- Lead with a direct, concise answer
 - Support with details and citations
 - End with confidence assessment and suggested follow-ups
- ```
"""
```

#### class AnswerGenerator:

```
 """Generates grounded, cited answers from retrieved context.
 Uses Ollama (FREE local) by default, OpenAI as optional paid fallback."""
```

```
 def __init__(self):
 self.provider = settings.llm_provider # "ollama" or "openai"
```

```
 async def generate(
 self,
 query: str,
```

```

 retrieved_chunks: list[RetrievalResult],
 graph_context: dict | None = None,
) -> QueryResponse:
 """Generate an answer from retrieved context."""

 # Build context string with numbered sources
 context_parts = []
 sources = []

 for i, chunk in enumerate(retrieved_chunks, 1):
 source_label = f"[Source {i}]"
 context_parts.append(
 f"{source_label}\n"
 f"Document: {chunk.filename}\n"
 f"Page: {chunk.page_number or 'N/A'}\n"
 f"Content: {chunk.content}\n"
 f"Retrieval Method: {chunk.source_type}\n"
)
 sources.append(RetrievedContext(
 chunk_id=chunk.chunk_id,
 document_id=chunk.document_id,
 filename=chunk.filename,
 content=chunk.content,
 page_number=chunk.page_number,
 relevance_score=chunk.score,
 source_type=chunk.source_type,
))

 # Add graph context if available
 if graph_context:
 context_parts.append(
 f"\n[Knowledge Graph Context]\n{json.dumps(graph_context, indent=2, default=str)}"
)

 full_context = "\n---\n".join(context_parts)

 # Generate answer
 if self.provider == "ollama":
 answer, confidence = await self._ollama_generate(query, full_context)
 elif self.provider == "openai" and settings.openai_api_key:
 answer, confidence = await self._openai_generate(query, full_context)
 else:
 # Fallback: return context summary without LLM

```

```

 answer = self._fallback_answer(query, retrieved_chunks)
 confidence = "low"

 # Generate follow-up suggestions
 followups = self._generate_followups(query, retrieved_chunks)

 # Extract related entities from context
 related_entities = self._extract_related_entities(retrieved_chunks)

 return QueryResponse(
 answer=answer,
 confidence=confidence,
 sources=sources,
 related_entities=related_entities,
 suggested_followups=followups,
)

 async def _ollama_generate(self, query: str, context: str) -> tuple[
 str, str]:
 """Generate answer using Ollama (FREE local LLM)."""
 try:
 async with httpx.AsyncClient(timeout=120.0) as client:
 response = await client.post(
 f"{settings.ollama_base_url}/api/chat",
 json={
 "model": settings.ollama_chat_model,
 "messages": [
 {"role": "system", "content": SYSTEM_PROMPT},
 {
 "role": "user",
 "content": f"Context:\n{context}\n\nQuestion: {query}",
 },
],
 "stream": False,
 "options": {
 "temperature": 0.1,
 "num_predict": 1500,
 },
 },
)

 if response.status_code == 200:
 result = response.json()

```

```

 answer = result["message"]["content"]
 confidence = self._assess_confidence(answer, context)

 return answer, confidence
else:
 logger.error("Ollama request failed", status=response.status_code)
 return self._fallback_answer(query, []), "low"

except Exception as e:
 logger.error("Ollama generation failed", error=str(e))
 return f"LLM unavailable. Ensure Ollama is running: `ollama serve`\nError: {str(e)}", "low"

async def _openai_generate(self, query: str, context: str) -> tuple[str, str]:
 """Generate answer using OpenAI (PAID fallback)."""
 try:
 from openai import AsyncOpenAI
 client = AsyncOpenAI(api_key=settings.openai_api_key)
 response = await client.chat.completions.create(
 model=settings.openai_chat_model,
 messages=[
 {"role": "system", "content": SYSTEM_PROMPT},
 {
 "role": "user",
 "content": f"Context:\n{context}\n\nQuestion: {query}",
 },
],
 temperature=0.1,
 max_tokens=1500,
)

 answer = response.choices[0].message.content
 confidence = self._assess_confidence(answer, context)
 return answer, confidence

 except Exception as e:
 logger.error("OpenAI generation failed", error=str(e))
 return f"Error generating answer: {str(e)}", "low"

def _fallback_answer(self, query: str, chunks: list[RetrievalResult]) -> str:
 """Generate a basic answer without LLM (fallback mode)."""
 if not chunks:

```

```

 return "No relevant documents found for your query."

 answer_parts = [
 f"Based on {len(chunks)} relevant document(s) found:\n"
]

 for i, chunk in enumerate(chunks[:3], 1):
 answer_parts.append(
 f"\n**[Source {i}]** From '{chunk.filename}'"
 f"{' (Page {chunk.page_number})' if chunk.page_number e"
lse '{}':\n"
 f"{chunk.content[:300]}{'...' if len(chunk.content) > 30
0 else ''}\n"
)

 answer_parts.append(
 "\n*Note: Running in fallback mode without LLM. "
 "Configure OPENAI_API_KEY for full answer generation.*"
)
 return "\n".join(answer_parts)

def _assess_confidence(self, answer: str, context: str) -> str:
 """Assess confidence level of the generated answer."""
 # High confidence: answer contains citations and context is rich
 citation_count = answer.count("[Source]")
 if citation_count >= 2 and len(context) > 1000:
 return "high"
 elif citation_count >= 1:
 return "medium"
 else:
 return "low"

def _generate_followups(
 self, query: str, chunks: list[RetrievalResult]
) -> list[str]:
 """Generate suggested follow-up questions."""
 followups = []

 # Extract equipment tags from results
 import re
 all_content = " ".join(c.content for c in chunks)
 equipment_tags = set(re.findall(r"\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b",
all_content))

 if equipment_tags:
 tag = list(equipment_tags)[0]

```

```

 followups.append(f"What is the maintenance history of {tag}?")
 followups.append(f"Are there any open work orders for {tag}?")

 # Generic follow-ups based on query type
 if "maintenance" in query.lower() or "repair" in query.lower():
 followups.append("What spare parts are needed?")
 followups.append("What is the recommended maintenance interval?")
 elif "inspection" in query.lower():
 followups.append("When is the next inspection due?")
 followups.append("What were the findings from the last inspection?")
 elif "procedure" in query.lower() or "sop" in query.lower():
 followups.append("What PPE is required for this procedure?")
 followups.append("What are the safety precautions?")

 return followups[:4]

def _extract_related_entities(self, chunks: list[RetrievalResult]) -> list[dict]:
 """Extract entities mentioned across retrieved chunks."""
 import re
 entities = []
 seen = set()

 all_content = " ".join(c.content for c in chunks)

 # Equipment
 for tag in re.findall(r"\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b", all_content):
 if tag not in seen:
 entities.append({"type": "equipment", "value": tag})
 seen.add(tag)

 # Regulations
 for reg in re.findall(r"\b(?:OISD|API|ISO)[-s]?\d+\b", all_content):
 if reg not in seen:
 entities.append({"type": "regulation", "value": reg})
 seen.add(reg)

 return entities[:10]

```

## backend/app/services/agents/sensor\_monitor\_agent.py

### Sensor Health Monitor Agent

3-strategy anomaly detection on every reading: (1) Threshold check against 3-band OEM limits (normal/warning/critical per ISO-10816 etc). Severity = proximity to critical. (2) Z-score:  $\text{abs}(\text{value} - \text{mean}) / \text{stdev}$  over rolling 50 readings.  $z > 2.5$  = warning,  $z > 3.5$  = anomaly. (3) Trend: splits window into older/recent halves, compares means.  $\text{Change} > 20\%$  = trending. Returns HealthStatus enum + AnomalyAlert with severity\_score 0-1.

```
"""
Sensor Health Monitor Agent
Ingests machine sensor logs, detects anomalies using multi-strategy detection,
and classifies equipment health status.
"""

import statistics
from dataclasses import dataclass, field
from enum import Enum
from datetime import datetime
from typing import Optional
import structlog

logger = structlog.get_logger()

class HealthStatus(str, Enum):
 NORMAL = "normal"
 WARNING = "warning"
 CRITICAL = "critical"
 ANOMALY = "anomaly"

@dataclass
class SensorReading:
 equipment_tag: str
 parameter: str # e.g., "vibration", "temperature", "pressure"
 value: float
 unit: str
 timestamp: datetime
 metadata: dict = field(default_factory=dict)

@dataclass
class ThresholdConfig:
 """OEM or engineering-defined limits for a parameter."""
```



```

normal_min: float
normal_max: float
warning_min: float
warning_max: float
critical_min: float
critical_max: float

```

```
@dataclass
```

```

class AnomalyAlert:
 equipment_tag: str
 parameter: str
 current_value: float
 unit: str
 status: HealthStatus
 severity_score: float # 0-1
 reason: str
 threshold_info: Optional[dict] = None
 trend_info: Optional[dict] = None
 timestamp: datetime = field(default_factory=datetime.utcnow)
 recommended_action: str = ""

```

```
class SensorHealthMonitorAgent:
```

```

 """
 Agent that monitors sensor logs and detects anomalies using:
 1. Threshold checking (OEM limits from Knowledge Graph)
 2. Statistical anomaly detection (Z-score on rolling window)
 3. Trend detection (drift/degradation over time)
 4. Pattern matching (compare to known pre-failure signatures)
 """

```

```

Default industrial thresholds (overridden by KG data when available)

```

```

DEFAULT_THRESHOLDS = {
 "vibration": ThresholdConfig(
 normal_min=0, normal_max=7.1, # ISO 10816 Group 2
 warning_min=0, warning_max=11.2,
 critical_min=0, critical_max=18.0,
),
 "temperature": ThresholdConfig(
 normal_min=20, normal_max=80,
 warning_min=10, warning_max=95,
 critical_min=0, critical_max=120,
),
 "pressure": ThresholdConfig(

```

```

 normal_min=0, normal_max=45,
 warning_min=0, warning_max=50,
 critical_min=0, critical_max=55,
),
 "current": ThresholdConfig(
 normal_min=0, normal_max=85, # % of rated
 warning_min=0, warning_max=95,
 critical_min=0, critical_max=105,
),
 "flow": ThresholdConfig(
 normal_min=80, normal_max=120, # % of design
 warning_min=60, warning_max=130,
 critical_min=40, critical_max=150,
),
}

def __init__(self):
 # Rolling history per equipment+parameter for statistical analysis
 self._history: dict[str, list[float]] = {}
 self._window_size = 50 # readings to keep for stats

def analyze_reading(
 self,
 reading: SensorReading,
 custom_threshold: Optional[ThresholdConfig] = None,
) -> AnomalyAlert | None:
 """
 Analyze a single sensor reading and return an alert if anomalous.

 Returns None if reading is NORMAL.
 """
 key = f"{reading.equipment_tag}_{reading.parameter}"

 # Update history
 if key not in self._history:
 self._history[key] = []
 self._history[key].append(reading.value)
 if len(self._history[key]) > self._window_size:
 self._history[key] = self._history[key][-self._window_size:]

 # Get threshold (custom from KG > default)
 threshold = custom_threshold or self.DEFAULT_THRESHOLDS.get(reading.parameter)

 # Run detection strategies

```

```

 status = HealthStatus.NORMAL
 reasons = []
 severity = 0.0

 # Strategy 1: Threshold check
 if threshold:
 thresh_result = self._check_threshold(reading.value, thresho
ld)

 if thresh_result[0].value != "normal":
 status = thresh_result[0]
 reasons.append(thresh_result[1])
 severity = max(severity, thresh_result[2])

 # Strategy 2: Statistical anomaly (Z-score)
 if len(self._history[key]) >= 10:
 stat_result = self._statistical_check(reading.value, self._h
istory[key])
 if stat_result:
 if stat_result[2] > severity:
 status = stat_result[0]
 reasons.append(stat_result[1])
 severity = max(severity, stat_result[2])

 # Strategy 3: Trend detection
 if len(self._history[key]) >= 20:
 trend_result = self._trend_check(self._history[key])
 if trend_result:
 reasons.append(trend_result[1])
 severity = max(severity, trend_result[2])
 if trend_result[0].value == "warning" and status.value ==
"normal":
 status = HealthStatus.WARNING

 # Only return alert if not normal
 if status == HealthStatus.NORMAL:
 return None

 return AnomalyAlert(
 equipment_tag=reading.equipment_tag,
 parameter=reading.parameter,
 current_value=reading.value,
 unit=reading.unit,
 status=status,
 severity_score=min(severity, 1.0),
 reason=" | ".join(reasons),
 threshold_info={

```

```

 "normal_max": threshold.normal_max if threshold else None
 e,
 "warning_max": threshold.warning_max if threshold else None
 one,
 "critical_max": threshold.critical_max if threshold else
 None,
 },
 trend_info=self._get_trend_info(self._history[key]),
 timestamp=reading.timestamp,
)

```

```

 def analyze_batch(self, readings: list[SensorReading]) -> list[AnomalyAlert]:
 """Analyze a batch of sensor readings and return all anomalies."""
 alerts = []
 for reading in readings:
 alert = self.analyze_reading(reading)
 if alert:
 alerts.append(alert)
 if alerts:
 logger.info(
 "Batch analysis complete",
 total_readings=len(readings),
 anomalies_found=len(alerts),
 critical=sum(1 for a in alerts if a.status == HealthStatus.CRITICAL),
)
 return alerts

 def get_equipment_health_summary(self, equipment_tag: str) -> dict:
 """Get current health summary for a piece of equipment."""
 relevant_keys = [k for k in self._history if k.startswith(equipment_tag)]
 summary = {"equipment_tag": equipment_tag, "parameters": {}}

 for key in relevant_keys:
 param = key.split("_", 1)[1] if "_" in key else key
 values = self._history[key]
 if values:
 summary["parameters"][param] = {
 "latest": values[-1],
 "mean": statistics.mean(values),
 "std": statistics.stdev(values) if len(values) > 1 else 0,
 "trend": self._calculate_trend_direction(values),
 "readings_count": len(values),
 }

 return summary

 def _check_threshold(
 self, value: float, threshold: ThresholdConfig
) -> tuple[HealthStatus, str, float]:
 """Check value against defined thresholds."""
 if value >= threshold.critical_max or value <= threshold.critical_min:
 return (
 HealthStatus.CRITICAL,
 f"CRITICAL: Value {value} exceeds critical limit "
 f"({threshold.critical_min}-{threshold.critical_max})",
 1.0,
)
 elif value >= threshold.warning_max or value <= threshold.warning_min:
 # Calculate how close to critical

```

```

 if threshold.critical_max > threshold.warning_max:
 proximity = (value - threshold.warning_max) / (
 threshold.critical_max - threshold.warning_max
)
 else:
 proximity = 0.5
 return (
 HealthStatus.WARNING,
 f"WARNING: Value {value} exceeds normal limit "
 f"({threshold.normal_min}-{threshold.normal_max})",
 0.5 + (proximity * 0.4),
)
 else:
 return (HealthStatus.NORMAL, "", 0.0)

def _statistical_check(
 self, value: float, history: list[float]
) -> tuple[HealthStatus, str, float] | None:
 """Detect statistical anomalies using Z-score."""
 if len(history) < 10:
 return None

 mean = statistics.mean(history[:-1]) # Exclude current reading
 std = statistics.stdev(history[:-1])

 if std == 0:
 return None

 z_score = abs(value - mean) / std

 if z_score > 3.5:
 return (
 HealthStatus.ANOMALY,
 f"STATISTICAL ANOMALY: Z-score={z_score:.1f} "
 f"(value={value}, mean={mean:.1f}, std={std:.2f})",
 min(z_score / 5.0, 1.0),
)
 elif z_score > 2.5:
 return (
 HealthStatus.WARNING,
 f"Statistical deviation: Z-score={z_score:.1f}",
 0.4,
)
 return None

def _trend_check(
 self, history: list[float]
) -> tuple[HealthStatus, str, float] | None:
 """Detect degradation trends (steadily increasing/decreasing)."""
 if len(history) < 20:
 return None

 # Compare recent half vs older half
 midpoint = len(history) // 2
 older_mean = statistics.mean(history[:midpoint])
 recent_mean = statistics.mean(history[midpoint:])

 if older_mean == 0:
 return None

 change_pct = ((recent_mean - older_mean) / abs(older_mean)) * 100

 if abs(change_pct) > 20:
 direction = "increasing" if change_pct > 0 else "decreasing"
 return (
 HealthStatus.WARNING,
 f"TREND: Parameter {direction} by {abs(change_pct):.1f}% "

```

```

 f"(older avg: {older_mean:.1f}, recent avg: {recent_mean:.1f})",
 min(abs(change_pct) / 50.0, 0.8),
)
 return None

def _get_trend_info(self, history: list[float]) -> dict:
 """Calculate trend metadata."""
 if len(history) < 5:
 return {"direction": "insufficient_data"}

 recent = history[-5:]
 direction = self._calculate_trend_direction(recent)
 rate = (recent[-1] - recent[0]) / max(len(recent) - 1, 1)

 return {
 "direction": direction,
 "rate_per_reading": round(rate, 4),
 "last_5_values": [round(v, 2) for v in recent],
 }

def _calculate_trend_direction(self, values: list[float]) -> str:
 """Determine if values are trending up, down, or stable."""
 if len(values) < 3:
 return "stable"

 # Simple linear regression slope
 n = len(values)
 x_mean = (n - 1) / 2
 y_mean = statistics.mean(values)

 numerator = sum((i - x_mean) * (v - y_mean) for i, v in enumerate(values))
 denominator = sum((i - x_mean) ** 2 for i in range(n))

 if denominator == 0:
 return "stable"

 slope = numerator / denominator
 # Normalize slope relative to mean
 relative_slope = slope / max(abs(y_mean), 0.001)

 if relative_slope > 0.02:
 return "increasing"
 elif relative_slope < -0.02:
 return "decreasing"
 else:
 return "stable"

```

```

`backend/app/services/agents/fault_diagnosis_agent.py`

Fault Diagnosis & Suggestion Agent

> On anomaly: (1) Pattern library lookup - 15+ failure mode hypotheses mapped by parameter (vibration->bearing/misalignment/imbalance, temp->cooling/overload, pressure->blockage/leak). (2) Knowledge Graph query for equipment history - confidence boosted +15% when history matches. (3) LLM reasoning via Ollama for additional root causes. (4) Action generation from category-mapped library (mechanical/process/instrumentation) with priority, effort, parts, SOP refs, safety precautions.

```python
"""
Fault Diagnosis & Suggestion Agent
When the Sensor Monitor detects an anomaly, this agent determines probable cause
and recommends corrective actions by reasoning across the Knowledge Graph.

```

```

"""

import httpx
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional
import structlog

from app.config import settings
from app.services.agents.sensor_monitor_agent import AnomalyAlert, HealthStatus
from app.services.knowledge_graph.neo4j_client import Neo4jClient

logger = structlog.get_logger()

@dataclass
class DiagnosisHypothesis:
    cause: str
    confidence: float # 0-1
    evidence: list[str] # Supporting references
    category: str # "mechanical", "electrical", "process", "instrumentation"

@dataclass
class CorrectionAction:
    action: str
    priority: str # "immediate", "short_term", "long_term"
    estimated_effort: str # "1h", "4h", "1 day", etc.
    parts_needed: list[str] = field(default_factory=list)
    sop_reference: str = ""
    safety_precautions: list[str] = field(default_factory=list)

@dataclass
class DiagnosisReport:
    equipment_tag: str
    alert_summary: str
    hypotheses: list[DiagnosisHypothesis]
    recommended_actions: list[CorrectionAction]
    similar_past_incidents: list[dict]
    confidence_level: str # "high", "medium", "low"
    generated_at: datetime = field(default_factory=datetime.utcnow)
    references: list[str] = field(default_factory=list)

class FaultDiagnosisAgent:
    """
    Diagnoses equipment faults by:
    1. Querying Knowledge Graph for equipment history and similar failures
    2. Cross-referencing OEM troubleshooting guides
    3. Using LLM reasoning (Ishikawa/5-Why) for root cause analysis
    4. Generating ranked corrective actions with SOP references
    """

    # Common failure mode patterns (used when KG data is limited)
    FAILURE_PATTERNS = {
        "vibration": {
            "high_value": [
                DiagnosisHypothesis(
                    cause="Bearing degradation (inner/outer race wear)",
                    confidence=0.75,
                    evidence=["Most common cause of vibration increase in rotating equipment"],
                    category="mechanical",
                ),
                DiagnosisHypothesis(
                    cause="Misalignment (angular or parallel)",
                    confidence=0.65,

```

```

        evidence=["Often occurs after maintenance involving coupling disconnect"],
        category="mechanical",
    ),
    DiagnosisHypothesis(
        cause="Imbalance (fouling, erosion, or loose component)",
        confidence=0.55,
        evidence=["Check for deposit buildup or missing balance weight"],
        category="mechanical",
    ),
    DiagnosisHypothesis(
        cause="Looseness (structural or rotating)",
        confidence=0.45,
        evidence=["Check foundation bolts and bearing housing"],
        category="mechanical",
    ),
],
"trending_up": [
    DiagnosisHypothesis(
        cause="Progressive bearing wear – lubrication degradation likely",
        confidence=0.7,
        evidence=["Gradual increase pattern typical of lube contamination"],
        category="mechanical",
    ),
],
},
"temperature": {
    "high_value": [
        DiagnosisHypothesis(
            cause="Cooling system insufficiency (fouled cooler or low coolant flow)",
            confidence=0.7,
            evidence=["Check heat exchanger pressure drop and coolant level"],
            category="process",
        ),
        DiagnosisHypothesis(
            cause="Bearing overheating (lubrication failure)",
            confidence=0.65,
            evidence=["Correlate with vibration data for confirmation"],
            category="mechanical",
        ),
        DiagnosisHypothesis(
            cause="Overloading (operating above design capacity)",
            confidence=0.5,
            evidence=["Check current draw and flow rate vs design"],
            category="process",
        ),
    ],
},
"pressure": {
    "high_value": [
        DiagnosisHypothesis(
            cause="Downstream blockage or restriction",
            confidence=0.7,
            evidence=["Check downstream valves, filters, and line condition"],
            category="process",
        ),
        DiagnosisHypothesis(
            cause="Control valve malfunction (stuck closed)",
            confidence=0.6,
            evidence=["Check valve position feedback vs command"],
            category="instrumentation",
        ),
    ],
    "low_value": [
        DiagnosisHypothesis(
            cause="Leak in system (flange, seal, or pipe)",
            confidence=0.7,
            evidence=["Inspect for visible leaks, check system inventory"],

```



```

        category="mechanical",
    ),
    DiagnosisHypothesis(
        cause="Pump/compressor degradation (worn impeller or valves)",
        confidence=0.6,
        evidence=["Compare discharge pressure vs speed curve"],
        category="mechanical",
    ),
],
},
}

STANDARD_ACTIONS = {
    "mechanical": [
        CorrectionAction(
            action="Perform vibration spectrum analysis to confirm failure mode",
            priority="immediate",
            estimated_effort="2h",
            safety_precautions=["Lock-out/Tag-out if accessing rotating parts"],
        ),
        CorrectionAction(
            action="Inspect and sample lubrication oil (send for analysis)",
            priority="immediate",
            estimated_effort="1h",
            sop_reference="SOP-MAINT-045: Lubrication Sampling Procedure",
        ),
        CorrectionAction(
            action="Schedule bearing replacement during next available window",
            priority="short_term",
            estimated_effort="8h",
            parts_needed=["Bearing (check OEM spec)", "Seal set", "Lubricant"],
            sop_reference="SOP-MAINT-023: Bearing Replacement Procedure",
            safety_precautions=["PTW required", "Isolate and depressurize", "LOTO"],
        ),
    ],
    "process": [
        CorrectionAction(
            action="Check and clean heat exchanger / cooling system",
            priority="short_term",
            estimated_effort="4h",
            sop_reference="SOP-MAINT-067: Heat Exchanger Cleaning",
        ),
        CorrectionAction(
            action="Verify process parameters against design basis",
            priority="immediate",
            estimated_effort="1h",
        ),
    ],
    "instrumentation": [
        CorrectionAction(
            action="Calibrate sensor / verify reading with portable instrument",
            priority="immediate",
            estimated_effort="1h",
            sop_reference="SOP-INST-012: Instrument Calibration",
        ),
        CorrectionAction(
            action="Check control valve stroke and positioner",
            priority="short_term",
            estimated_effort="2h",
        ),
    ],
}

def __init__(self, neo4j_client: Optional[Neo4jClient] = None):
    self.neo4j = neo4j_client

    async def diagnose(self, alert: AnomalyAlert) -> DiagnosisReport:

```

```

"""
Generate a full diagnosis report for an anomaly alert.
Combines pattern matching, KG lookup, and LLM reasoning.
"""
logger.info(
    "Starting fault diagnosis",
    equipment=alert.equipment_tag,
    parameter=alert.parameter,
    status=alert.status.value,
)

# Step 1: Get hypotheses from pattern library
hypotheses = self._get_pattern_hypotheses(alert)

# Step 2: Enrich with Knowledge Graph data (if available)
similar_incidents = []
if self.neo4j:
    kg_context = await self._query_knowledge_graph(alert)
    similar_incidents = kg_context.get("similar_incidents", [])
    # Adjust confidence based on historical evidence
    hypotheses = self._adjust_confidence_from_history(hypotheses, kg_context)

# Step 3: Get LLM-powered deeper analysis (if Ollama available)
llm_hypotheses = await self._llm_reasoning(alert, hypotheses)
if llm_hypotheses:
    hypotheses.extend(llm_hypotheses)

# Step 4: Generate corrective actions
actions = self._generate_actions(hypotheses, alert)

# Step 5: Determine overall confidence
confidence = self._assess_overall_confidence(hypotheses, similar_incidents)

# Build report
report = DiagnosisReport(
    equipment_tag=alert.equipment_tag,
    alert_summary=(
        f"{alert.parameter} {alert.status.value} on {alert.equipment_tag}: "
        f"{alert.current_value} {alert.unit} - {alert.reason}"
    ),
    hypotheses=sorted(hypotheses, key=lambda h: h.confidence, reverse=True)[:5],
    recommended_actions=actions,
    similar_past_incidents=similar_incidents,
    confidence_level=confidence,
    references=[],
)

logger.info(
    "Diagnosis complete",
    equipment=alert.equipment_tag,
    hypotheses=len(report.hypotheses),
    actions=len(report.recommended_actions),
    confidence=confidence,
)

return report

def _get_pattern_hypotheses(self, alert: AnomalyAlert) -> list[DiagnosisHypothesis]:
    """Get initial hypotheses from the failure pattern library."""
    param_patterns = self.FAILURE_PATTERNS.get(alert.parameter, {})

    # Determine sub-pattern based on alert characteristics
    if alert.status == HealthStatus.CRITICAL:
        key = "high_value"
    elif "trending" in (alert.trend_info or {}).get("direction", ""):
        key = "trending_up"
    else:

```

```

        key = "high_value"

    hypotheses = param_patterns.get(key, [])

    # Deep copy to avoid mutating the class patterns
    return [
        DiagnosisHypothesis(
            cause=h.cause,
            confidence=h.confidence,
            evidence=list(h.evidence),
            category=h.category,
        )
        for h in hypotheses
    ]

async def _query_knowledge_graph(self, alert: AnomalyAlert) -> dict:
    """Query KG for equipment history, similar failures, and OEM data."""
    context = {
        "equipment_history": {},
        "similar_incidents": [],
        "oem_recommendations": [],
    }

    if not self.neo4j:
        return context

    try:
        # Get equipment full context
        equip_context = await self.neo4j.get_equipment_context(alert.equipment_tag)
        if equip_context:
            context["equipment_history"] = equip_context

            # Extract similar incidents
            incidents = equip_context.get("incidents", [])
            work_orders = equip_context.get("work_orders", [])
            context["similar_incidents"] = [
                {"type": "incident", "data": inc} for inc in incidents[:5]
            ] + [
                {"type": "work_order", "data": wo} for wo in work_orders[:5]
            ]

    except Exception as e:
        logger.warning("KG query failed", error=str(e))

    return context

def _adjust_confidence_from_history(
    self, hypotheses: list[DiagnosisHypothesis], kg_context: dict
) -> list[DiagnosisHypothesis]:
    """Increase confidence for hypotheses supported by historical data."""
    history_text = str(kg_context).lower()

    for hypothesis in hypotheses:
        # Check if similar cause appears in history
        cause_keywords = hypothesis.cause.lower().split()
        matches = sum(1 for kw in cause_keywords if kw in history_text)
        if matches > 2:
            hypothesis.confidence = min(hypothesis.confidence + 0.15, 0.98)
            hypothesis.evidence.append("Supported by equipment history in Knowledge Graph")

    return hypotheses

async def _llm_reasoning(
    self, alert: AnomalyAlert, existing_hypotheses: list[DiagnosisHypothesis]
) -> list[DiagnosisHypothesis]:
    """Use LLM for deeper diagnostic reasoning."""
    try:

```

```

        prompt = f"""You are an industrial equipment diagnostic expert.

Equipment: {alert.equipment_tag}
Parameter: {alert.parameter}
Current Value: {alert.current_value} {alert.unit}
Status: {alert.status.value}
Alert Reason: {alert.reason}
Trend: {alert.trend_info}

Existing hypotheses:
{chr(10).join(f'- {h.cause} (confidence: {h.confidence:.0%})' for h in existing_hypotheses)}

Based on your expertise, are there any additional root causes we should consider?
Particularly think about:
1. Recently changed conditions that could cause this
2. Interaction effects between multiple parameters
3. Less obvious causes specific to this equipment type

Provide 1-2 additional hypotheses if relevant, or confirm the existing ones are comprehensive.
Format: JSON array of objects with "cause", "confidence" (0-1), "category", "evidence" fields."""

    async with httpx.AsyncClient(timeout=60.0) as client:
        response = await client.post(
            f"{settings.ollama_base_url}/api/chat",
            json={
                "model": settings.ollama_chat_model,
                "messages": [{"role": "user", "content": prompt}],
                "stream": False,
                "format": "json",
                "options": {"temperature": 0.1, "num_predict": 500},
            },
        )

        if response.status_code == 200:
            import json
            content = response.json()["message"]["content"]
            data = json.loads(content)
            hypotheses_data = data if isinstance(data, list) else data.get("hypotheses", [])
            return [
                DiagnosisHypothesis(
                    cause=h.get("cause", ""),
                    confidence=float(h.get("confidence", 0.5)),
                    evidence=[h.get("evidence", "LLM analysis")],
                    category=h.get("category", "general"),
                )
                for h in hypotheses_data[:2]
            ]
        except Exception as e:
            logger.debug("LLM reasoning unavailable", error=str(e))

    return []

def _generate_actions(
    self, hypotheses: list[DiagnosisHypothesis], alert: AnomalyAlert
) -> list[CorrectionAction]:
    """Generate corrective actions based on top hypotheses."""
    actions = []

    # Get actions for the top hypothesis category
    if hypotheses:
        top_category = hypotheses[0].category
        category_actions = self.STANDARD_ACTIONS.get(top_category, [])
        actions.extend(category_actions)

    # Add severity-specific immediate action
    if alert.status == HealthStatus.CRITICAL:
        actions.insert(0, CorrectionAction(

```

```

        action=f"IMMEDIATE: Reduce load on {alert.equipment_tag} or prepare for controlled shutdown",
        priority="immediate",
        estimated_effort="15min",
        safety_precautions=[
            "Notify control room",
            "Alert maintenance team on standby",
            "Verify backup equipment readiness",
        ],
    ))

    return actions

def _assess_overall_confidence(
    self, hypotheses: list[DiagnosisHypothesis], similar_incidents: list
) -> str:
    """Assess overall diagnosis confidence."""
    if not hypotheses:
        return "low"

    top_confidence = hypotheses[0].confidence if hypotheses else 0
    has_history = len(similar_incidents) > 0

    if top_confidence > 0.7 and has_history:
        return "high"
    elif top_confidence > 0.5:
        return "medium"
    else:
        return "low"

```

backend/app/services/agents/predictive_maintenance_agent.py

Predictive Maintenance Agent

Two RUL prediction methods: (1) Trend extrapolation: linear regression on readings, $\text{days_to_failure} = (\text{limit} - \text{current}) / \text{slope} / 24$, CI widened by $(1 - R^2) * 2$. (2) Weibull reliability: $e^{-(\text{hours} / \text{MTBF})^2}$ with $\beta = 2$ wear-out mode. Cost/benefit analysis per equipment type (pump: \$2.5K planned vs \$45K unplanned). Generates MaintenanceRecommendation with urgency (immediate/advance_pm/defer_ok).

```

"""
Predictive Maintenance Agent
Uses historical failure patterns + current sensor trends to predict
WHEN equipment will likely fail (Remaining Useful Life estimation).
"""

import math
import statistics
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Optional
import structlog

from app.services.agents.sensor_monitor_agent import HealthStatus

```

```

logger = structlog.get_logger()

@dataclass
class RULEstimate:
    equipment_tag: str
    parameter: str
    estimated_days_to_failure: float
    confidence_interval: tuple[float, float] # (lower, upper) in days
    risk_score: float # 0-1 (probability of failure before next scheduled PM)
    method: str # "trend_extrapolation", "weibull", "pattern_match"
    current_health_pct: float # 0-100
    degradation_rate: float # units per day

@dataclass
class MaintenanceRecommendation:
    equipment_tag: str
    action: str
    recommended_date: datetime
    urgency: str # "advance_pm", "schedule_new", "defer_ok", "immediate"

    cost_of_early_intervention: float
    cost_of_unplanned_failure: float
    confidence: str
    reasoning: str

@dataclass
class PredictiveReport:
    equipment_tag: str
    rul_estimates: list[RULEstimate]
    overall_health_score: float # 0-100
    next_scheduled_pm: Optional[datetime]
    maintenance_recommendations: list[MaintenanceRecommendation]
    risk_summary: str
    generated_at: datetime = field(default_factory=datetime.utcnow)

class PredictiveMaintenanceAgent:
    """
    Predicts equipment failure timing using:
    1. Trend extrapolation (project when parameter hits critical limit)
    2. Weibull reliability analysis (statistical failure distribution)

```

```

3. Pattern matching (compare degradation curve to historical failure
s)
"""

# Typical MTBF data for common industrial equipment (hours)
EQUIPMENT_MTBFS = {
    "pump": 25000,
    "compressor": 30000,
    "motor": 40000,
    "valve": 50000,
    "heat_exchanger": 60000,
    "bearing": 20000,
}

# Cost estimates for cost/benefit analysis
COST_ESTIMATES = {
    "planned_maintenance": {
        "pump": 2500,
        "compressor": 8000,
        "motor": 3000,
        "valve": 1500,
    },
    "unplanned_failure": {
        "pump": 45000,          # Parts + labor + downtime
        "compressor": 150000,
        "motor": 35000,
        "valve": 25000,
    },
}

def __init__(self):
    self._failure_history: dict[str, list[dict]] = {} # equipment_tag -> past failures

def register_failure_history(self, equipment_tag: str, failures: list[dict]):
    """Register historical failure data for an equipment."""
    self._failure_history[equipment_tag] = failures

def predict(
    self,
    equipment_tag: str,
    parameter_history: dict[str, list[float]], # param_name -> readings
    critical_limits: dict[str, float], # param_name -> critical threshold

```

```

        next_scheduled_pm: Optional[datetime] = None,
        equipment_type: str = "pump",
        hours_since_last_maintenance: float = 0,
    ) -> PredictiveReport:
        """
        Generate a full predictive maintenance report.

        Args:
            equipment_tag: Equipment identifier
            parameter_history: Recent readings per parameter
            critical_limits: Threshold that indicates failure
            next_scheduled_pm: When the next PM is currently scheduled
            equipment_type: For cost/MTBF lookups
            hours_since_last_maintenance: Operating hours since last major maintenance
        """
        logger.info("Running predictive analysis", equipment=equipment_tag)

        rul_estimates = []

        # Estimate RUL for each monitored parameter
        for param, readings in parameter_history.items():
            if len(readings) < 10:
                continue

            limit = critical_limits.get(param)
            if limit is None:
                continue

            # Method 1: Trend extrapolation
            trend_rul = self._trend_extrapolation(readings, limit, param)

            if trend_rul:
                rul_estimates.append(RULEstimate(
                    equipment_tag=equipment_tag,
                    parameter=param,
                    estimated_days_to_failure=trend_rul["days"],
                    confidence_interval=trend_rul["ci"],
                    risk_score=trend_rul["risk"],
                    method="trend_extrapolation",
                    current_health_pct=trend_rul["health_pct"],
                    degradation_rate=trend_rul["rate"],
                ))

        # Method 2: Weibull reliability (if failure history available)

```



```

        weibull_rul = self._weibull_estimate(
            equipment_tag, equipment_type, hours_since_last_maintenance
        )
        if weibull_rul:
            rul_estimates.append(weibull_rul)

        # Calculate overall health score
        overall_health = self._calculate_overall_health(rul_estimates)

        # Generate maintenance recommendations
        recommendations = self._generate_recommendations(
            equipment_tag, rul_estimates, next_scheduled_pm, equipment_t
ype
        )

        # Risk summary
        risk_summary = self._generate_risk_summary(rul_estimates, next_s
cheduled_pm)

        return PredictiveReport(
            equipment_tag=equipment_tag,
            rul_estimates=rul_estimates,
            overall_health_score=overall_health,
            next_scheduled_pm=next_scheduled_pm,
            maintenance_recommendations=recommendations,
            risk_summary=risk_summary,
        )

    def _trend_extrapolation(
        self, readings: list[float], critical_limit: float, param: str
    ) -> Optional[dict]:
        """Extrapolate current trend to estimate when limit will be reac
hed."""
        if len(readings) < 10:
            return None

        n = len(readings)
        # Linear regression
        x_mean = (n - 1) / 2
        y_mean = statistics.mean(readings)

        numerator = sum((i - x_mean) * (v - y_mean) for i, v in enumerat
e(readings))
        denominator = sum((i - x_mean) ** 2 for i in range(n))

        if denominator == 0:

```

```

        return None

    slope = numerator / denominator # units per reading
    intercept = y_mean - slope * x_mean

    if slope <= 0 and readings[-1] < critical_limit:
        # Not degrading toward limit
        return None

    # Current value and distance to limit
    current = readings[-1]
    distance_to_limit = critical_limit - current

    if slope == 0:
        return None

    # Readings to failure
    readings_to_failure = distance_to_limit / slope if slope != 0 else float("inf")

    if readings_to_failure < 0:
        # Already past limit
        readings_to_failure = 0

    # Assume 1 reading per hour (adjust based on actual frequency)
    hours_to_failure = readings_to_failure
    days_to_failure = hours_to_failure / 24

    # Confidence interval (based on R2 of linear fit)
    residuals = [readings[i] - (slope * i + intercept) for i in range(n)]

    ss_res = sum(r**2 for r in residuals)
    ss_tot = sum((v - y_mean)**2 for v in readings)
    r_squared = 1 - (ss_res / ss_tot) if ss_tot > 0 else 0

    # Wider CI for lower R2
    ci_factor = 1 + (1 - r_squared) * 2
    ci_lower = max(0, days_to_failure / ci_factor)
    ci_upper = days_to_failure * ci_factor

    # Health percentage
    health_pct = max(0, min(100, (1 - current / critical_limit) * 100))

    # Risk score: how likely to fail before next scheduled PM
    risk = 1.0 if days_to_failure <= 0 else min(1.0, 7 / max(days_to

```

```

_failure, 0.1))

    return {
        "days": round(days_to_failure, 1),
        "ci": (round(ci_lower, 1), round(ci_upper, 1)),
        "risk": round(risk, 2),
        "health_pct": round(health_pct, 1),
        "rate": round(slope, 4),
    }

def _weibull_estimate(
    self, equipment_tag: str, equipment_type: str, hours_in_service:
float
) -> Optional[RULEstimate]:
    """Estimate RUL using Weibull distribution parameters."""
    mtbf = self.EQUIPMENT_MTBf.get(equipment_type)
    if not mtbf:
        return None

    # Simplified Weibull with shape parameter beta=2 (wear-out mode)
    beta = 2.0
    eta = mtbf # Characteristic life

    # Reliability at current age
    reliability = math.exp(-((hours_in_service / eta) ** beta))

    # Estimated remaining life (conditional)
    if reliability <= 0.01:
        remaining_hours = 0
    else:
        # Mean residual life approximation
        remaining_hours = eta * math.gamma(1 + 1/beta) - hours_in_se
rvice
        remaining_hours = max(0, remaining_hours)

    days_remaining = remaining_hours / 24

    # Confidence interval
    ci_lower = days_remaining * 0.6
    ci_upper = days_remaining * 1.5

    # Risk score based on reliability
    risk_score = 1 - reliability

    return RULEstimate(
        equipment_tag=equipment_tag,

```

```

        parameter="reliability",
        estimated_days_to_failure=round(days_remaining, 1),
        confidence_interval=(round(ci_lower, 1), round(ci_upper,
1))),

        risk_score=round(risk_score, 2),
        method="weibull",
        current_health_pct=round(reliability * 100, 1),
        degradation_rate=0,
    )

    def _calculate_overall_health(self, rul_estimates: list[RULEstimate
e]) -> float:
        """Calculate overall equipment health from individual estimate
s."""
        if not rul_estimates:
            return 100.0

        # Use the worst (lowest) health score
        health_scores = [r.current_health_pct for r in rul_estimates]
        return min(health_scores)

    def _generate_recommendations(
        self,
        equipment_tag: str,
        rul_estimates: list[RULEstimate],
        next_pm: Optional[datetime],
        equipment_type: str,
    ) -> list[MaintenanceRecommendation]:
        """Generate maintenance schedule recommendations."""
        recommendations = []
        now = datetime.utcnow()

        # Find the most critical RUL
        if not rul_estimates:
            return recommendations

        worst_rul = min(rul_estimates, key=lambda r: r.estimated_days_to
_failure)
        estimated_failure_date = now + timedelta(days=worst_rul.estimate
d_days_to_failure)

        # Cost estimates
        planned_cost = self.COST_ESTIMATES["planned_maintenance"].get(eq
uipment_type, 5000)
        failure_cost = self.COST_ESTIMATES["unplanned_failure"].get(equi
pment_type, 50000)

```

```

        if worst_rul.estimated_days_to_failure <= 3:
            recommendations.append(MaintenanceRecommendation(
                equipment_tag=equipment_tag,
                action=f"URGENT: Schedule immediate maintenance on {equipment_tag}. "
                    f"Estimated failure within {worst_rul.estimated_days_to_failure:.0f} days.",
                recommended_date=now + timedelta(days=1),
                urgency="immediate",
                cost_of_early_intervention=planned_cost,
                cost_of_unplanned_failure=failure_cost,
                confidence=worst_rul.method,
                reasoning=f"Parameter '{worst_rul.parameter}' degradation rate "
                    f"indicates failure by {estimated_failure_date.strftime('%d-%b-%Y')}",
            ))
        elif next_pm and estimated_failure_date < next_pm:
            days_to_advance = (next_pm - estimated_failure_date).days
            recommendations.append(MaintenanceRecommendation(
                equipment_tag=equipment_tag,
                action=f"Advance scheduled PM by {days_to_advance} days. "
                    f"Current schedule ({next_pm.strftime('%d-%b')}) is after predicted failure.",
                recommended_date=estimated_failure_date - timedelta(days=3),
                urgency="advance_pm",
                cost_of_early_intervention=planned_cost,
                cost_of_unplanned_failure=failure_cost,
                confidence=worst_rul.method,
                reasoning=f"Predicted failure: {estimated_failure_date.strftime('%d-%b-%Y')}, "
                    f"next PM: {next_pm.strftime('%d-%b-%Y')}. Gap = {days_to_advance} days.",
            ))
        elif worst_rul.risk_score < 0.2:
            recommendations.append(MaintenanceRecommendation(
                equipment_tag=equipment_tag,
                action=f"Continue monitoring. {equipment_tag} health is acceptable.",
                recommended_date=next_pm or (now + timedelta(days=30)),
                urgency="defer_ok",
                cost_of_early_intervention=planned_cost,
                cost_of_unplanned_failure=failure_cost,

```

```

        confidence=worst_rul.method,
        reasoning=f"RUL = {worst_rul.estimated_days_to_failure:.
0f} days, "
                f"risk score = {worst_rul.risk_score:.0%}. With
in acceptable limits.",
            ))

    return recommendations

def _generate_risk_summary(
    self, rul_estimates: list[RULEstimate], next_pm: Optional[dateti
me]
) -> str:
    """Generate a human-readable risk summary."""
    if not rul_estimates:
        return "Insufficient data for prediction. Continue monitorin
g."

    worst = min(rul_estimates, key=lambda r: r.estimated_days_to_fai
lure)

    if worst.estimated_days_to_failure <= 7:
        return (
            f"HIGH RISK: Equipment predicted to fail within {worst.e
stimated_days_to_failure:.0f} days "
            f"based on {worst.parameter} degradation ({worst.metho
d}). "
            f"Immediate maintenance planning recommended."
        )
    elif worst.estimated_days_to_failure <= 30:
        return (
            f"MODERATE RISK: Estimated {worst.estimated_days_to_fai
lure:.0f} days remaining life. "
            f"Plan maintenance within 2 weeks."
        )
    else:
        return (
            f"LOW RISK: Estimated {worst.estimated_days_to_failure:.
0f} days remaining life. "
            f"Continue routine monitoring."
        )

```

backend/app/services/agents/shift_handover_agent.py

Shift Handover Agent

Collects all events from shift window (alarms, maintenance, operator actions). Categorizes by type/severity. Watch item detection: equipment with 2+ alarms = recurring issue. Executive summary via LLM (Ollama) or data-driven fallback. Formatted report with sections: summary, critical items, alarm stats, WIP, watch items, upcoming activities.

```
"""
Shift Handover Agent
Automatically generates comprehensive shift handover reports by summarizing
all events, alarms, actions, and pending items from a shift period.
"""

import httpx
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Optional
import structlog

from app.config import settings
from app.services.agents.sensor_monitor_agent import AnomalyAlert, HealthStatus

logger = structlog.get_logger()

@dataclass
class ShiftEvent:
    timestamp: datetime
    event_type: str # "alarm", "maintenance", "process_deviation", "safety", "operator_action"
    severity: str # "critical", "high", "medium", "low", "info"
    equipment_tag: str
    description: str
    status: str # "resolved", "pending", "in_progress"
    action_taken: str = ""
    assigned_to: str = ""

@dataclass
class HandoverSection:
    title: str
    items: list[dict]
    priority: str # "critical", "high", "medium", "low"

@dataclass
```

```

class HandoverReport:
    shift_start: datetime
    shift_end: datetime
    shift_type: str # "day", "night", "evening"
    generated_at: datetime = field(default_factory=datetime.utcnow)

    # Summary
    executive_summary: list[str] = field(default_factory=list)
    critical_items: list[dict] = field(default_factory=list)

    # Detailed sections
    alarms_summary: dict = field(default_factory=dict)
    equipment_health_changes: list[dict] = field(default_factory=list)
    work_in_progress: list[dict] = field(default_factory=list)
    completed_activities: list[dict] = field(default_factory=list)
    upcoming_activities: list[dict] = field(default_factory=list)
    watch_items: list[dict] = field(default_factory=list)

    # Stats
    total_alarms: int = 0
    critical_alarms: int = 0
    maintenance_actions: int = 0

class ShiftHandoverAgent:
    """
    Generates structured shift handover reports by:
    1. Collecting all events from the shift period
    2. Categorizing and prioritizing
    3. Summarizing using LLM
    4. Formatting for incoming team
    """

    def __init__(self):
        self._events: list[ShiftEvent] = []

    def log_event(self, event: ShiftEvent):
        """Log a shift event for inclusion in handover report."""
        self._events.append(event)

    def log_alarm(
        self,
        alert: AnomalyAlert,
        action_taken: str = "",
        resolved: bool = False,
    ):

```



```

        """Convert an anomaly alert to a shift event."""
        self._events.append(ShiftEvent(
            timestamp=alert.timestamp,
            event_type="alarm",
            severity="critical" if alert.status == HealthStatus.CRITICAL
else "high",
            equipment_tag=alert.equipment_tag,
            description=f"{alert.parameter}: {alert.current_value} {aler
t.unit} - {alert.reason}",
            status="resolved" if resolved else "pending",
            action_taken=action_taken,
        ))

    async def generate_report(
        self,
        shift_start: datetime,
        shift_end: datetime,
        upcoming_pm: list[dict] | None = None,
    ) -> HandoverReport:
        """Generate the full shift handover report."""
        logger.info(
            "Generating handover report",
            shift_start=shift_start.isoformat(),
            shift_end=shift_end.isoformat(),
        )

        # Filter events to this shift
        shift_events = [
            e for e in self._events
            if shift_start <= e.timestamp <= shift_end
        ]

        # Determine shift type
        hour = shift_start.hour
        shift_type = "day" if 6 <= hour < 14 else ("evening" if 14 <= ho
ur < 22 else "night")

        # Categorize events
        alarms = [e for e in shift_events if e.event_type == "alarm"]
        maintenance = [e for e in shift_events if e.event_type == "maint
enance"]
        deviations = [e for e in shift_events if e.event_type == "proces
s_deviation"]
        safety = [e for e in shift_events if e.event_type == "safety"]

        # Build alarm summary

```

```

alarms_summary = {
    "total": len(alarms),
    "critical": sum(1 for a in alarms if a.severity == "critical"),
    "high": sum(1 for a in alarms if a.severity == "high"),
    "resolved": sum(1 for a in alarms if a.status == "resolved"),
    "pending": sum(1 for a in alarms if a.status == "pending"),
}

# Critical items (unresolved critical/high severity)
critical_items = [
    {
        "equipment": e.equipment_tag,
        "issue": e.description,
        "status": e.status,
        "action": e.action_taken or "NEEDS ATTENTION",
        "time": e.timestamp.strftime("%H:%M"),
    }
    for e in shift_events
    if e.severity in ("critical", "high") and e.status != "resolved"
]

# Equipment health changes
equipment_changes = self._summarize_equipment_changes(shift_events)

# Work in progress
wip = [
    {
        "equipment": e.equipment_tag,
        "activity": e.description,
        "status": e.status,
        "assigned_to": e.assigned_to,
    }
    for e in shift_events
    if e.status == "in_progress"
]

# Completed activities
completed = [
    {
        "equipment": e.equipment_tag,
        "activity": e.description,
        "completed_at": e.timestamp.strftime("%H:%M"),
    }
    for e in shift_events
    if e.status == "completed"
]

```

```

    }
    for e in shift_events
        if e.status == "resolved" and e.event_type == "maintenance"
    ]

    # Watch items (trending parameters, recurring alarms)
    watch_items = self._identify_watch_items(shift_events)

    # Generate executive summary
    executive_summary = await self._generate_summary(
        shift_events, alarms_summary, critical_items, shift_type
    )

    report = HandoverReport(
        shift_start=shift_start,
        shift_end=shift_end,
        shift_type=shift_type,
        executive_summary=executive_summary,
        critical_items=critical_items,
        alarms_summary=alarms_summary,
        equipment_health_changes=equipment_changes,
        work_in_progress=wip,
        completed_activities=completed,
        upcoming_activities=upcoming_pm or [],
        watch_items=watch_items,
        total_alarms=len(alarms),
        critical_alarms=alarms_summary["critical"],
        maintenance_actions=len(maintenance),
    )

    logger.info(
        "Handover report generated",
        events=len(shift_events),
        critical_items=len(critical_items),
        watch_items=len(watch_items),
    )

    return report

def format_report_text(self, report: HandoverReport) -> str:
    """Format the handover report as readable text."""
    lines = []
    lines.append("=" * 70)
    lines.append(f"  SHIFT HANDOVER REPORT")
    lines.append(f"  Shift: {report.shift_type.upper()} "
                f"({report.shift_start.strftime('%d-%b %H:%M')} to

```

```

"
        f"{report.shift_end.strftime('%d-%b %H:%M'))}")
    lines.append(f"    Generated: {report.generated_at.strftime('%d-%b
-%Y %H:%M'))}")
    lines.append("=" * 70)

    # Executive Summary
    lines.append("\n    EXECUTIVE SUMMARY:")
    for item in report.executive_summary:
        lines.append(f"        • {item}")

    # Critical Items
    if report.critical_items:
        lines.append(f"\n    △ CRITICAL ITEMS REQUIRING IMMEDIATE ATT
ENTION ({len(report.critical_items)}):")
        for item in report.critical_items:
            lines.append(f"        [{item['time']}] {item['equipment']}:
{item['issue']}")
            lines.append(f"                Status: {item['status']} | A
ction: {item['action']}")

    # Alarm Summary
    lines.append(f"\n    ALARM SUMMARY:")
    lines.append(f"        Total: {report.alarms_summary.get('total',
0)} | "
                f"Critical: {report.alarms_summary.get('critical',
0)} | "
                f"Resolved: {report.alarms_summary.get('resolved',
0)} | "
                f"Pending: {report.alarms_summary.get('pending',
0)}")

    # Work in Progress
    if report.work_in_progress:
        lines.append(f"\n    WORK IN PROGRESS ({len(report.work_in_pro
gress)}):")
        for wip in report.work_in_progress:
            lines.append(f"        • {wip['equipment']}: {wip['activit
y']}")
            if wip.get('assigned_to'):
                lines.append(f"                Assigned to: {wip['assigned_t
o']}")

    # Watch Items
    if report.watch_items:
        lines.append(f"\n    WATCH ITEMS (monitor closely):")

```

```

        for item in report.watch_items:
            lines.append(f"    • {item['equipment']}: {item['reason']}\n'"]})

# Upcoming
if report.upcoming_activities:
    lines.append(f"\n  UPCOMING ACTIVITIES (next 12 hours):")
    for act in report.upcoming_activities[:5]:
        lines.append(f"    • {act.get('description', act)}")

lines.append("\n" + "=" * 70)
return "\n".join(lines)

def _summarize_equipment_changes(self, events: list[ShiftEvent]) -> list[dict]:
    """Identify equipment whose health status changed during the shift."""
    equipment_events: dict[str, list[ShiftEvent]] = {}
    for e in events:
        equipment_events.setdefault(e.equipment_tag, []).append(e)

    changes = []
    for tag, tag_events in equipment_events.items():
        severities = [e.severity for e in tag_events]
        if "critical" in severities or "high" in severities:
            changes.append({
                "equipment": tag,
                "events_count": len(tag_events),
                "worst_severity": "critical" if "critical" in severities else "high",
                "latest_status": tag_events[-1].status,
            })

    return changes

def _identify_watch_items(self, events: list[ShiftEvent]) -> list[dict]:
    """Identify items that need continued monitoring."""
    watch = []

    # Recurring alarms (same equipment, multiple times)
    equipment_alarm_count: dict[str, int] = {}
    for e in events:
        if e.event_type == "alarm":
            equipment_alarm_count[e.equipment_tag] = (
                equipment_alarm_count.get(e.equipment_tag, 0) + 1

```

```

    )

    for tag, count in equipment_alarm_count.items():
        if count >= 2:
            watch.append({
                "equipment": tag,
                "reason": f"Recurring alarms ({count} times this shift) – may indicate developing issue",
            })

    # Unresolved items
    pending = [e for e in events if e.status == "pending" and e.severity in ("high", "medium")]
    for e in pending:
        watch.append({
            "equipment": e.equipment_tag,
            "reason": f"Unresolved: {e.description}",
        })

    return watch

async def _generate_summary(
    self,
    events: list[ShiftEvent],
    alarms: dict,
    critical: list,
    shift_type: str,
) -> list[str]:
    """Generate executive summary bullets, using LLM if available.

    e."""
    # Build summary from data (works without LLM)
    summary = []

    if not events:
        summary.append("Quiet shift – no significant events or alarms.")
        return summary

    # Alarm overview
    if alarms["critical"] > 0:
        pending = alarms["pending"]
        resolved_msg = "All resolved." if pending == 0 else f"{pending} still pending."
        summary.append(
            f"{alarms['critical']} critical alarm(s) occurred. {resolved_msg}"
        )

```

```

    )
    elif alarms["total"] > 0:
        summary.append(f"{alarms['total']} alarms total, none critical. {alarms['resolved']} resolved.")
    else:
        summary.append("No alarms during this shift.")

    # Critical items
    if critical:
        tags = ", ".join(set(item["equipment"] for item in critical))

        summary.append(f"ATTENTION NEEDED: {tags}")

    # Equipment count
    unique_equipment = set(e.equipment_tag for e in events)
    summary.append(f"{len(unique_equipment)} equipment items had events logged.")

    # Try LLM for more natural summary
    try:
        event_text = "\n".join(
            f"[{e.timestamp.strftime('%H:%M')}] {e.equipment_tag}: {e.description} ({e.status})"
            for e in events[:20]
        )

        async with httpx.AsyncClient(timeout=30.0) as client:
            response = await client.post(
                f"{settings.ollama_base_url}/api/chat",
                json={
                    "model": settings.ollama_chat_model,
                    "messages": [{
                        "role": "user",
                        "content": f"Summarize this {shift_type} shift in 2-3 bullet points "
                                f"for the incoming maintenance team:\n\n{event_text}",
                    }],
                    "stream": False,
                    "options": {"temperature": 0.2, "num_predict": 200},
                },
            )
            if response.status_code == 200:
                llm_summary = response.json()["message"]["content"]
                # Add LLM summary as additional context

```

```

        for line in llm_summary.strip().split("\n"):
            line = line.strip().lstrip("•-* ")
            if line and line not in summary:
                summary.append(line)

    except Exception:
        pass # LLM summary is optional enhancement

    return summary[:5]

def clear_events(self):
    """Clear event log (after shift handover is generated)."""
    self._events.clear()

```

backend/app/services/agents/report_saver.py

Report Persistence Service

Auto-saves all agent reports to data/reports/ with naming convention: {equipment}{shift}{status}_{YYYYMMDD_HHMMSS}.json. Equipment: single tag, multi-tag (up to 3), or MULTI-Neq. Shift: auto-detected from hour (day/evening/night). Status: NORMAL/WARNING/CRITICAL/DIAGNOSIS-HIGH/PREDICTION-HIGH-RISK/HANDOVER-CRITICAL. 4 subdirectories: sensor_analysis, fault_diagnosis, predictive, shift_handover.

```

"""
Report persistence service.
Saves all agent-generated reports to disk with structured naming convention.

Naming Convention:
    {equipment_tag}_{shift}_{status}_{YYYYMMDD_HHMMSS}.json

Examples:
    P-101A_day_CRITICAL_20240701_143022.json
    C-301_night_NORMAL_20240701_230015.json
    MULTI_day_WARNING_20240702_060000.json          (multi-equipment batch)
    FLEET_day_HANDOVER_20240702_180000.json          (shift handover)
    P-101A_NA_PREDICTION_20240703_091500.json        (predictive report)

Directory structure:
    data/reports/
    ├── sensor_analysis/      # From /analyze-sensors
    ├── fault_diagnosis/      # Individual diagnosis reports
    ├── predictive/           # From /predict-maintenance
    └── shift_handover/        # From /shift-handover
"""

```



```

import json
import os
from pathlib import Path
from datetime import datetime
from typing import Any
import structlog

from app.config import settings

logger = structlog.get_logger()

REPORTS_BASE = Path(settings.upload_dir).parent / "reports"

def _sanitize(value: str) -> str:
    """Sanitize a string for use in filenames. Remove/replace unsafe characters."""
    safe = value.replace("/", "-").replace("\\", "-").replace(" ", "_")
    # Keep only alphanumeric, dash, underscore
    return "".join(c for c in safe if c.isalnum() or c in "-_").strip("_-")

def _determine_shift(timestamp: datetime | None = None) -> str:
    """Determine shift name from hour: day (06-14), evening (14-22), night (22-06)."""
    if timestamp is None:
        timestamp = datetime.utcnow()
    hour = timestamp.hour
    if 6 <= hour < 14:
        return "day"
    elif 14 <= hour < 22:
        return "evening"
    else:
        return "night"

def _determine_status(alerts: list | None = None, has_critical: bool = False) -> str:
    """Determine overall status label for filename."""
    if has_critical:
        return "CRITICAL"
    if alerts:
        severities = []
        for a in alerts:

```

```

        if hasattr(a, "status"):
            severities.append(a.status.value if hasattr(a.status, "value") else str(a.status))
        elif isinstance(a, dict):
            severities.append(a.get("status", ""))
    if "critical" in severities:
        return "CRITICAL"
    if "warning" in severities or "anomaly" in severities:
        return "WARNING"
    return "NORMAL"

```

```

class ReportSaver:

```

```

    """Persists agent reports to structured directories with naming convention."""

```

```

    def __init__(self, base_dir: str | Path | None = None):
        self.base_dir = Path(base_dir) if base_dir else REPORTS_BASE
        # Create subdirectories
        for subdir in ["sensor_analysis", "fault_diagnosis", "predictive", "shift_handover"]:
            (self.base_dir / subdir).mkdir(parents=True, exist_ok=True)

```

```

    def save_sensor_analysis(
        self,
        report_data: dict,
        equipment_tags: list[str],
        alerts: list,
        timestamp: datetime | None = None,
    ) -> str:
        """
        Save sensor analysis report.
        Name: {equipment}_{shift}_{status}_{timestamp}.json
        """
        ts = timestamp or datetime.utcnow()
        equipment = self._equipment_label(equipment_tags)
        shift = _determine_shift(ts)
        status = _determine_status(alerts)

        filename = f"{equipment}_{shift}_{status}_{ts.strftime('%Y%m%d_%H%M%S')}.json"
        filepath = self.base_dir / "sensor_analysis" / filename

        self._write(filepath, report_data)
        return str(filepath)

```

```

def save_fault_diagnosis(
    self,
    report_data: dict,
    equipment_tag: str,
    confidence_level: str,
    timestamp: datetime | None = None,
) -> str:
    """
    Save individual fault diagnosis report.
    Name: {equipment}_{shift}_{confidence}_{timestamp}.json
    """
    ts = timestamp or datetime.utcnow()
    equipment = _sanitize(equipment_tag)
    shift = _determine_shift(ts)
    status = confidence_level.upper() # HIGH / MEDIUM / LOW

    filename = f"{equipment}_{shift}_DIAGNOSIS-{status}_{ts.strftime(
('%Y%m%d_%H%M%S'))}.json"
    filepath = self.base_dir / "fault_diagnosis" / filename

    self._write(filepath, report_data)
    return str(filepath)

def save_predictive_report(
    self,
    report_data: dict,
    equipment_tag: str,
    risk_level: str,
    timestamp: datetime | None = None,
) -> str:
    """
    Save predictive maintenance report.
    Name: {equipment}_{shift}_{risk}_{timestamp}.json
    """
    ts = timestamp or datetime.utcnow()
    equipment = _sanitize(equipment_tag)
    shift = _determine_shift(ts)

    # Map health to status label
    if "HIGH RISK" in risk_level.upper():
        status = "HIGH-RISK"
    elif "MODERATE" in risk_level.upper():
        status = "MODERATE-RISK"
    else:
        status = "LOW-RISK"

```

```

        filename = f"{equipment}_{shift}_PREDICTION-{status}_{ts.strftime(
e('%Y%m%d_%H%M%S'))}.json"
        filepath = self.base_dir / "predictive" / filename

        self._write(filepath, report_data)
        return str(filepath)

def save_shift_handover(
    self,
    report_data: dict,
    shift_type: str,
    has_critical: bool,
    shift_start: datetime,
    equipment_tags: list[str] | None = None,
) -> str:
    """
    Save shift handover report.
    Name: {equipment}_{shift}_{status}_{timestamp}.json
    """

    ts = shift_start
    equipment = self._equipment_label(equipment_tags) if equipment_t
ags else "FLEET"
    shift = shift_type.lower()
    status = "CRITICAL" if has_critical else ("WARNING" if report_da
ta.get("stats", {}).get("total_alarms", 0) > 0 else "NORMAL")

    filename = f"{equipment}_{shift}_HANDOVER-{status}_{ts.strftime
('%Y%m%d_%H%M%S'))}.json"
    filepath = self.base_dir / "shift_handover" / filename

    self._write(filepath, report_data)
    return str(filepath)

def _equipment_label(self, tags: list[str]) -> str:
    """Build equipment portion of filename from tag list."""
    if not tags:
        return "UNKNOWN"
    unique = sorted(set(_sanitize(t) for t in tags if t))
    if len(unique) == 1:
        return unique[0]
    elif len(unique) <= 3:
        return "_".join(unique)
    else:
        return f"MULTI-{len(unique)}eq"

def _write(self, filepath: Path, data: dict):

```

```

"""Write report data as JSON."""
# Make datetimes serializable
serializable = json.loads(
    json.dumps(data, default=str, ensure_ascii=False)
)
with open(filepath, "w", encoding="utf-8") as f:
    json.dump(serializable, f, indent=2, ensure_ascii=False)

logger.info("Report saved", path=str(filepath))

```

backend/app/services/agents/orchestrator.py

Agent Orchestrator

Coordinates the 4-agent pipeline: process_sensor_batch() feeds readings through Monitor->Diagnosis->Handover. For each WARNING/CRITICAL alert, runs FaultDiagnosisAgent.diagnose() and logs to ShiftHandoverAgent. Also exposes get_predictive_report() and generate_shift_handover() as standalone operations.

```

"""
Agent Orchestrator – coordinates the multi-agent system.
Routes sensor data through the correct agent pipeline.
"""

from dataclasses import dataclass
from datetime import datetime
from typing import Optional
import structlog

from app.services.agents.sensor_monitor_agent import (
    SensorHealthMonitorAgent, SensorReading, AnomalyAlert, HealthStatus,
)
from app.services.agents.fault_diagnosis_agent import FaultDiagnosisAgent,
    DiagnosisReport
from app.services.agents.predictive_maintenance_agent import PredictiveM
    aintenanceAgent
from app.services.agents.shift_handover_agent import ShiftHandoverAgent,
    ShiftEvent
from app.services.agents.report_saver import ReportSaver
from app.services.knowledge_graph.neo4j_client import Neo4jClient

logger = structlog.get_logger()

@dataclass
class OrchestratorResult:

```

```

    """Combined result from multi-agent processing."""
    alerts: list[AnomalyAlert]
    diagnoses: list[DiagnosisReport]
    health_summary: dict
    actions_generated: int

class AgentOrchestrator:
    """
    Coordinates the multi-agent pipeline:

    Sensor Data → Monitor Agent → (if anomaly) → Diagnosis Agent
                                                → Predictive Agent
                                                → Handover Agent (logs ev
ent)
    """

    def __init__(self, neo4j_client: Optional[Neo4jClient] = None):
        self.monitor = SensorHealthMonitorAgent()
        self.diagnosis = FaultDiagnosisAgent(neo4j_client=neo4j_client)
        self.predictive = PredictiveMaintenanceAgent()
        self.handover = ShiftHandoverAgent()
        self._neo4j = neo4j_client

    async def process_sensor_batch(
        self, readings: list[SensorReading]
    ) -> OrchestratorResult:
        """
        Process a batch of sensor readings through the full agent pipeli
ne.

        1. Monitor Agent analyzes all readings
        2. For anomalies, Diagnosis Agent generates root cause hypothes
es
        3. All events logged for Shift Handover Agent
        """
        # Step 1: Sensor Health Monitor
        alerts = self.monitor.analyze_batch(readings)

        # Step 2: For each alert, run Fault Diagnosis
        diagnoses = []
        for alert in alerts:
            if alert.status in (HealthStatus.CRITICAL, HealthStatus.WARN
ING):
                diagnosis = await self.diagnosis.diagnose(alert)
                diagnoses.append(diagnosis)

```

```

        # Log to handover agent
        self.handover.log_alarm(
            alert,
            action_taken=(
                diagnosis.recommended_actions[0].action
                if diagnosis.recommended_actions
                else "Under investigation"
            ),
        )

    # Build health summary
    equipment_tags = set(r.equipment_tag for r in readings)
    health_summary = {}
    for tag in equipment_tags:
        health_summary[tag] = self.monitor.get_equipment_health_summ
ary(tag)

    logger.info(
        "Orchestrator batch complete",
        readings=len(readings),
        alerts=len(alerts),
        diagnoses=len(diagnoses),
    )

    return OrchestratorResult(
        alerts=alerts,
        diagnoses=diagnoses,
        health_summary=health_summary,
        actions_generated=sum(len(d.recommended_actions) for d in di
agnoses),
    )

    async def get_predictive_report(
        self,
        equipment_tag: str,
        parameter_history: dict[str, list[float]],
        critical_limits: dict[str, float],
        equipment_type: str = "pump",
        hours_in_service: float = 10000,
        next_pm: Optional[datetime] = None,
    ):
        """Run predictive maintenance analysis for specific equipmen
t."""
        return self.predictive.predict(
            equipment_tag=equipment_tag,

```

```

        parameter_history=parameter_history,
        critical_limits=critical_limits,
        next_scheduled_pm=next_pm,
        equipment_type=equipment_type,
        hours_since_last_maintenance=hours_in_service,
    )

    async def generate_shift_handover(
        self,
        shift_start: datetime,
        shift_end: datetime,
    ):
        """Generate shift handover report."""
        report = await self.handover.generate_report(shift_start, shift_
end)
        return report, self.handover.format_report_text(report)

```

frontend/next.config.js

Next.js Configuration

Proxies /api/* requests to backend at localhost:8000 via rewrites. Enables seamless frontend-backend communication without CORS issues in production.

```

/** @type {import('next').NextConfig} */
const nextConfig = {
  async rewrites() {
    return [
      {
        source: '/api/:path*',
        destination: '<http://localhost:8000/api/:path*>',
      },
    ],
  },
};

export default nextConfig;

```

frontend/src/app/layout.js

Root Layout

Minimal HTML shell with title 'AXIOM - Industrial Knowledge Intelligence'.

```

export const metadata = { title: 'AXIOM - Industrial Knowledge Intellige
nce' };

```



```
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

frontend/src/app/globals.css

Global Styles

Dark theme industrial UI. CSS variables for colors (--bg, --surface, --primary, etc). Sidebar navigation, card grid, chat interface, tables, upload zone, status badges (normal/warning/critical), spinner animation, report list items.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

:root {
  --bg: #0a0e17;
  --surface: #111827;
  --surface2: #1a2234;
  --border: #2a3444;
  --text: #e2e8f0;
  --text-muted: #8892a4;
  --primary: #3b82f6;
  --primary-hover: #2563eb;
  --green: #10b981;
  --yellow: #f59e0b;
  --red: #ef4444;
  --orange: #f97316;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  background: var(--bg);
  color: var(--text);
  min-height: 100vh;
}
```

```

.layout {
  display: flex;
  min-height: 100vh;
}

.sidebar {
  width: 240px;
  background: var(--surface);
  border-right: 1px solid var(--border);
  padding: 20px 0;
  flex-shrink: 0;
}

.sidebar h1 {
  font-size: 20px;
  font-weight: 700;
  padding: 0 20px 16px;
  border-bottom: 1px solid var(--border);
  color: var(--primary);
}

.sidebar h1 span {
  color: var(--text-muted);
  font-size: 11px;
  font-weight: 400;
  display: block;
  margin-top: 2px;
}

.nav-section {
  padding: 12px 12px 4px;
  font-size: 10px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 1px;
  color: var(--text-muted);
}

.nav-item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 8px 20px;
  cursor: pointer;
  color: var(--text-muted);
  font-size: 13px;
}

```

```

    transition: all 0.15s;
    border-left: 3px solid transparent;
}

.nav-item:hover, .nav-item.active {
    background: var(--surface2);
    color: var(--text);
    border-left-color: var(--primary);
}

.main {
    flex: 1;
    display: flex;
    flex-direction: column;
    overflow: hidden;
}

.header {
    padding: 16px 24px;
    border-bottom: 1px solid var(--border);
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.header h2 {
    font-size: 18px;
    font-weight: 600;
}

.content {
    flex: 1;
    padding: 24px;
    overflow-y: auto;
}

/* Cards */
.card-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
    gap: 16px;
    margin-bottom: 24px;
}

.card {
    background: var(--surface);

```

```

border: 1px solid var(--border);
border-radius: 8px;
padding: 20px;
}

.card h3 {
  font-size: 12px;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  color: var(--text-muted);
  margin-bottom: 8px;
}

.card .value {
  font-size: 28px;
  font-weight: 700;
}

.card .sub {
  font-size: 12px;
  color: var(--text-muted);
  margin-top: 4px;
}

/* Status badges */
.badge {
  display: inline-block;
  padding: 2px 8px;
  border-radius: 9999px;
  font-size: 11px;
  font-weight: 600;
  text-transform: uppercase;
}

.badge.normal { background: #064e3b; color: var(--green); }
.badge.warning { background: #78350f; color: var(--yellow); }
.badge.critical { background: #7f1d1d; color: var(--red); }
.badge.high { background: #7f1d1d; color: var(--red); }
.badge.medium { background: #78350f; color: var(--yellow); }
.badge.low { background: #064e3b; color: var(--green); }

/* Chat */
.chat-container {
  display: flex;
  flex-direction: column;
  height: calc(100vh - 120px);

```

```

}

.chat-messages {
  flex: 1;
  overflow-y: auto;
  padding: 16px 0;
}

.message {
  max-width: 80%;
  margin-bottom: 16px;
  padding: 12px 16px;
  border-radius: 12px;
  font-size: 14px;
  line-height: 1.5;
}

.message.user {
  background: var(--primary);
  color: white;
  margin-left: auto;
  border-bottom-right-radius: 4px;
}

.message.assistant {
  background: var(--surface2);
  border: 1px solid var(--border);
  border-bottom-left-radius: 4px;
}

.message .sources {
  margin-top: 8px;
  padding-top: 8px;
  border-top: 1px solid var(--border);
  font-size: 11px;
  color: var(--text-muted);
}

.chat-input-row {
  display: flex;
  gap: 8px;
  padding: 12px 0;
}

.chat-input-row input {
  flex: 1;

```

```

padding: 12px 16px;
background: var(--surface);
border: 1px solid var(--border);
border-radius: 8px;
color: var(--text);
font-size: 14px;
outline: none;
}

.chat-input-row input:focus {
  border-color: var(--primary);
}

.chat-input-row button {
  padding: 12px 24px;
  background: var(--primary);
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 600;
  font-size: 14px;
}

.chat-input-row button:hover {
  background: var(--primary-hover);
}

.chat-input-row button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

/* Tables */
.table-wrap {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow: hidden;
}

table {
  width: 100%;
  border-collapse: collapse;
  font-size: 13px;
}

```

```

th {
  text-align: left;
  padding: 10px 16px;
  background: var(--surface2);
  color: var(--text-muted);
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  border-bottom: 1px solid var(--border);
}

td {
  padding: 10px 16px;
  border-bottom: 1px solid var(--border);
}

tr:last-child td {
  border-bottom: none;
}

/* Upload */
.upload-zone {
  border: 2px dashed var(--border);
  border-radius: 12px;
  padding: 48px;
  text-align: center;
  cursor: pointer;
  transition: all 0.2s;
}

.upload-zone:hover {
  border-color: var(--primary);
  background: rgba(59, 130, 246, 0.05);
}

.upload-zone input { display: none; }

/* Sensor chart placeholder */
.health-bar {
  height: 8px;
  background: var(--surface2);
  border-radius: 4px;
  overflow: hidden;
  margin-top: 8px;
}

```

```

.health-bar-fill {
  height: 100%;
  border-radius: 4px;
  transition: width 0.3s;
}

/* Loading */
.spinner {
  display: inline-block;
  width: 16px;
  height: 16px;
  border: 2px solid var(--border);
  border-top-color: var(--primary);
  border-radius: 50%;
  animation: spin 0.6s linear infinite;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

/* Reports list */
.report-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 12px 16px;
  border-bottom: 1px solid var(--border);
  cursor: pointer;
  transition: background 0.15s;
}

.report-item:hover {
  background: var(--surface2);
}

.report-item .name {
  font-size: 13px;
  font-family: monospace;
}

.report-item .meta {
  font-size: 11px;
  color: var(--text-muted);
}

```


frontend/src/app/page.js

Main Application Page (All 6 Tabs)

Single-page app with sidebar navigation. 6 tabs: (1) Dashboard: graph stats, architecture diagram, API reference table. (2) ChatTab: RAG copilot with message history, citations, confidence badges, follow-up suggestions. (3) IngestTab: file upload with entity extraction results table. (4) AgentsTab: sensor analysis demo with 30 simulated readings, alert table, diagnosis hypotheses, recommended actions. (5) ReportsTab: filter by category/equipment/status, view saved report JSON. (6) GraphTab: search nodes, quick-access equipment buttons, equipment context viewer.

```
'use client';
import './globals.css';
import { useState, useRef, useEffect } from 'react';

const API = '<http://localhost:8000>';

const TABS = [
  { id: 'dashboard', label: 'Dashboard', icon: '~' },
  { id: 'chat', label: 'Knowledge Copilot', icon: '>' },
  { id: 'ingest', label: 'Document Upload', icon: '+' },
  { id: 'agents', label: 'Sensor Analysis', icon: '#' },
  { id: 'reports', label: 'Reports', icon: '=' },
  { id: 'graph', label: 'Knowledge Graph', icon: 'o' },
];

export default function Home() {
  const [tab, setTab] = useState('dashboard');
  const [health, setHealth] = useState(null);

  useEffect(() => {
    fetch(`${API}/health`).then(r => r.json()).then(setHealth).catch(() => {});
  }, []);

  return (
    <div className="layout">
      <aside className="sidebar">
        <h1>AXIOM <span>Industrial Knowledge Intelligence</span></h1>
        <div className="nav-section">Main</div>
        {TABS.map(t => (
          <div key={t.id} className={`nav-item ${tab === t.id ? 'active' : ''}`} onClick={() => setTab(t.id)}>
            <span style={{ fontFamily: 'monospace', width: 16 }}>{t.icon}</span> {t.label}
          </div>
        ))}
      </aside>
    </div>
  );
}
```

```

    </div>
  ))}
  <div className="nav-section" style={{ marginTop: 'auto', padding
Top: 24 }}>System</div>
  <div className="nav-item" style={{ color: health ? 'var(--gree
n)' : 'var(--red)' }}>
    <span style={{ fontFamily: 'monospace', width: 16 }}>*</span>
    Backend: {health ? 'Connected' : 'Offline'}
  </div>
</aside>
<main className="main">
  <div className="header">
    <h2>{TABS.find(t => t.id === tab)?.label}</h2>
    <span style={{ fontSize: 12, color: 'var(--text-muted)' }}>AXI
OM v1.0 | $0 Cost</span>
  </div>
  <div className="content">
    {tab === 'dashboard' && <DashboardTab />}
    {tab === 'chat' && <ChatTab />}
    {tab === 'ingest' && <IngestTab />}
    {tab === 'agents' && <AgentsTab />}
    {tab === 'reports' && <ReportsTab />}
    {tab === 'graph' && <GraphTab />}
  </div>
</main>
</div>
);
}

```

/* — Dashboard — */

```

function DashboardTab() {
  const [stats, setStats] = useState(null);
  useEffect(() => {
    fetch(`${API}/api/v1/graph/stats`).then(r => r.json()).then(setStat
s).catch(() => {});
  }, []);
  return (
    <>
      <div className="card-grid">
        <div className="card"><h3>Knowledge Graph Nodes</h3><div classNa
me="value">{stats?.total_nodes ?? '--'}</div><div className="sub">Across
all entity types</div></div>
        <div className="card"><h3>Relationships</h3><div className="valu
e">{stats?.total_relationships ?? '--'}</div><div className="sub">Cross-
document links</div></div>
        <div className="card"><h3>Platform Cost</h3><div className="valu

```

```
e" style={{ color: 'var(--green)' }}>$0</div><div className="sub">100% free local stack</div></div>
```

```
<div className="card"><h3>LLM Provider</h3><div className="value" style={{ fontSize: 20 }}>Ollama</div><div className="sub">Llama 3.1 8B (local)</div></div>
```

```
</div>
<div className="card" style={{ marginBottom: 16 }}>
  <h3>System Architecture</h3>
  <pre style={{ fontSize: 11, color: 'var(--text-muted)', lineHeight: 1.6, marginTop: 8, overflow: 'auto' }}>
    {`Document Upload → Ingestion Pipeline → OCR/Parsing → Entity Extraction
      → Knowledge Graph (Neo4j) + Vector Store (Qdrant)
      → Hybrid Retrieval (Vector + Graph + BM25)
      → LLM Answer Generation with Citations (Ollama - FREE)
  `}
  </pre>
</div>
```

Multi-Agent Pipeline:

Sensor Data → Monitor Agent → Fault Diagnosis → Predictive Maintenance
→ Shift Handover Report

```
t
All reports auto-saved to data/reports/ with naming convention`}
```

```
</pre>
</div>
<div className="card">
  <h3>API Endpoints</h3>
  <div className="table-wrap" style={{ marginTop: 8 }}>
    <table>
      <thead><tr><th>Endpoint</th><th>Method</th><th>Description</th></tr></thead>
      <tbody>
        {[
          ['/api/v1/ingest/document', 'POST', 'Upload & process document'],
          ['/api/v1/query/ask', 'POST', 'Ask question with RAG'],
          ['/api/v1/graph/equipment/{tag}', 'GET', 'Equipment context'],
          ['/api/v1/agents/analyze-sensors', 'POST', 'Sensor analysis pipeline'],
          ['/api/v1/agents/predict-maintenance', 'POST', 'RUL prediction'],
          ['/api/v1/agents/shift-handover', 'POST', 'Shift report'],
          ['/api/v1/agents/reports', 'GET', 'Browse saved reports'],
        ].map(([e,m,d],i) => <tr key={i}><td style={{ fontFamily: 'monospace', fontSize: 12 }}>{e}</td><td><span className="badge normal">{m}</span></td><td>{d}</td></tr>)}
        </tbody>
      </table>
    </div>
  </div>
```

```

        </div>
      </div>
    </>
  );
}

/* — Chat / RAG Copilot — */
function ChatTab() {
  const [messages, setMessages] = useState([
    { role: 'assistant', content: 'I\'m AXIOM, your Industrial Knowledge Copilot. Ask me about equipment, maintenance history, SOPs, or regulations. I\'ll answer with source citations.', sources: [] }
  ]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(false);
  const endRef = useRef(null);

  useEffect(() => { endRef.current?.scrollIntoView({ behavior: 'smooth' }); }, [messages]);

  const send = async () => {
    if (!input.trim() || loading) return;
    const q = input.trim();
    setInput('');
    setMessages(m => [...m, { role: 'user', content: q }]);
    setLoading(true);
    try {
      const res = await fetch(`${API}/api/v1/query/ask`, {
        method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ question: q, top_k: 5 }),
      });
      const data = await res.json();
      setMessages(m => [...m, {
        role: 'assistant',
        content: data.answer || data.detail || 'No answer generated.',
        sources: data.sources || [],
        confidence: data.confidence,
        followups: data.suggested_followups || [],
      }]);
    } catch (e) {
      setMessages(m => [...m, { role: 'assistant', content: `Error: ${e.message}. Is the backend running?` }]);
    }
    setLoading(false);
  };
}

```

```

return (
  <div className="chat-container">
    <div className="chat-messages">
      {messages.map((m, i) => (
        <div key={i} className={`message ${m.role}`}>
          <div>{m.content}</div>
          {m.confidence && <div style={{ marginTop: 6, fontSize: 11 }}>
            >Confidence: <span className={`badge ${m.confidence}`}>{m.confidence}</span></div>
          {m.sources?.length > 0 && (
            <div className="sources">Sources: {m.sources.map((s, j) =>
              <span key={j} style={{ marginRight: 8 }}>[{s.filename} p.{s.page_number}
                || '?'</span>)}</div>
            )}
            {m.followups?.length > 0 && (
              <div style={{ marginTop: 8, fontSize: 12 }}>
                {m.followups.map((f, j) => (
                  <span key={j} onClick={() => { setInput(f); }} style={{
                    cursor: 'pointer', color: 'var(--primary)', marginRight: 12, textDecoration: 'underline' }}>{f}</span>
                ))}
              </div>
            )}
          </div>
        )}
      )}
      {loading && <div className="message assistant"><span className="spinner" /> Searching knowledge base...</div>}
      <div ref={endRef} />
    </div>
    <div className="chat-input-row">
      <input value={input} onChange={e => setInput(e.target.value)} onKeyDown={e => e.key === 'Enter' && send()}
        placeholder="Ask about equipment, maintenance, SOPs, regulations..." />
      <button onClick={send} disabled={loading}>Ask</button>
    </div>
  </div>
);
}

/* — Document Upload — */
function IngestTab() {
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const fileRef = useRef(null);

```

```

const upload = async (file) => {
  setLoading(true); setResult(null);
  const fd = new FormData(); fd.append('file', file);
  try {
    const res = await fetch(`${API}/api/v1/ingest/document`, { method:
'POST', body: fd });
    setResult(await res.json());
  } catch (e) { setResult({ error: e.message }); }
  setLoading(false);
};

return (
  <>
    <div className="upload-zone" onClick={() => fileRef.current?.click
()}>
      <input type="file" ref={fileRef} accept=".pdf,.docx,.doc,.png,.j
pg,.jpeg,.tiff,.txt,.json,.csv" onChange={e => e.target.files[0] && uplo
ad(e.target.files[0])} />
      <div style={{ fontSize: 32, marginBottom: 8 }}>+</div>
      <div>Click to upload a document</div>
      <div style={{ fontSize: 12, color: 'var(--text-muted)', marginTo
p: 4 }}>PDF, DOCX, TXT, JSON, CSV, images (PNG/JPG/TIFF)</div>
    </div>
    {loading && <div style={{ textAlign: 'center', marginTop: 24 }}><s
pan className="spinner" /> Processing document...</div>}
    {result && (
      <div className="card" style={{ marginTop: 16 }}>
        <h3>Ingestion Result</h3>
        {result.error ? <div style={{ color: 'var(--red)' }}>{result.e
rror}</div> : (
          <>
            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1
fr 1fr', gap: 12, margin: '12px 0' }}>
              <div><strong>{result.entities_extracted}</strong> entiti
es</div>
              <div><strong>{result.relationships_found}</strong> relat
ionships</div>
              <div><strong>{result.chunks_created}</strong> chunks</di
v>
            </div>
            <div style={{ fontSize: 12, marginBottom: 8 }}>Category: <
span className="badge normal">{result.category}</span></div>
            {result.entities?.length > 0 && (
              <div className="table-wrap">
                <table>
                  <thead><tr><th>Type</th><th>Value</th><th>Confidence

```

```

</th></tr></thead>
      <tbody>{result.entities.slice(0, 15).map((e, i) => <
tr key={i}><td>{e.type}</td><td style={{ fontFamily: 'monospace'}}>{e.valu
e}</td><td>{(e.confidence * 100).toFixed(0)}%</td></tr>)}</tbody>
      </table>
    </div>
  )}
</>
)}
</div>
)}
</>
);
}

/* — Sensor Analysis — */
function AgentsTab() {
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);

  const runDemo = async () => {
    setLoading(true);
    // Generate sample degrading sensor data
    const readings = [];
    for (let i = 0; i < 30; i++) {
      readings.push({
        equipment_tag: 'P-101A', parameter: 'vibration',
        value: 3.5 + i * 0.3 + Math.random() * 0.5, unit: 'mm/s',
      });
    }
    try {
      const res = await fetch(`${API}/api/v1/agents/analyze-sensors`, {
        method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ readings }),
      });
      setResult(await res.json());
    } catch (e) { setResult({ error: e.message }); }
    setLoading(false);
  };

  return (
    <>
      <div className="card-grid">
        <div className="card" style={{ cursor: 'pointer', borderColor:
'var(--primary)' }} onClick={runDemo}>
          <h3>Run Sensor Demo</h3>

```

```

        <div className="value" style={{ fontSize: 16 }}>Simulate P-101
A Degradation</div>
        <div className="sub">30 readings: normal → warning → critical
</div>
    </div>
</div>
    {loading && <div><span className="spinner" /> Analyzing sensors th
rough multi-agent pipeline...</div>}
    {result && !result.error && (
    <div className="card-grid">
        <div className="card"><h3>Readings</h3><div className="valu
e">{result.total_readings}</div></div>
        <div className="card"><h3>Anomalies</h3><div className="valu
e" style={{ color: result.anomalies_detected > 0 ? 'var(--red)' : 'var(-
-green)' }}>{result.anomalies_detected}</div></div>
        <div className="card"><h3>Diagnoses</h3><div className="valu
e">{result.diagnoses_generated}</div></div>
        <div className="card"><h3>Actions</h3><div className="valu
e">{result.actions_suggested}</div></div>
    </div>
    {result.report_saved_to && <div className="card" style={{ marg
inBottom: 16 }}><h3>Report Saved</h3><div style={{ fontFamily: 'monospac
e', fontSize: 12, wordBreak: 'break-all' }}>{result.report_saved_to}</di
v></div>}
    {result.alerts?.length > 0 && (
        <div className="card" style={{ marginBottom: 16 }}>
            <h3>Alerts ({result.alerts.length})</h3>
            <div className="table-wrap" style={{ marginTop: 8 }}>
                <table>
                    <thead><tr><th>Equipment</th><th>Parameter</th><th>Val
ue</th><th>Status</th><th>Reason</th></tr></thead>
                    <tbody>{result.alerts.slice(0, 10).map((a, i) => (
                        <tr key={i}><td>{a.equipment_tag}</td><td>{a.paramet
er}</td><td>{a.value?.toFixed(1)} {a.unit}</td><td><span className={`bad
ge ${a.status}`}>{a.status}</span></td><td style={{ fontSize: 11, maxWidth:
300}}>{a.reason?.substring(0, 80)}</td></tr>
                    )}</tbody>
                </table>
            </div>
        </div>
    )}
    {result.diagnoses?.length > 0 && result.diagnoses.map((d, i) =
> (
        <div key={i} className="card" style={{ marginBottom: 16 }}>
            <h3>Diagnosis: {d.equipment_tag} <span className={`badge

```



```

    ${d.confidence}`}>{d.confidence} confidence</span></h3>
      <div style={{ fontSize: 13, margin: '8px 0', color: 'var(--text-muted)' }}>{d.alert_summary}</div>
      {d.hypotheses?.map((h, j) => (
        <div key={j} style={{ padding: '6px 0', borderBottom: '1px solid var(--border)', fontSize: 13 }}>
          <strong>{(h.confidence * 100).toFixed(0)}%</strong>
          {h.cause}
          <span style={{ fontSize: 11, color: 'var(--text-muted)', marginLeft: 8 }}>[{h.category}]</span>
        </div>
      ))}
      {d.recommended_actions?.length > 0 && (
        <div style={{ marginTop: 12 }}>
          <h3 style={{ marginBottom: 6 }}>Recommended Actions</h3>
          {d.recommended_actions.map((a, j) => (
            <div key={j} style={{ padding: '4px 0', fontSize: 13 }}>
              <span className={`badge ${a.priority === 'immediate' ? 'critical' : a.priority === 'short_term' ? 'warning' : 'normal'}`}>{a.priority}</span>
              <span style={{ marginLeft: 8 }}>{a.action}</span>
              {a.sop_reference && <span style={{ fontSize: 11, color: 'var(--primary)', marginLeft: 8 }}>{a.sop_reference}</span>}
            </div>
          ))}
        </div>
      )}
    </div>
  )}
</>
)
}

```

```

/* — Reports — */
function ReportsTab() {
  const [reports, setReports] = useState(null);
  const [filter, setFilter] = useState({ category: '', equipment: '', status: '' });
  const [selected, setSelected] = useState(null);

```

```

const load = async () => {
  const params = new URLSearchParams();
  if (filter.category) params.set('category', filter.category);
  if (filter.equipment) params.set('equipment', filter.equipment);
  if (filter.status) params.set('status', filter.status);
  try {
    const res = await fetch(`${API}/api/v1/agents/reports?${params}`);
    setReports(await res.json());
  } catch (e) { setReports({ error: e.message }); }
};

const openReport = async (cat, fname) => {
  try {
    const res = await fetch(`${API}/api/v1/agents/reports/${cat}/${fname}`);
    setSelected(await res.json());
  } catch (e) { setSelected({ error: e.message }); }
};

useEffect(() => { load(); }, []);

return (
  <>
    <div style={{ display: 'flex', gap: 8, marginBottom: 16 }}>
      <select value={filter.category} onChange={e => setFilter(f => ({
        ...f, category: e.target.value })))}
        style={{ padding: '8px 12px', background: 'var(--surface)', bo
rder: '1px solid var(--border)', borderRadius: 6, color: 'var(--text)',
fontSize: 13 }}>
        <option value="">All Categories</option>
        <option value="sensor_analysis">Sensor Analysis</option>
        <option value="fault_diagnosis">Fault Diagnosis</option>
        <option value="predictive">Predictive</option>
        <option value="shift_handover">Shift Handover</option>
      </select>
      <input placeholder="Equipment tag..." value={filter.equipment} o
nChange={e => setFilter(f => ({ ...f, equipment: e.target.value })))}
        style={{ padding: '8px 12px', background: 'var(--surface)', bo
rder: '1px solid var(--border)', borderRadius: 6, color: 'var(--text)',
fontSize: 13, width: 160 }} />
      <select value={filter.status} onChange={e => setFilter(f => ({
        ...f, status: e.target.value })))}
        style={{ padding: '8px 12px', background: 'var(--surface)', bo
rder: '1px solid var(--border)', borderRadius: 6, color: 'var(--text)',
fontSize: 13 }}>
        <option value="">All Statuses</option>
    </div>
  </>
)

```

```

        <option value="CRITICAL">Critical</option>
        <option value="WARNING">Warning</option>
        <option value="NORMAL">Normal</option>
    </select>
    <button onClick={load} style={{ padding: '8px 16px', background:
'var(--primary)', color: 'white', border: 'none', borderRadius: 6, curso
r: 'pointer' }}>Filter</button>
</div>
{reports?.error && <div className="card"><div style={{ color: 'var
(--red)' }}>Error: {reports.error}</div></div>}
{reports && !reports.error && (
    <div className="card">
        <h3>Saved Reports ({reports.total_reports})</h3>
        <div style={{ fontSize: 11, color: 'var(--text-muted)', margin
Bottom: 8 }}>{reports.reports_directory}</div>
        {reports.reports?.length === 0 && <div style={{ padding: 24, t
extAlign: 'center', color: 'var(--text-muted)' }}>No reports found. Run
a sensor analysis first.</div>}
        {reports.reports?.map((r, i) => (
            <div key={i} className="report-item" onClick={() => openRepo
rt(r.category, r.filename)}>
                <div>
                    <div className="name">{r.filename}</div>
                    <div className="meta">{r.category} | {r.created}</div>
                </div>
                {r.status_label && <span className={`badge ${r.status_labe
l.includes('CRITICAL') ? 'critical' : r.status_label.includes('WARNING')
? 'warning' : 'normal'}`}>{r.status_label}</span>}
            </div>
        ))}
    </div>
)}
{selected && (
    <div className="card" style={{ marginTop: 16 }}>
        <h3>Report: {selected.filename} <button onClick={() => setSele
cted(null)} style={{ float: 'right', background: 'none', border: 'none',
color: 'var(--text-muted)', cursor: 'pointer' }}>Close</button></h3>
        <pre style={{ fontSize: 11, color: 'var(--text-muted)', margin
Top: 8, overflow: 'auto', maxHeight: 500 }}>{JSON.stringify(selected.dat
a, null, 2)}</pre>
    </div>
)}
</>
);
}

```

```

/* — Knowledge Graph — */
function GraphTab() {
  const [searchTerm, setSearchTerm] = useState('');
  const [results, setResults] = useState(null);
  const [equipDetail, setEquipDetail] = useState(null);

  const search = async () => {
    if (!searchTerm.trim()) return;
    try {
      const res = await fetch(`${API}/api/v1/graph/search`, {
        method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ query: searchTerm }),
      });
      setResults(await res.json());
    } catch (e) { setResults({ error: e.message }); }
  };

  const getEquip = async (tag) => {
    try {
      const res = await fetch(`${API}/api/v1/graph/equipment/${tag}`);
      setEquipDetail(await res.json());
    } catch (e) { setEquipDetail({ error: e.message }); }
  };

  return (
    <>
      <div style={{ display: 'flex', gap: 8, marginBottom: 16 }}>
        <input value={searchTerm} onChange={e => setSearchTerm(e.target.value)} onKeyDown={e => e.key === 'Enter' && search()}
          placeholder="Search knowledge graph (equipment tag, regulation, person...)"
          style={{ flex: 1, padding: '10px 16px', background: 'var(--surface)', border: '1px solid var(--border)', borderRadius: 8, color: 'var(--text)', fontSize: 14 }} />
        <button onClick={search} style={{ padding: '10px 20px', background: 'var(--primary)', color: 'white', border: 'none', borderRadius: 8, cursor: 'pointer' }}>Search</button>
      </div>
      <div style={{ display: 'flex', gap: 8, marginBottom: 16 }}>
        {[ 'P-101A', 'PSV-3001', 'OISD-154', 'R-601' ].map(tag => (
          <button key={tag} onClick={() => getEquip(tag)} style={{ padding: '6px 12px', background: 'var(--surface)', border: '1px solid var(--border)', borderRadius: 6, color: 'var(--primary)', cursor: 'pointer', fontSize: 12 }}>{tag}</button>
        ))}
      </div>
    </>
  );
}

```

```

    {results && !results.error && (
      <div className="card" style={{ marginBottom: 16 }}>
        <h3>Search Results ({results.results?.length || 0})</h3>
        {results.results?.map((r, i) => (
          <div key={i} style={{ padding: '8px 0', borderBottom: '1px solid var(--border)', fontSize: 13 }}>
            <span className="badge normal">{r.label}</span>
            <span style={{ marginLeft: 8 }}>{JSON.stringify(r.props)}</span>
          </div>
        ))}
      </div>
    )}
    {equipDetail && !equipDetail.error && (
      <div className="card">
        <h3>Equipment Context: {equipDetail.equipment?.tag || 'N/A'}</h3>
        <pre style={{ fontSize: 11, color: 'var(--text-muted)', margin: '8px 0', overflow: 'auto', maxHeight: 400 }}>{JSON.stringify(equipDetail, null, 2)}</pre>
      </div>
    )}
    {(results?.error || equipDetail?.error) && <div className="card"><div style={{ color: 'var(--red)' }}>Error: {results?.error || equipDetail?.error}. Is the backend running with Neo4j?</div></div>}
  </>
);
}

```

scripts/generate_synthetic_docs.py

Sample Data Generator

Generates realistic industrial documents: 20 work orders (with equipment tags, failure modes, parts, standards), 10 inspection reports (thickness surveys, NDT, compliance status), 8 incident reports (3 scenarios: seal failure, PSV stuck, compressor trip - with RCA and corrective actions), 1 SOP (hot work procedure with steps, PPE, emergency procedure). Outputs JSON + plain text versions.

```

"""
Generate synthetic industrial documents for demo/testing.
Creates realistic work orders, inspection reports, SOPs, and incident reports.
"""

import json

```

```

import os
from pathlib import Path
from datetime import datetime, timedelta
import random

OUTPUT_DIR = Path("data/sample_documents")

def generate_work_orders(count: int = 20):
    """Generate synthetic maintenance work orders."""
    equipment_tags = [
        "P-101A", "P-101B", "P-201A", "C-301", "C-301B",
        "V-401", "V-402", "HX-501", "HX-502", "R-601",
        "FV-1001", "XV-1002", "PSV-3001", "TIT-4001", "FIC-2001",
    ]

    work_types = ["Corrective Maintenance", "Preventive Maintenance", "Predictive Maintenance"]
    priorities = ["Critical", "High", "Medium", "Low"]
    statuses = ["Completed", "In Progress", "Planned", "Overdue"]

    technicians = [
        "Rajesh Kumar", "Amit Sharma", "Suresh Patel", "Vikram Singh",
        "Anil Reddy", "Pradeep Joshi", "Manoj Verma", "Sanjay Gupta",
    ]

    failure_modes = [
        "Bearing failure due to inadequate lubrication",
        "Seal leakage - mechanical seal worn",
        "High vibration - misalignment detected",
        "Impeller erosion causing reduced flow",
        "Coupling damage - fatigue crack",
        "Motor overheating - winding insulation degradation",
        "Control valve sticking - stem packing worn",
        "Corrosion under insulation (CUI)",
        "Gasket failure at flange joint",
        "Instrument drift - requires recalibration",
    ]

    work_orders = []
    base_date = datetime(2022, 1, 1)

    for i in range(count):
        wo_date = base_date + timedelta(days=random.randint(0, 900))
        equipment = random.choice(equipment_tags)

```

```

wo = {
    "work_order_id": f"WO-{100000 + i}",
    "equipment_tag": equipment,
    "work_type": random.choice(work_types),
    "priority": random.choice(priorities),
    "status": random.choice(statuses),
    "date_raised": wo_date.strftime("%d-%b-%Y"),
    "date_completed": (wo_date + timedelta(days=random.randint
(1, 14))).strftime("%d-%b-%Y"),
    "assigned_to": random.choice(technicians),
    "description": random.choice(failure_modes),
    "findings": f"Inspection of {equipment} revealed {random.cho
ice(failure_modes).lower()}. "
                f"Equipment was isolated as per PTW-{random.randi
nt(1000, 9999)}. "
                f"Maintenance carried out as per SOP-MAINT-{rando
m.randint(100, 999)}. ",
    "parts_used": random.choice([
        "Bearing SKF 6205-2RS (qty: 2)",
        "Mechanical seal Type A (qty: 1)",
        "Gasket set DN150 PN40 (qty: 1)",
        "Coupling element (qty: 1)",
        "Packing rings (qty: 4)",
    ]),
    "man_hours": random.randint(2, 48),
    "applicable_standard": random.choice(["OISD-163", "API-610",
"ISO-10816", "ASME B31.3"]),
}
work_orders.append(wo)

return work_orders

def generate_inspection_reports(count: int = 10):
    """Generate synthetic inspection reports."""
    reports = []

    equipment_tags = ["V-401", "V-402", "P-101A", "HX-501", "R-601", "C-
301"]
    inspection_types = [
        "Thickness Survey", "NDT Inspection", "Internal Inspection",
        "External Visual Inspection", "Pressure Test",
    ]
    inspectors = ["M. Krishnan", "S. Banerjee", "R. Iyer", "P. Nair"]

```

```

for i in range(count):
    tag = random.choice(equipment_tags)
    insp_date = datetime(2023, 1, 1) + timedelta(days=random.randint(
0, 500))

    report = {
        "report_id": f"IR-{2023000 + i}",
        "equipment_tag": tag,
        "inspection_type": random.choice(inspection_types),
        "date": insp_date.strftime("%d-%b-%Y"),
        "inspector": random.choice(inspectors),
        "next_inspection_due": (insp_date + timedelta(days=random.ra
ndint(180, 730))).strftime("%d-%b-%Y"),
        "findings": f"Thickness measurement at location TML-{random.
randint(1,12)}: "
                        f"{random.uniform(4.0, 12.0):.1f} mm (minimum req
uired: 6.0 mm per API-510). "
                        f"Corrosion rate: {random.uniform(0.05, 0.5):.2f}
mm/year. "
                        f"{'No defects found. Fit for continued service.'
if random.random() > 0.3 else 'Localized thinning detected. Recommend re
pair or monitoring at reduced interval.'}",
        "recommendation": random.choice([
            "Continue normal service. Next inspection as schedule
d.",
            "Reduce inspection interval to 12 months.",
            "Weld repair required at location TML-5 before next turn
around.",
            "Install corrosion monitoring probe.",
            "Apply protective coating during next shutdown.",
        ]),
        "applicable_standards": ["API-510", "API-570", "OISD-128",
"ASME PCC-2"],
        "compliance_status": random.choice(["Compliant", "Compliant
with conditions", "Non-compliant - action required"]),
    }
    reports.append(report)

return reports

def generate_incident_reports(count: int = 8):
    """Generate synthetic incident/near-miss reports."""
    reports = []

    for i in range(count):

```



```

date = datetime(2022, 6, 1) + timedelta(days=random.randint(0, 7
00))

scenarios = [
    {
        "title": "Pump Seal Failure - Hydrocarbon Release",
        "equipment": "P-101A",
        "severity": "High",
        "description": "Mechanical seal failure on pump P-101A r
esulted in minor hydrocarbon release. "
            "Area gas detectors activated. Emergency is
olation carried out within 3 minutes. "
            "No injuries. Approximately 50 liters relea
sed to containment area.",
        "root_cause": "Seal operated beyond recommended life (18
months vs 12 months recommended). "
            "PM schedule not updated after OEM bulletin
OEM-2022-045.",
        "corrective_actions": [
            "Replace seal with upgraded Type B seal as per OEM r
ecommendation",
            "Update PM schedule to 12-month seal replacement",
            "Review all similar pumps for seal life compliance",
            "Issue safety alert to all operating teams",
        ],
    },
    {
        "title": "Near Miss - Pressure Safety Valve Failure to O
perate",
        "equipment": "PSV-3001",
        "severity": "Critical",
        "description": "During routine PSV testing, PSV-3001 on
reactor R-601 failed to lift at set pressure. "
            "Valve found stuck due to corrosion on sea
t. Discovered during planned test - no actual overpressure event.",
        "root_cause": "Process fluid causing accelerated corrosi
on on valve internals. "
            "Testing interval of 24 months inadequate fo
r this service.",
        "corrective_actions": [
            "Replace PSV-3001 with corrosion-resistant trim mate
rial",
            "Reduce testing interval to 12 months for all PSVs i
n similar service",
            "Conduct immediate testing of all PSVs in corrosive
service",
        ],
    },
]

```

```

        "Update risk assessment per OISD-154 requirements",
    ],
},
{
    "title": "Compressor Trip - High Discharge Temperature",
    "equipment": "C-301",
    "severity": "Medium",
    "description": "Compressor C-301 tripped on high discharge temperature alarm. "
                    "Temperature reached 185°C (alarm at 180°C, design limit 200°C). "
                    "Automatic shutdown functioned correctly. No equipment damage.",
    "root_cause": "Intercooler HX-301 fouled, reducing cooling capacity by 30%. "
                    "Fouling inspection was 4 months overdue.",
    "corrective_actions": [
        "Clean intercooler HX-301",
        "Establish fouling monitoring via pressure drop trending",
        "Add cooling efficiency KPI to daily operator rounds",
        "Review inspection scheduling compliance for all heat exchangers",
    ],
},
]

```

```
scenario = random.choice(scenarios)
```

```

report = {
    "incident_id": f"INC-{2022000 + i}",
    "date": date.strftime("%d-%b-%Y"),
    "title": scenario["title"],
    "equipment_tag": scenario["equipment"],
    "severity": scenario["severity"],
    "area": f"Unit-{random.randint(1,5)}",
    "reported_by": random.choice(["Control Room Operator", "Field Technician", "Shift Supervisor"]),
    "description": scenario["description"],
    "root_cause_analysis": scenario["root_cause"],
    "corrective_actions": scenario["corrective_actions"],
    "lessons_learned": f"This incident highlights the importance of adhering to OEM maintenance recommendations "
                        f"and timely execution of inspection schedules. Reference: {random.choice(['OISD-154', 'OISD-163', 'API-580'])}.",
}

```

```

        "investigation_status": "Closed" if random.random() > 0.2 else "Open",
    }
    reports.append(report)

    return reports

def generate_sop():
    """Generate a sample SOP document."""
    return {
        "document_id": "SOP-OPS-101",
        "title": "Standard Operating Procedure: Hot Work Near Hydrocarbon Lines",
        "revision": "Rev 3",
        "effective_date": "01-Jan-2024",
        "approved_by": "Plant Manager - D.K. Mehta",
        "applicable_regulations": ["OISD-105", "Factory Act Section 38", "IS-3016"],
        "scope": "This procedure covers all hot work activities (welding, cutting, grinding) "
            "within 15 meters of equipment containing or previously containing hydrocarbons.",
        "steps": [
            {
                "step": 1,
                "action": "Obtain Hot Work Permit (PTW) from Control Room Supervisor",
                "responsible": "Maintenance Supervisor",
                "reference": "PTW Procedure SOP-SAFE-001",
            },
            {
                "step": 2,
                "action": "Confirm gas test shows <1% LEL in work area. Gas test valid for 4 hours maximum.",
                "responsible": "Safety Officer",
                "reference": "Gas Testing Procedure SOP-SAFE-012",
            },
            {
                "step": 3,
                "action": "Verify equipment isolation (Double Block and Bleed). Check isolation certificate.",
                "responsible": "Operations Engineer",
                "reference": "Isolation Procedure SOP-OPS-050",
            },
        ]
    }

```

```

        "step": 4,
        "action": "Position fire extinguisher and fire watch personnel. Minimum 2 DCP extinguishers (10 kg).",
        "responsible": "Fire & Safety",
        "reference": "OISD-105 Clause 6.3",
    },
    {
        "step": 5,
        "action": "Commence hot work. Continuous gas monitoring required. Stop work if LEL exceeds 10%.",
        "responsible": "Executing Technician",
        "reference": "OISD-105 Clause 7.1",
    },
],
    "ppe_required": [
        "Fire-retardant coveralls",
        "Welding helmet with appropriate shade",
        "Leather gloves",
        "Safety shoes with metatarsal guard",
        "Portable gas detector (4-gas)",
    ],
    "emergency_procedure": "In case of fire/gas leak: Stop work immediately -> Activate area alarm -> "
                           "Evacuate to Assembly Point B -> Call Emergency: Ext 999",
}

```

```

def save_documents():
    """Generate and save all sample documents."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    # Work Orders
    work_orders = generate_work_orders(20)
    with open(OUTPUT_DIR / "work_orders.json", "w") as f:
        json.dump(work_orders, f, indent=2)
    print(f"Generated {len(work_orders)} work orders")

    # Inspection Reports
    inspections = generate_inspection_reports(10)
    with open(OUTPUT_DIR / "inspection_reports.json", "w") as f:
        json.dump(inspections, f, indent=2)
    print(f"Generated {len(inspections)} inspection reports")

    # Incident Reports
    incidents = generate_incident_reports(8)

```

```

with open(OUTPUT_DIR / "incident_reports.json", "w") as f:
    json.dump(incidents, f, indent=2)
print(f"Generated {len(incidents)} incident reports")

# SOP
sop = generate_sop()
with open(OUTPUT_DIR / "sop_hot_work.json", "w") as f:
    json.dump(sop, f, indent=2)
print("Generated SOP document")

# Also create text versions for ingestion testing
_create_text_versions(work_orders, inspections, incidents, sop)

print(f"\nAll documents saved to {OUTPUT_DIR}")

def _create_text_versions(work_orders, inspections, incidents, sop):
    """Create plain text versions of documents for testing the ingestion
    pipeline."""

    # Work order as text
    with open(OUTPUT_DIR / "work_order_sample.txt", "w") as f:
        for wo in work_orders[:5]:
            f.write(f"WORK ORDER: {wo['work_order_id']}\n")
            f.write(f"Equipment: {wo['equipment_tag']}\n")
            f.write(f"Type: {wo['work_type']}\n")
            f.write(f"Priority: {wo['priority']}\n")
            f.write(f>Date: {wo['date_raised']}\n")
            f.write(f"Assigned To: {wo['assigned_to']}\n")
            f.write(f>Description: {wo['description']}\n")
            f.write(f"Findings: {wo['findings']}\n")
            f.write(f"Parts Used: {wo['parts_used']}\n")
            f.write(f"Man-Hours: {wo['man_hours']}\n")
            f.write(f"Standard: {wo['applicable_standard']}\n")
            f.write("\n" + "="*60 + "\n\n")

    # Incident report as text
    with open(OUTPUT_DIR / "incident_report_sample.txt", "w") as f:
        for inc in incidents[:3]:
            f.write(f"INCIDENT REPORT: {inc['incident_id']}\n")
            f.write(f>Date: {inc['date']}\n")
            f.write(f>Title: {inc['title']}\n")
            f.write(f"Equipment: {inc['equipment_tag']}\n")
            f.write(f>Severity: {inc['severity']}\n")
            f.write(f"\nDescription:\n{inc['description']}\n")
            f.write(f"\nRoot Cause Analysis:\n{inc['root_cause_analysis']}

```

```

s']}\n")
        f.write(f"\nCorrective Actions:\n")
        for action in inc['corrective_actions']:
            f.write(f"    - {action}\n")
        f.write(f"\nLessons Learned:\n{inc['lessons_learned']}\n")
        f.write("\n" + "="*60 + "\n\n")

# SOP as text
with open(OUTPUT_DIR / "sop_hot_work.txt", "w") as f:
    f.write(f"STANDARD OPERATING PROCEDURE\n")
    f.write(f"Document ID: {sop['document_id']}\n")
    f.write(f>Title: {sop['title']}\n")
    f.write(f"Revision: {sop['revision']}\n")
    f.write(f"Effective Date: {sop['effective_date']}\n")
    f.write(f"Approved By: {sop['approved_by']}\n")
    f.write(f"\nApplicable Regulations: {' '.join(sop['applicable_regulations'])}\n")
    f.write(f"\nScope:\n{sop['scope']}\n")
    f.write(f"\nPROCEDURE STEPS:\n")
    for step in sop['steps']:
        f.write(f"\n    Step {step['step']}: {step['action']}\n")
        f.write(f"        Responsible: {step['responsible']}\n")
        f.write(f"        Reference: {step['reference']}\n")
    f.write(f"\nPPE Required:\n")
    for ppe in sop['ppe_required']:
        f.write(f"    - {ppe}\n")
    f.write(f"\nEmergency Procedure:\n{sop['emergency_procedure']}\n")

if __name__ == "__main__":
    save_documents()

```

scripts/demo_standalone.py

Standalone Pipeline Demo (Zero Dependencies)

Self-contained demo of entity extraction, classification, chunking, and relationship extraction using pure Python stdlib. Includes shimmed modules for corporate environments. Processes 3 sample documents and shows cross-document intelligence.

```

"""
Standalone demo of the AXIOM pipeline core logic.
Runs without ANY external dependencies – pure Python 3.11 stdlib.
Demonstrates: Entity Extraction, Document Classification, Chunking, Relationships.

```

```

Usage:
    python scripts/demo_standalone.py
    """

import re
import sys
from dataclasses import dataclass, field
from typing import Optional
from pathlib import Path

# =====
# ENTITY EXTRACTION (pure regex + pattern-based, no spaCy needed)
# =====

@dataclass
class Entity:
    entity_type: str
    value: str
    confidence: float
    context: str = ""

@dataclass
class Relation:
    source: str
    relation: str
    target: str
    confidence: float
    context: str = ""

class IndustrialEntityExtractor:
    """Multi-pattern industrial entity extraction – zero dependencies."""

    _MONTHS = {"JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG",
               "SEP", "OCT", "NOV", "DEC"}
    _NOT_EQUIPMENT = {"SOP", "WO", "MOC", "PTW", "NCR", "IR", "OR", "IS", "OEM"}
    _STANDARDS = {"OISD", "API", "ISO", "ASME", "ASTM"}

    EQUIPMENT_PATTERNS = [

```

```

        r"\b([A-Z]{1,4})-(\d{2,5}[A-Z]?)\b",
        r"\b(Pump|Compressor|Valve|Reactor|Vessel|Tank|Heat Exchanger|Motor)\s+([A-Z]?-\d{2,5}[A-Z]?)\b",
    ]

    PARAMETER_PATTERNS = [
        r"(\d+\.\d*)\s*(?:mm/s|bar|psi|MPa|kPa|°C|°F|m3/h|GPM|kg/hr|μm)",
    ]

    REGULATION_PATTERNS = [
        r"\b(OISD[-\s]?\d{3})\b",
        r"\b(API[-\s]?\d{3,4}[A-Z]?)\b",
        r"\b(ISO[-\s]?\d{4,5})\b",
        r"\b(ASME[-\s]?[A-Z]+[-\s]?\d+)\b",
    ]

    DATE_PATTERNS = [
        r"\b(\d{1,2}[-/]\d{1,2}[-/]\d{2,4})\b",
        r"\b(\d{1,2}\s*-\s*(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\s*-\s*\d{2,4})\b",
        r"\b(\d{1,2}\s*(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\s+\d{4})\b",
    ]

    DOCUMENT_REF_PATTERNS = [
        r"\b(WO[-\s]?\d{5,10})\b",
        r"\b(MOC[-\s]?\d{4,8})\b",
        r"\b(PTW[-\s]?\d{4,8})\b",
        r"\b(SOP[-\s]?\d{3,15})\b",
        r"\b(INC[-\s]?\d{4,8})\b",
    ]

    PERSONNEL_PATTERN = r"\b([A-Z][a-z]+(?:\s+[A-Z]\.?)?\s+[A-Z][a-z]+(?:\s+[A-Z][a-z]+)?)\b"

    def extract_all(self, text: str) -> list[Entity]:
        entities = []
        entities.extend(self._extract("equipment", self.EQUIPMENT_PATTERNS, text, 0.9))
        entities.extend(self._extract("parameter", self.PARAMETER_PATTERNS, text, 0.85))
        entities.extend(self._extract("regulation", self.REGULATION_PATTERNS, text, 0.95))
        entities.extend(self._extract("date", self.DATE_PATTERNS, text, 0.85))

```



```

        entities.extend(self._extract("document_reference", self.DOCUMENT_REF_PATTERNS, text, 0.9))
        entities.extend(self._extract_personnel(text))
        return self._deduplicate(entities)

    def _extract(self, entity_type: str, patterns: list, text: str, confidence: float) -> list[Entity]:
        entities = []
        for pattern in patterns:
            for match in re.finditer(pattern, text, re.IGNORECASE):
                value = match.group(0).strip()
                if entity_type == "equipment":
                    value = value.upper()
                    prefix = value.split("-")[0] if "-" in value else ""
                    # Filter false positives
                    if prefix in self._MONTHS or prefix in self._NOT_EQUIPMENT or prefix in self._STANDARDS:
                        continue
                context = text[max(0, match.start() - 40):match.end() + 40]

                entities.append(Entity(
                    entity_type=entity_type,
                    value=value,
                    confidence=confidence,
                    context=context,
                ))
        return entities

    def _extract_personnel(self, text: str) -> list[Entity]:
        entities = []
        # Look for names in typical assignment patterns
        patterns = [
            r"(?:Assigned To|Verified By|Reported By|Inspector|Lead|Approved By)[:\s]+([A-Z][a-z]+(?:\s+[A-Z]\.?)?\s+[A-Z][a-z]+)",
            r"([A-Z][a-z]+\s+[A-Z][a-z]+\s+)((?:Maintenance|Plant|Safety|Operations)",
        ]
        for pattern in patterns:
            for match in re.finditer(pattern, text):
                entities.append(Entity(
                    entity_type="personnel",
                    value=match.group(1).strip(),
                    confidence=0.8,
                    context=text[max(0, match.start() - 20):match.end() + 20],
                ))

```

```

        return entities

def _deduplicate(self, entities: list[Entity]) -> list[Entity]:
    seen = {}
    for e in entities:
        key = (e.entity_type, e.value.lower())
        if key not in seen or e.confidence > seen[key].confidence:
            seen[key] = e
    return list(seen.values())

# =====
#
# DOCUMENT CLASSIFICATION
# =====

class DocumentClassifier:
    SIGNALS = {
        "work_order": [
            (r"work\s+order|WO[-\s]?d{4,}|maintenance\s+order", 3),
            (r"corrective\s+maintenance|preventive\s+maintenance|breakdo
wn", 2),
            (r"spare\s+parts|man[-\s]?hours|downtime|repair|findings",
1),
        ],
        "sop": [
            (r"standard\s+operating\s+procedure|SOP|operating\s+instruct
ion", 3),
            (r"step\s+\d+|precaution|PPE\s+required", 1),
            (r"procedure\s+no|rev(ision)?|approved\s+by", 2),
        ],
        "inspection_report": [
            (r"inspection\s+report|NDT|thickness\s+survey", 3),
            (r"corrosion|defect|finding|recommendation", 1),
            (r"fitness\s+for\s+service|remaining\s+life", 2),
        ],
        "incident_report": [
            (r"incident\s+report|near[-\s]?miss", 3),
            (r"root\s+cause\s+analysis|RCA|investigation", 2),
            (r"corrective\s+action|CAPA", 2),
        ],
        "regulatory": [
            (r"OISD|PES0|Factory\s+Act|BIS|statutory|compliance", 3),
            (r"regulation|standard|code\s+of\s+practice", 2),
        ],
    }

```

```

    }

    def classify(self, text: str, filename: str = "") -> str:
        scores = {cat: 0 for cat in self.SIGNALS}
        sample = text[:5000]
        for category, patterns in self.SIGNALS.items():
            for pattern, weight in patterns:
                matches = len(re.findall(pattern, sample, re.IGNORECASE))

E))

                scores[category] += matches * weight

        best = max(scores, key=scores.get)
        return best if scores[best] >= 2 else "general"

# =====
# =====
# TEXT CHUNKING
# =====
# =====

class TextChunker:
    def __init__(self, chunk_size: int = 400, overlap: int = 50):
        self.chunk_size = chunk_size
        self.overlap = overlap

    def chunk(self, text: str) -> list[str]:
        sentences = re.split(r"(?<=[.!?\\n])\\s+", text)
        chunks = []
        current = []
        current_len = 0

        for sent in sentences:
            if current_len + len(sent) > self.chunk_size and current:
                chunks.append(" ".join(current))
                # Keep overlap
                overlap_sents = []
                olen = 0
                for s in reversed(current):
                    if olen + len(s) <= self.overlap:
                        overlap_sents.insert(0, s)
                        olen += len(s)
                    else:
                        break
                current = overlap_sents
                current_len = olen
            else:
                current.append(sent)
                current_len += len(sent)

        chunks.append(" ".join(current))

        return chunks

```

```

        current.append(sent)
        current_len += len(sent)

    if current:
        chunks.append(" ".join(current))
    return chunks

# =====
# RELATIONSHIP EXTRACTION
# =====

class RelationshipExtractor:
    PATTERNS = [
        (r"(\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b).*?(?:failed|tripped).*?(?:due to|because of|caused by)\s+(.+?)(?:\.|$)", "FAILED_DUE_TO"),
        (r"(\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b).*?(?:as per|per|reference)\s+((?:SOP|OISD|API|ISO)\S+)", "GOVERNED_BY"),
        (r"(\b[A-Z]{1,4}-\d{2,5}[A-Z]?\b).*?(?:protecting|on)\s+(?:react or|vessel|tank)\s+(\b[A-Z]{1,4}-\d{2,5}\b)", "PROTECTS"),
    ]

    def extract(self, text: str, entities: list[Entity]) -> list[Relation]:
        relations = []

        # Pattern-based
        for pattern, rel_type in self.PATTERNS:
            for match in re.finditer(pattern, text, re.IGNORECASE | re.DOTALL):
                groups = match.groups()
                if len(groups) >= 2:
                    relations.append(Relation(
                        source=groups[0].strip().upper(),
                        relation=rel_type,
                        target=groups[1].strip()[:80],
                        confidence=0.8,
                        context=text[max(0, match.start()-20):match.end()+20],
                    ))

        # Co-occurrence: equipment + regulation in same sentence
        sentences = re.split(r"[.!?]\n", text)

```

```

equip = {e.value for e in entities if e.entity_type == "equipment"}
regs = {e.value for e in entities if e.entity_type == "regulation"}

for sent in sentences:
    sent_equip = [e for e in equip if e in sent.upper()]
    sent_regs = [r for r in regs if r in sent]
    for eq in sent_equip:
        for reg in sent_regs:
            relations.append(Relation(
                source=eq, relation="GOVERNED_BY", target=reg,
                confidence=0.65, context=sent[:100],
            ))

# Deduplicate
seen = set()
unique = []
for r in relations:
    key = (r.source, r.relation, r.target)
    if key not in seen:
        seen.add(key)
        unique.append(r)
return unique

```

```

# =====
#
# SAMPLE DOCUMENTS
# =====
#

```

SAMPLE_WORK_ORDER = """

WORK ORDER: WO-100234

Equipment: P-101A (Boiler Feed Water Pump A)

Type: Corrective Maintenance

Priority: High

Date Raised: 15-Mar-2024

Assigned To: Rajesh Kumar

Description:

Pump P-101A tripped on high vibration alarm at 08:45 hrs. Vibration readings showed

12.5 mm/s on drive end bearing (alarm at 11.2 mm/s, trip at 14.0 mm/s as per ISO-10816).

Pump was isolated and decontaminated as per SOP-OPS-050.

Findings:

Drive end bearing (SKF 6310-2RS) found with severe pitting on inner race. Root cause identified as contaminated lubrication oil. Oil analysis report OR-2024-156 confirms particulate contamination at 45 µm (limit: 25 µm per OEM manual).

Corrective Actions:

1. Replaced bearing SKF 6310-2RS (both ends as precaution)
2. Flushed lubrication system
3. Replaced lube oil with Shell Omala S4 GX 320
4. Realignment performed - readings within 0.05 mm (spec: 0.1 mm per API-610)
5. Post-maintenance vibration: 2.8 mm/s (acceptable)

Parts Used: Bearing SKF 6310-2RS (qty: 2), Mechanical seal (qty: 1), Lubrication oil 20L

Man-Hours: 16

Completed: 16-Mar-2024

Verified By: Suresh Patel (Maintenance Supervisor)

Reference: Similar failure occurred on P-101B in Aug-2023 (WO-098876). Recommend reviewing lubrication PM frequency per OISD-163 guidelines.

""

SAMPLE_INCIDENT = ""

INCIDENT REPORT: INC-2024012

Date: 22-Apr-2024

Severity: High

Area: Unit-2 Hydrocracker

Title: Pressure Safety Valve PSV-3001 Failed to Lift During Routine Testing

Description:

During scheduled PSV testing as per OISD-154 requirements, PSV-3001 protecting reactor R-601 failed to lift at set pressure of 45 bar. Valve required 52 bar (115% of set pressure) to actuate. This exceeds the +/- 3% tolerance per API-520.

The valve had been in service for 18 months since last overhaul. Previous test (Oct-2023) showed normal operation. No actual overpressure event occurred.

d as this
was discovered during planned testing.

Root Cause Analysis:

- Primary cause: Corrosion product buildup on valve seat due to process fluid containing trace H₂S (measured at 15 ppm, design basis was <5 ppm)
- Contributing factor: Change in feed composition since upstream unit modification (MOC-2023-445) not reflected in PSV testing frequency assessment
- Systemic factor: MOC procedure did not trigger downstream equipment review

Corrective Actions:

1. PSV-3001 replaced with upgraded trim material (Stellite 6 seat) - IMMEDIATE
2. All PSVs in H₂S service to be tested within 30 days - WITHIN 1 MONTH
3. MOC procedure updated to include downstream impact assessment - WITHIN 2 WEEKS
4. Feed composition monitoring added to daily lab analysis - IMMEDIATE
5. Risk assessment update for reactor R-601 per API-580 - WITHIN 1 MONTH

Lessons Learned:

Process changes upstream can have delayed effects on downstream safety devices.

The MOC system must explicitly address secondary and tertiary impacts. This incident

is similar to the findings in CSB report 2019-003 on the importance of PSV management.

Investigation Lead: D.K. Mehta (Plant Manager)

Status: Open - Actions 2, 3, 5 pending

""

SAMPLE_SOP = ""

STANDARD OPERATING PROCEDURE

Document ID: SOP-OPS-101

Title: Hot Work Near Hydrocarbon Lines

Revision: Rev 3

Effective Date: 01-Jan-2024

Approved By: D.K. Mehta (Plant Manager)

Applicable Regulations: OISD-105, Factory Act Section 38, IS-3016

Scope:

This procedure covers all hot work activities (welding, cutting, grinding)

within 15 meters of equipment containing or previously containing hydrocarbons.

Step 1: Obtain Hot Work Permit (PTW) from Control Room Supervisor

Responsible: Maintenance Supervisor

Reference: SOP-SAFE-001

Step 2: Confirm gas test shows <1% LEL in work area. Test valid for 4 hours max.

Responsible: Safety Officer

Reference: SOP-SAFE-012

Step 3: Verify equipment isolation (Double Block and Bleed). Check certificate.

Responsible: Operations Engineer

Reference: SOP-OPS-050

Step 4: Position fire extinguisher and fire watch. Min 2 DCP extinguishers (10 kg).

Responsible: Fire & Safety

Reference: OISD-105 Clause 6.3

Step 5: Commence hot work. Continuous gas monitoring. Stop if LEL exceeds 10%.

Responsible: Executing Technician

Reference: OISD-105 Clause 7.1

PPE Required: Fire-retardant coveralls, Welding helmet, Leather gloves, Safety shoes with metatarsal guard, Portable gas detector (4-gas)

"""

```
# =====  
=====
```

```
# MAIN DEMO
```

```
# =====  
=====
```

```
def run_demo():  
    print("=" * 70)  
    print("  AXIOM Industrial Knowledge Intelligence - Pipeline Demo")  
    print("  (Standalone - no external dependencies required)")  
    print("=" * 70)
```

```
extractor = IndustrialEntityExtractor()  
classifier = DocumentClassifier()
```



```

chunker = TextChunker(chunk_size=300, overlap=50)
rel_extractor = RelationshipExtractor()

documents = [
    ("Work Order WO-100234", SAMPLE_WORK_ORDER, "work_order_P101A.pdf"),
    ("Incident Report INC-2024012", SAMPLE_INCIDENT, "incident_PSV3001.pdf"),
    ("SOP Hot Work", SAMPLE_SOP, "sop_hot_work.pdf"),
]

all_entities = {}
all_relations = {}

# — STEP 1: Process Each Document —————
—————
for doc_name, text, filename in documents:
    print(f"\n\n{'='*70}")
    print(f"  DOCUMENT: {doc_name}")
    print(f"{'='*70}")

    # Classification
    category = classifier.classify(text, filename)
    print(f"\n  [Classification] Category: {category}")

    # Entity Extraction
    entities = extractor.extract_all(text)
    all_entities[doc_name] = entities
    print(f"\n  [Entity Extraction] Found {len(entities)} entities:")

    by_type = {}
    for e in entities:
        by_type.setdefault(e.entity_type, []).append(e)

    for etype, elist in sorted(by_type.items()):
        print(f"    {etype:22s}: {' '.join(e.value for e in elist[:5])}")

    # Chunking
    chunks = chunker.chunk(text)
    print(f"\n  [Chunking] Split into {len(chunks)} chunks (300 characters each)")
    for i, chunk in enumerate(chunks[:2]):
        print(f"    Chunk {i}: \"{chunk[:70]}...\")

```

```

# Relationship Extraction
relations = rel_extractor.extract(text, entities)
all_relations[doc_name] = relations
print(f"\n [Relationships] Found {len(relations)} relationship
s:")

for r in relations[:5]:
    print(f"    {r.source} -[{r.relation}]-> {r.target}")

# — STEP 2: Cross-Document Intelligence —————
—————

print(f"\n\n{'='*70}")
print("  CROSS-DOCUMENT INTELLIGENCE")
print(f"{'='*70}")

# Find shared entities across documents
doc_equipment = {}
for doc_name, entities in all_entities.items():
    doc_equipment[doc_name] = {e.value for e in entities if e.entity
_type == "equipment"}

print("\n  Equipment entities per document:")
for doc_name, equip_set in doc_equipment.items():
    print(f"    {doc_name}: {equip_set}")

# Cross-document links
all equip = set()
for s in doc_equipment.values():
    all equip.update(s)

print(f"\n  Total unique equipment across corpus: {len(all equip)}")
print(f"  Equipment tags: {sorted(all equip)}")

# Find equipment mentioned in multiple documents
multi_doc equip = []
for tag in all equip:
    docs_with_tag = [d for d, s in doc_equipment.items() if tag in
s]

    if len(docs_with_tag) > 1:
        multi_doc equip.append((tag, docs_with_tag))

if multi_doc equip:
    print(f"\n  Cross-document links found:")
    for tag, docs in multi_doc equip:
        print(f"    {tag} appears in: {docs}")

# — STEP 3: Knowledge Graph Preview —————

```

```

print(f"\n\n{'='*70}")
print("    KNOWLEDGE GRAPH (would be created in Neo4j)")
print(f"{'='*70}")

print("\n  Nodes:")
node_count = 0
for doc_name, entities in all_entities.items():
    for e in entities:
        if e.entity_type in ("equipment", "regulation", "personnel"):
            print(f"      ({e.entity_type.title()} {{value: '{e.value}'}})")
            node_count += 1
            if node_count > 15:
                print("      ... (truncated)")
                break
    if node_count > 15:
        break

print("\n  Relationships:")
rel_count = 0
for doc_name, relations in all_relations.items():
    for r in relations:
        print(f"      ({r.source})-[:{r.relation}]->({r.target})")
        rel_count += 1
        if rel_count > 10:
            print("      ... (truncated)")
            break
    if rel_count > 10:
        break

# — STEP 4: Simulated RAG Query —————


---


print(f"\n\n{'='*70}")
print("    SIMULATED RAG QUERY")
print(f"{'='*70}")

query = "What caused the failure of pump P-101A and what standards apply?"
print(f"\n  User Query: \"{query}\"")

# Simulate retrieval
print("\n  --- Retrieval Results ---")
print("    [1] VECTOR SEARCH (semantic match):")
print("      -> Work Order W0-100234, relevance: 0.94")

```

```

print("          \"Pump P-101A tripped on high vibration...bearing fa
ilure...\")

print(" [2] GRAPH TRAVERSAL (relationship-aware):")
print("      -> P-101A -[GOVERNED_BY]-> ISO-10816")
print("      -> P-101A -[GOVERNED_BY]-> API-610")
print("      -> P-101A -[GOVERNED_BY]-> OISD-163")
print("      -> P-101A -[HAS_WORK_ORDER]-> W0-100234")

print(" [3] KEYWORD SEARCH (BM25 exact match):")
print("      -> Match on 'P-101A' in Work Order, score: 12.4")
print("      -> Match on 'P-101A' in Incident Report reference, scor
e: 3.1")

# Simulate answer generation
print("\n --- Generated Answer (with citations) ---")
print(" " + "-" * 60)
print(""" Pump P-101A failed due to a drive end bearing (SKF 6310-2
RS)
with severe pitting caused by contaminated lubrication oil
[Source 1]. Oil analysis confirmed particulate contamination at
45 microns, exceeding the 25 micron OEM limit [Source 1].

The vibration reached 12.5 mm/s against an alarm setpoint of
11.2 mm/s and trip at 14.0 mm/s [Source 1].

Applicable Standards:
- ISO-10816: Vibration severity limits [Source 2]
- API-610: Pump alignment specification (0.1 mm) [Source 2]
- OISD-163: Maintenance frequency guidelines [Source 2]

Note: A similar failure occurred on P-101B in Aug-2023
(W0-098876), suggesting a systemic lubrication issue [Source 1].

Confidence: HIGH (multiple corroborating sources)

Suggested follow-ups:
- What is the lubrication PM schedule for P-101A and P-101B?
- Has the oil contamination source been identified?
- Are there other pumps with SKF 6310-2RS bearings to check?""")
print(" " + "-" * 60)

# — Summary —————
print(f"\n\n{' '*70}")
print(" PIPELINE SUMMARY")

```

```

print(f"{'='*70}")
total_entities = sum(len(e) for e in all_entities.values())
total_relations = sum(len(r) for r in all_relations.values())
print(f"""
Documents processed:    3
Total entities found:   {total_entities}
Total relationships:    {total_relations}
Unique equipment tags:  {len(all_equip)}
Cross-doc links:        {len(multi_doc_equip)}

Pipeline Components:
[x] Document Ingestion (PDF/DOCX/Image)
[x] OCR Engine (Tesseract fallback for scanned pages)
[x] Document Classification (regex pattern scoring)
[x] Entity Extraction (equipment, parameters, regulations, personnel)
[x] Relationship Extraction (pattern + co-occurrence)
[x] Text Chunking (semantic boundary-aware)
[x] Vector Store Indexing (Qdrant + OpenAI embeddings)
[x] Knowledge Graph Population (Neo4j)
[x] Hybrid Retrieval (Vector + Graph + BM25 fusion)
[x] Answer Generation (LLM with citation enforcement)
""")
print("=" * 70)

if __name__ == "__main__":
    run_demo()

```

scripts/demo_agents.py

Multi-Agent Demo (Zero Dependencies)

Self-contained demo of all 4 agents. Simulates 30 degrading vibration readings, shows monitor alerts, fault diagnosis hypotheses, predictive RUL estimation, and shift handover report. Includes full agent coordination scenario.

```

"""
Standalone demo of the Multi-Agent System.
No external dependencies - runs with pure Python 3.11.

Demonstrates:
1. Sensor Health Monitor detecting anomalies
2. Fault Diagnosis generating root cause hypotheses
3. Predictive Maintenance estimating RUL
4. Shift Handover report generation

```

```

"""

import sys
import math
import statistics
import types
from pathlib import Path
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from enum import Enum

# Shim structlog and other missing modules so agent code imports cleanly
class _NullLogger:
    def info(self, *a, **kw): pass
    def debug(self, *a, **kw): pass
    def warning(self, *a, **kw): pass
    def error(self, *a, **kw): pass

_structlog = types.ModuleType("structlog")
_structlog.get_logger = lambda *a, **kw: _NullLogger()
sys.modules["structlog"] = _structlog

# Shim pydantic_settings (only needed by config.py)
_ps = types.ModuleType("pydantic_settings")
class _FakeBaseSettings:
    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"
    def __init_subclass__(cls, **kw): pass
    def __init__(self, **kw):
        import os
        for k, v in type(self).__dict__.items():
            if not k.startswith("_") and k != "Config":
                setattr(self, k, os.environ.get(k.upper(), v))
_ps.BaseSettings = _FakeBaseSettings
sys.modules["pydantic_settings"] = _ps

# Shim httpx (used by agents for Ollama calls - not needed in standalone
demo)
_httpx = types.ModuleType("httpx")
class _FakeAsyncClient:
    async def __aenter__(self): return self
    async def __aexit__(self, *a): pass
    async def post(self, *a, **kw):
        r = types.SimpleNamespace(status_code=503, json=lambda: {})
        return r

```

```

_httpx.AsyncClient = lambda **kw: _FakeAsyncClient()
sys.modules["httpx"] = _httpx

# Shim neo4j driver
_neo4j_mod = types.ModuleType("neo4j")
_neo4j_mod.AsyncGraphDatabase = type("AsyncGraphDatabase", (), {"drive
r": staticmethod(lambda *a, **kw: None)})
_neo4j_mod.AsyncDriver = type("AsyncDriver", (), {})
sys.modules["neo4j"] = _neo4j_mod

# Add backend to path
sys.path.insert(0, str(Path(__file__).parent.parent / "backend"))

def demo_sensor_monitor():
    """Demo: Sensor Health Monitor Agent detects anomalies."""
    print("\n" + "=" * 70)
    print("  AGENT 1: SENSOR HEALTH MONITOR")
    print("  Ingesting machine sensor logs and detecting anomalies")
    print("=" * 70)

    from app.services.agents.sensor_monitor_agent import (
        SensorHealthMonitorAgent, SensorReading, HealthStatus,
    )

    agent = SensorHealthMonitorAgent()

    # Simulate sensor readings for Pump P-101A
    # Normal operation followed by degradation
    print("\n  Simulating 30 vibration readings for Pump P-101A...")
    print("  (Normal -> Degrading -> Warning -> Critical)\n")

    readings = []
    base_time = datetime(2024, 7, 1, 8, 0)

    # Normal readings (1-15)
    for i in range(15):
        readings.append(SensorReading(
            equipment_tag="P-101A",
            parameter="vibration",
            value=3.5 + (i * 0.1) + (0.2 * (i % 3)), # Slight variation
            unit="mm/s",
            timestamp=base_time + timedelta(hours=i),
        ))

    # Degrading readings (16-25)

```

```

for i in range(15, 25):
    readings.append(SensorReading(
        equipment_tag="P-101A",
        parameter="vibration",
        value=5.0 + ((i - 15) * 0.6), # Steeper increase
        unit="mm/s",
        timestamp=base_time + timedelta(hours=i),
    ))

# Critical readings (26-30)
for i in range(25, 30):
    readings.append(SensorReading(
        equipment_tag="P-101A",
        parameter="vibration",
        value=11.0 + ((i - 25) * 0.5), # Past warning limit
        unit="mm/s",
        timestamp=base_time + timedelta(hours=i),
    ))

# Process readings
alerts = []
for reading in readings:
    alert = agent.analyze_reading(reading)
    status_icon = "." if not alert else ("!!" if alert.status == HealthStatus.WARNING else "[X]")
    if alert:
        alerts.append(alert)

print(f" Readings processed: {len(readings)}")
print(f" Anomalies detected: {len(alerts)}")
print(f"\n Alerts generated:")
for a in alerts:
    print(f"      [{a.timestamp.strftime('%H:%M')}] {a.status.value.upper()}: "
          f"{a.parameter}={a.current_value:.1f} {a.unit}")
    print(f"          Reason: {a.reason}")
    print(f"          Severity Score: {a.severity_score:.2f}")
    print()

# Health summary
summary = agent.get_equipment_health_summary("P-101A")
print(f" Equipment Health Summary:")
for param, data in summary.get("parameters", {}).items():
    print(f"      {param}: latest={data['latest']:.1f}, "
          f"mean={data['mean']:.1f}, trend={data['trend']}")

```



```

return alerts

def demo_fault_diagnosis(alerts):
    """Demo: Fault Diagnosis Agent generates root cause hypotheses."""
    print("\n\n" + "=" * 70)
    print("  AGENT 2: FAULT DIAGNOSIS & SUGGESTION")
    print("  Analyzing anomalies and suggesting corrective actions")
    print("=" * 70)

    from app.services.agents.fault_diagnosis_agent import FaultDiagnosis
    Agent

    agent = FaultDiagnosisAgent(neo4j_client=None) # No KG in standalone demo

    if not alerts:
        print("\n  No alerts to diagnose.")
        return

    # Take the most critical alert
    critical_alert = max(alerts, key=lambda a: a.severity_score)
    print(f"\n  Diagnosing: {critical_alert.equipment_tag} - "
          f"{critical_alert.parameter} {critical_alert.status.value}")
    print(f"  Value: {critical_alert.current_value} {critical_alert.unit}")

    # Run synchronous part of diagnosis (pattern matching)
    hypotheses = agent._get_pattern_hypotheses(critical_alert)

    print(f"\n  Root Cause Hypotheses ({len(hypotheses)} generated):")
    for i, h in enumerate(sorted(hypotheses, key=lambda x: x.confidence,
reverse=True), 1):
        print(f"    {i}. {h.cause}")
        print(f"      Confidence: {h.confidence:.0%} | Category: {h.category}")
        print(f"      Evidence: {h.evidence[0]}")
        print()

    # Get standard actions
    actions = agent._generate_actions(hypotheses, critical_alert)
    print(f"  Recommended Actions ({len(actions)}):")
    for i, a in enumerate(actions, 1):
        print(f"    {i}. [{a.priority.upper()}] {a.action}")
        if a.parts_needed:
            print(f"      Parts: {' '.join(a.parts_needed)}")

```

```

        if a.sop_reference:
            print(f"          Reference: {a.sop_reference}")
        if a.safety_precautions:
            print(f"          Safety: {'', '.join(a.safety_precautions)}")
        print()

def demo_predictive_maintenance():
    """Demo: Predictive Maintenance Agent estimates RUL."""
    print("\n\n" + "=" * 70)
    print("  AGENT 3: PREDICTIVE MAINTENANCE")
    print("  Estimating Remaining Useful Life (RUL)")
    print("=" * 70)

    from app.services.agents.predictive_maintenance_agent import PredictiveMaintenanceAgent

    agent = PredictiveMaintenanceAgent()

    # Simulate degrading vibration history (50 readings over 50 hours)
    vibration_history = [3.5 + (i * 0.15) + (0.1 * (i % 4)) for i in range(50)]
    # Latest reading: ~10.5 mm/s, limit: 14.0 mm/s

    print(f"\n  Equipment: P-101A (Pump)")
    print(f"  Current vibration: {vibration_history[-1]:.1f} mm/s")
    print(f"  Critical limit: 14.0 mm/s")
    print(f"  Trend: Increasing at ~0.15 mm/s per hour")
    print(f"  Hours in service since last maintenance: 18000")

    report = agent.predict(
        equipment_tag="P-101A",
        parameter_history={"vibration": vibration_history},
        critical_limits={"vibration": 14.0},
        next_scheduled_pm=datetime.utcnow() + timedelta(days=45),
        equipment_type="pump",
        hours_since_last_maintenance=18000,
    )

    print(f"\n  --- Predictive Report ---")
    print(f"  Overall Health Score: {report.overall_health_score:.1f}%")
    print(f"  Risk Summary: {report.risk_summary}")

    print(f"\n  RUL Estimates:")
    for rul in report.rul_estimates:
        print(f"    Parameter: {rul.parameter}")

```

```

        print(f"    Days to failure: {rul.estimated_days_to_failure:.1f}"
              f"(CI: {rul.confidence_interval[0]:.1f} - {rul.confidence_
interval[1]:.1f})")
        print(f"    Risk Score: {rul.risk_score:.0%}")
        print(f"    Method: {rul.method}")
        print(f"    Health: {rul.current_health_pct:.1f}%")
        print()

    print(f" Maintenance Recommendations:")
    for rec in report.maintenance_recommendations:
        print(f"    Urgency: {rec.urgency.upper()}")
        print(f"    Action: {rec.action}")
        print(f"    Recommended Date: {rec.recommended_date.strftime('%d
-%b-%Y')}")
        print(f"    Cost of planned fix: ${rec.cost_of_early_interventio
n:,.0f}")
        print(f"    Cost of unplanned failure: ${rec.cost_of_unplanned_f
ailure:,.0f}")
        print(f"    Reasoning: {rec.reasoning}")
        print()

def demo_shift_handover(alerts):
    """Demo: Shift Handover Agent generates report."""
    print("\n\n" + "=" * 70)
    print(" AGENT 4: SHIFT HANDOVER")
    print(" Generating automated shift report")
    print("=" * 70)

    from app.services.agents.shift_handover_agent import (
        ShiftHandoverAgent, ShiftEvent, HandoverReport,
    )

    agent = ShiftHandoverAgent()

    shift_start = datetime(2024, 7, 1, 6, 0)
    shift_end = datetime(2024, 7, 1, 18, 0)

    # Log events from this shift
    agent.log_event(ShiftEvent(
        timestamp=shift_start + timedelta(hours=1),
        event_type="maintenance",
        severity="medium",
        equipment_tag="HX-501",
        description="Scheduled cleaning of heat exchanger tubes",

```

```

        status="resolved",
        assigned_to="Rajesh Kumar",
        action_taken="Cleaned, back in service at 08:30",
    ))

    # Log the sensor alerts
    for alert in alerts[:3]:
        agent.log_alarm(alert, action_taken="Monitoring, maintenance team notified")

    agent.log_event(ShiftEvent(
        timestamp=shift_start + timedelta(hours=8),
        event_type="operator_action",
        severity="info",
        equipment_tag="C-301",
        description="Compressor speed reduced from 3500 to 3200 RPM per operator decision",
        status="resolved",
        action_taken="Load reduced to manage temperature",
    ))

    agent.log_event(ShiftEvent(
        timestamp=shift_start + timedelta(hours=10),
        event_type="maintenance",
        severity="high",
        equipment_tag="P-101A",
        description="Vibration inspection requested - bearing check pending",
        status="in_progress",
        assigned_to="Suresh Patel",
    ))

    # Generate report (synchronous parts only for demo)
    import asyncio

    async def gen():
        return await agent.generate_report(shift_start, shift_end)

    try:
        report = asyncio.run(gen())
    except Exception:
        # Fallback if Ollama not available
        report = HandoverReport(
            shift_start=shift_start,
            shift_end=shift_end,
            shift_type="day",

```

```

        executive_summary=[
            "3 vibration alarms on P-101A - trending upward, mainten
ance notified",
            "HX-501 cleaning completed successfully",
            "C-301 speed reduced (operator decision) - monitor tempe
rature",
            "P-101A bearing inspection IN PROGRESS (Suresh Patel)",
        ],
        critical_items=[{
            "equipment": "P-101A",
            "issue": "Vibration exceeding warning limit (11.5 mm/
s)",
            "status": "pending",
            "action": "Monitoring, maintenance team notified",
            "time": "14:00",
        }],
        alarms_summary={"total": 3, "critical": 1, "high": 2, "resol
ved": 0, "pending": 3},
        work_in_progress=[{
            "equipment": "P-101A",
            "activity": "Bearing inspection",
            "status": "in_progress",
            "assigned_to": "Suresh Patel",
        }],
        watch_items=[{
            "equipment": "P-101A",
            "reason": "Recurring alarms (3 times this shift) - devel
oping bearing issue",
        }],
        total_alarms=3,
        critical_alarms=1,
        maintenance_actions=2,
    )

    formatted = agent.format_report_text(report)
    print(f"\n{formatted}")

def demo_agent_coordination():
    """Show how all agents work together."""
    print("\n\n" + "=" * 70)
    print("  AGENT COORDINATION - Full Pipeline Example")
    print("=" * 70)

    print("""

```

SCENARIO: Bearing degradation detected on Pump P-101A

Timeline:

[08:00] SENSOR MONITOR AGENT:

- > Vibration reading: 7.5 mm/s (trending up at 0.3 mm/s/hour)
- > Status: NORMAL (but trend detected)
- > Action: Continue monitoring, log trend

[12:00] SENSOR MONITOR AGENT:

- > Vibration reading: 9.8 mm/s
- > Status: WARNING (approaching 11.2 mm/s alarm)
- > Action: Trigger Fault Diagnosis Agent

[12:01] FAULT DIAGNOSIS AGENT:

- > Queried Knowledge Graph:
 - P-101A had bearing failure in Mar-2024 (W0-100234)
 - Similar pattern seen on P-101B in Aug-2023
 - OEM recommends bearing inspection at 8.0 mm/s
- > Top Hypothesis: "Bearing wear - lubrication degradation" (85%)
- > Recommended Actions:
 1. IMMEDIATE: Sample lube oil (1 hour)
 2. SHORT-TERM: Schedule bearing inspection (4 hours)
 3. LONG-TERM: Review lubrication PM frequency

[12:02] PREDICTIVE MAINTENANCE AGENT:

- > Current degradation rate: 0.3 mm/s per hour
- > Critical limit: 14.0 mm/s
- > Estimated failure: 14 hours (by 02:00 tomorrow)
- > Next scheduled PM: 45 days away
- > RECOMMENDATION: "Advance PM immediately. Failure imminent."
- > Cost analysis:
 - Planned repair now: \$2,500
 - Unplanned failure tonight: \$45,000 + 72h downtime

[18:00] SHIFT HANDOVER AGENT:

- > Generates report for incoming night shift:

"CRITICAL: P-101A vibration WARNING. Bearing failure predicted within 14 hours. Diagnosis suggests lube degradation. Oil sampling in progress (Suresh Patel). Recommend preparing for emergency bearing replacement if oil results confirm."

```
OUTCOME: Night shift team takes preemptive action, avoids
unplanned failure. $42,500 saved. Zero downtime.
```

```
""")

if __name__ == "__main__":
    print("=" * 70)
    print("  AXIOM - Multi-Agent Operations Intelligence Demo")
    print("  4 Agents working together for equipment health management")
    print("=" * 70)

    # Run all agent demos
    alerts = demo_sensor_monitor()
    demo_fault_diagnosis(alerts)
    demo_predictive_maintenance()
    demo_shift_handover(alerts)
    demo_agent_coordination()

    print("\n" + "=" * 70)
    print("  DEMO COMPLETE - All 4 agents demonstrated successfully")
    print("=" * 70)
```

scripts/test_pdf_ingest.py

PDF Ingestion Test

Creates a test PDF with PyMuPDF containing a sample work order, then ingests it via the API endpoint. Verifies entity extraction (equipment, parameters, regulations) and relationship detection from PDF content.

```
"""Quick test: create a PDF and ingest it via the API."""
import httpx
import json
import fitz

# Create a test PDF
doc = fitz.open()
page = doc.new_page()
text = """WORK ORDER: WO-200001
Equipment: P-101A (Boiler Feed Water Pump A)
Type: Corrective Maintenance
Priority: High
Date Raised: 15-Mar-2024
Assigned To: Rajesh Kumar
```

Description:

Pump P-101A tripped on high vibration alarm. Vibration readings showed 12.5 mm/s on drive end bearing (alarm at 11.2 mm/s as per ISO-10816).

Findings:

Drive end bearing found with severe pitting. Root cause identified as contaminated lubrication oil per OISD-163 guidelines. Replaced bearing and realigned per API-610.

Corrective Actions:

1. Replaced bearing SKF 6310-2RS
2. Flushed lubrication system
3. Realignment performed within 0.05 mm spec

Reference: Similar failure on P-101B (W0-098876). Review per OISD-163.

Verified By: Suresh Patel (Maintenance Supervisor)

"""

```
page.insert_text((72, 72), text, fontsize=11)
doc.save("data/test_workorder.pdf")
doc.close()
print("Test PDF created: data/test_workorder.pdf")
```

```
# Ingest via API
```

```
with open("data/test_workorder.pdf", "rb") as f:
    r = httpx.post(
        "<http://localhost:8000/api/v1/ingest/document>",
        files={"file": ("test_workorder.pdf", f, "application/pdf")},
        timeout=30,
    )
```

```
d = r.json()
print(f"\nStatus: {d.get('status', d.get('detail', 'unknown'))}")
print(f"Category: {d.get('category')}")
print(f"Pages: {d.get('total_pages')}")
print(f"Entities: {d.get('entities_extracted')}")
print(f"Relationships: {d.get('relationships_found')}")
print(f"Chunks: {d.get('chunks_created')}")
```

```
if d.get("entities"):
    print("\nExtracted Entities:")
    for e in d["entities"]:
        print(f"    [{e['type']}:20s] {e['value']}:30s ({e['confidence']:.0%})")
```



```
if d.get("relationships"):
    print(f"\nRelationships ({len(d['relationships'])}):")
    for r in d["relationships"][:5]:
        print(f"  {r['source']} -[{r['relation']}]> {r['target']}")
```